

Diva: Range Filter for Var-Length Keys and Queries



Navid Eslami
Ioana O. Bercea
Niv Dayan





**University of
Toronto**



North East



til 2019

Diva: Range Filter for Var-Length Keys and Queries



Diva: Range Filter for Var-Length Keys and Queries



Navid Eslami

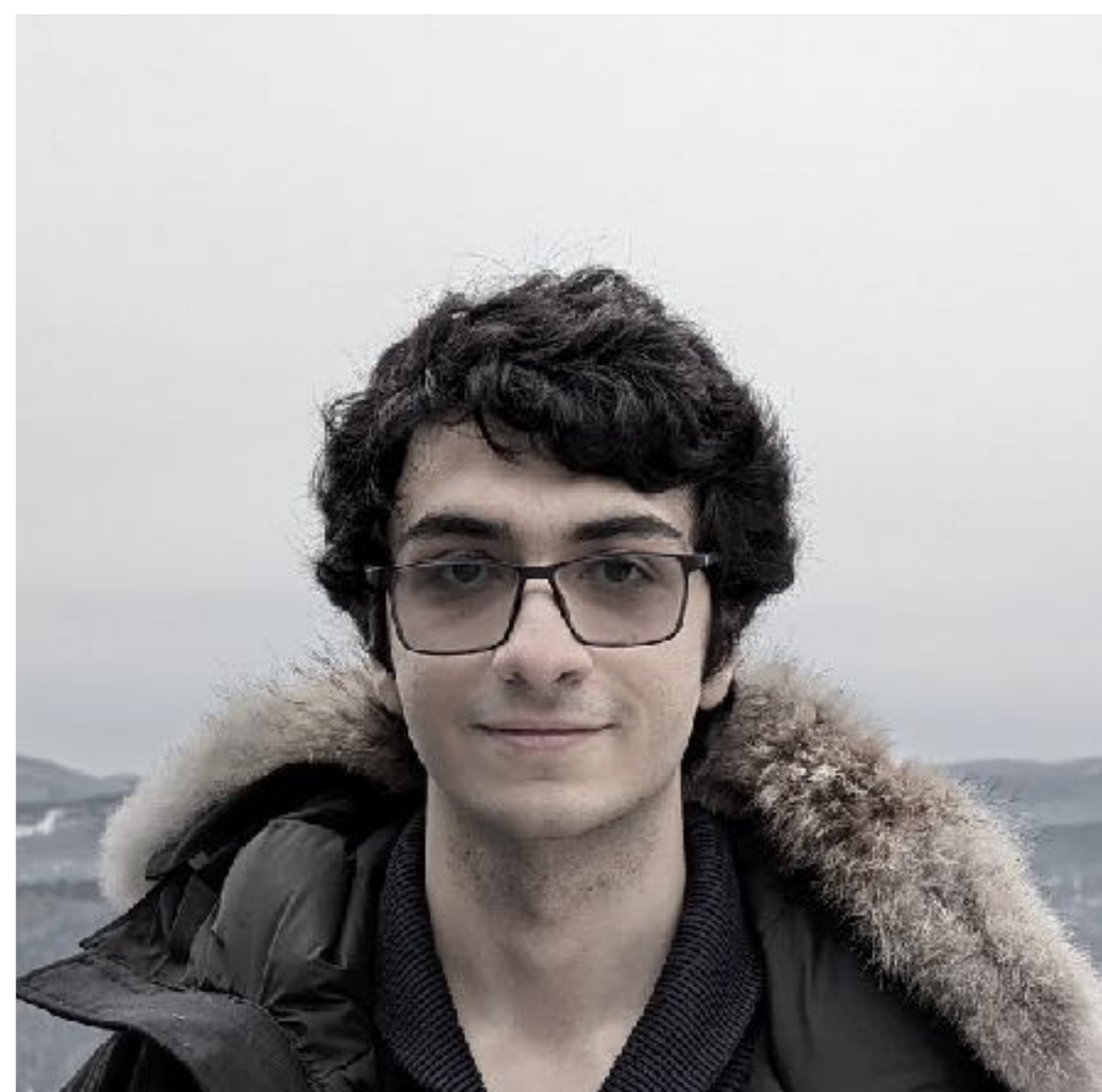
Uni of Toronto



Ioana O. Bercea

KTH, Sweden

VLDB'25 Best Paper



Navid Eslami

Uni of Toronto



Ioana O. Bercea

KTH, Sweden

VLDB'25 Best Paper



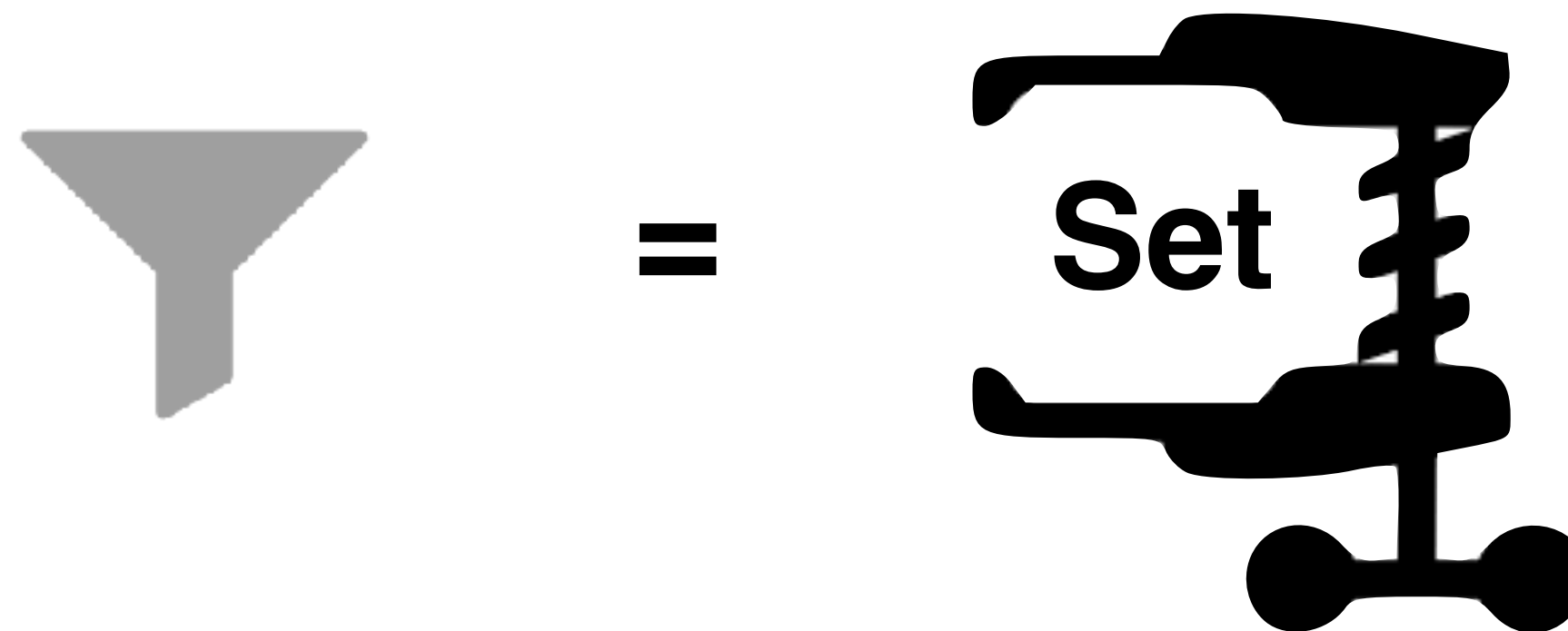
Navid Eslami

Uni of Toronto

What is a Filter?



What is a Filter?



What is a Filter?

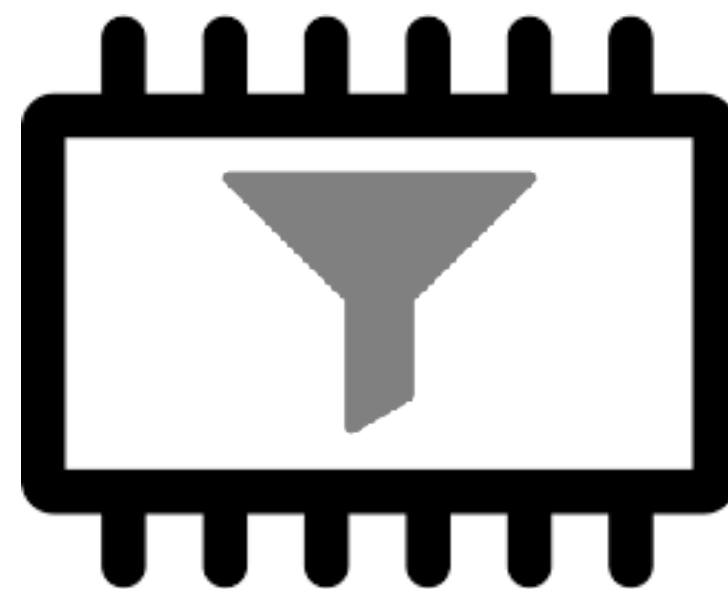




Does key
X exist



Memory



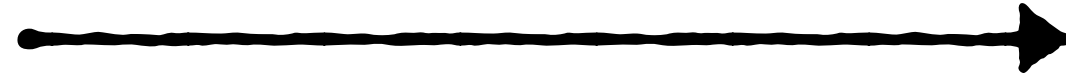
Data



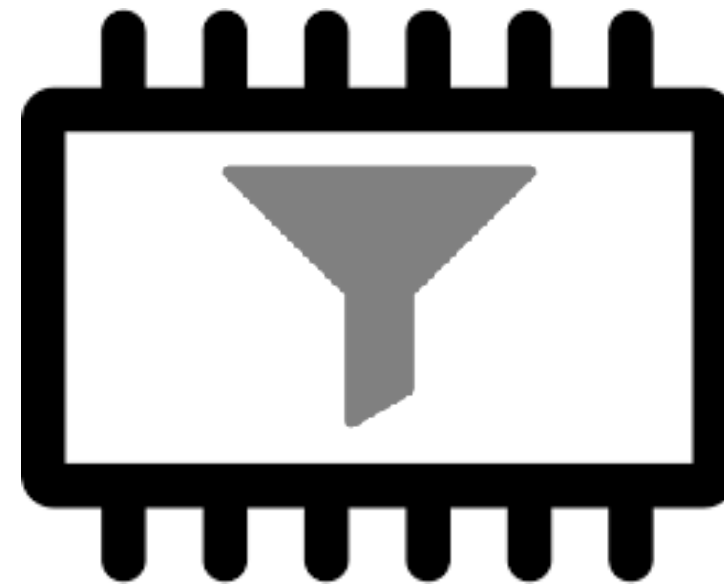
If key X exists



Does key
X exist



Memory



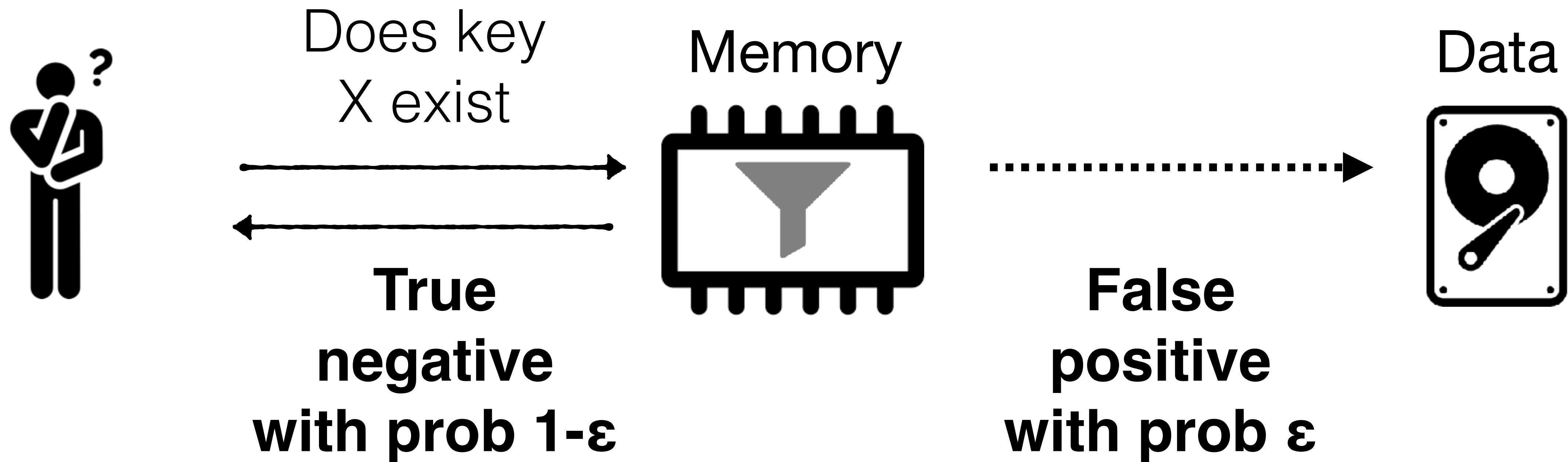
**True
positive**

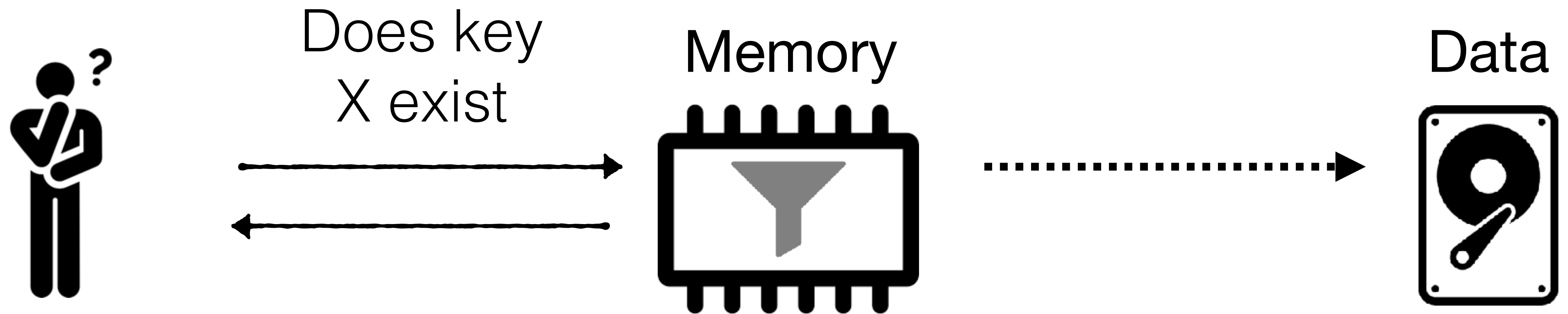


Data



If key X doesn't exist





Saves storage accesses & network hops

Only support point queries (to one key)

Does X exist?



Set



Only support point queries (to one key)

How about a range?

Does X exist?

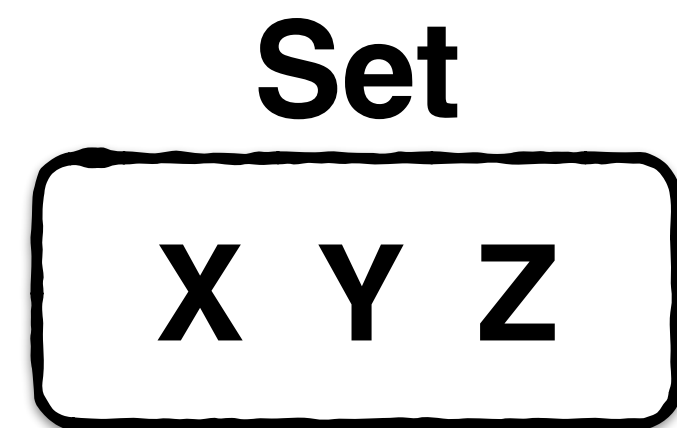


Set



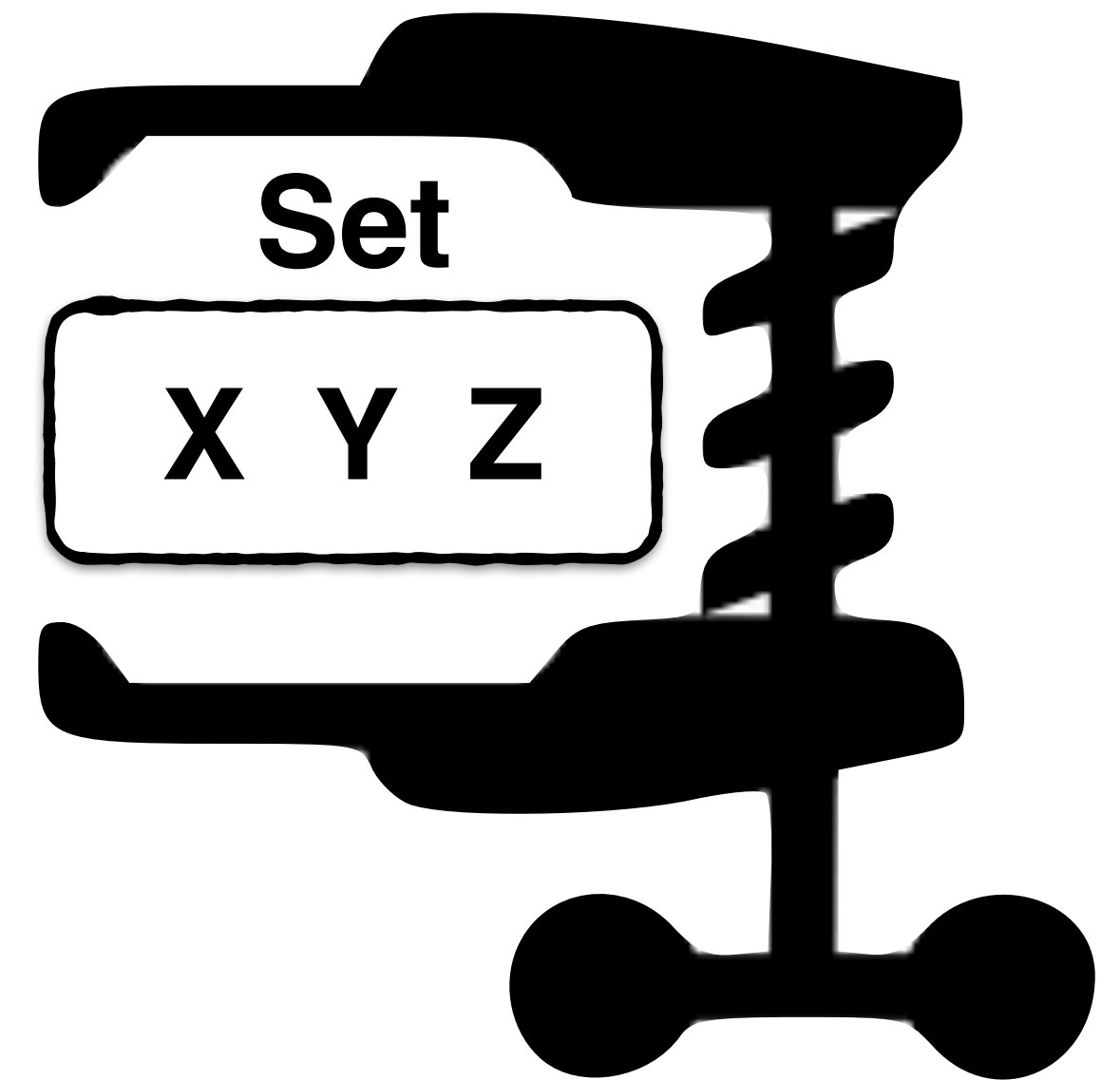
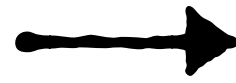
How about a range?

**Does anything in
[A, B] exist?**

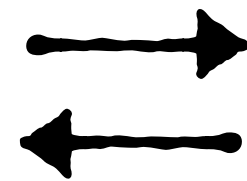


How about a range?

**Does anything in
[A, B] exist?**

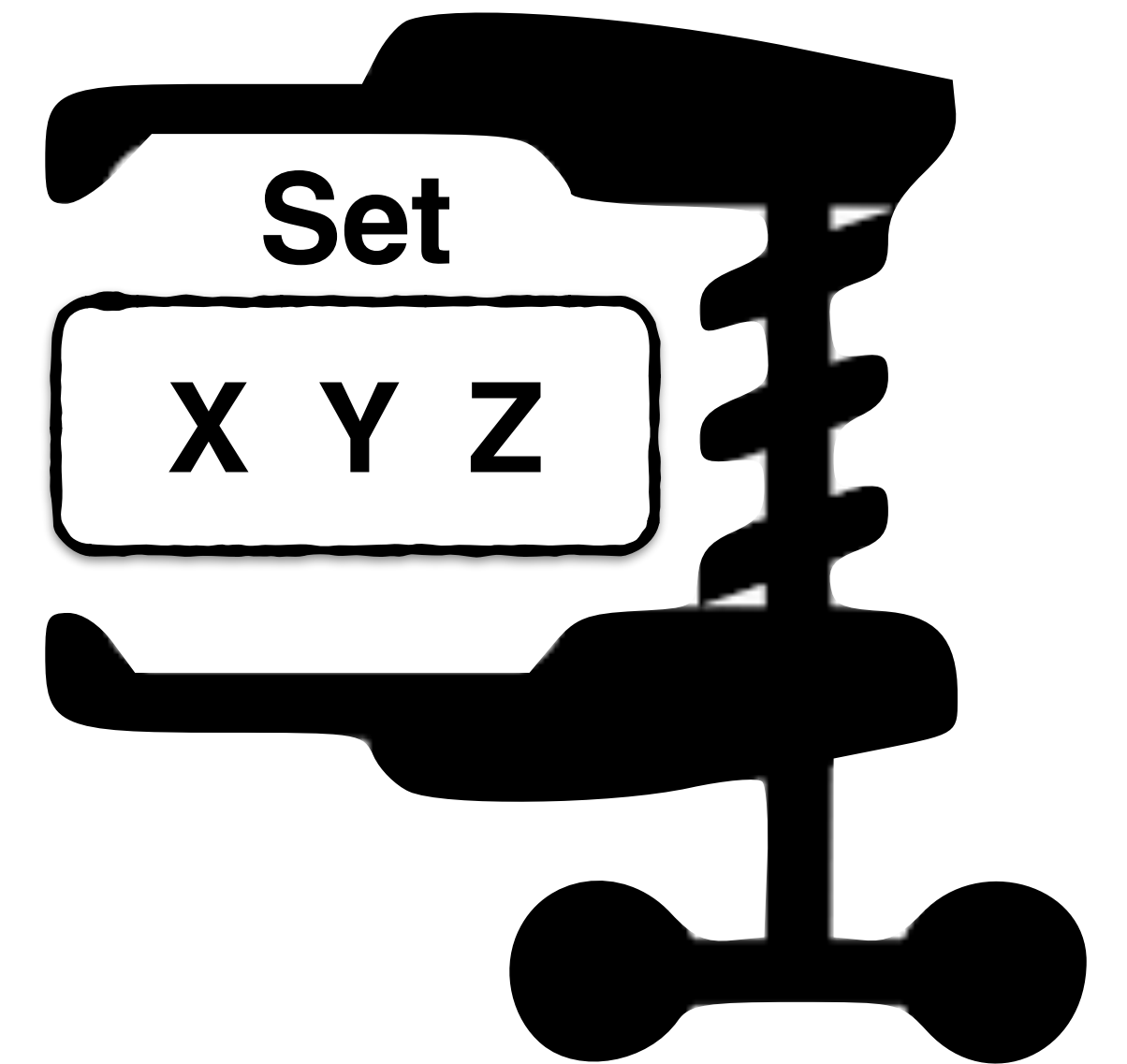


Does anything in
[A, B] exist?

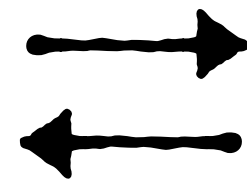


**True negative
with prob $1-\epsilon$**

**False positive
with prob ϵ**

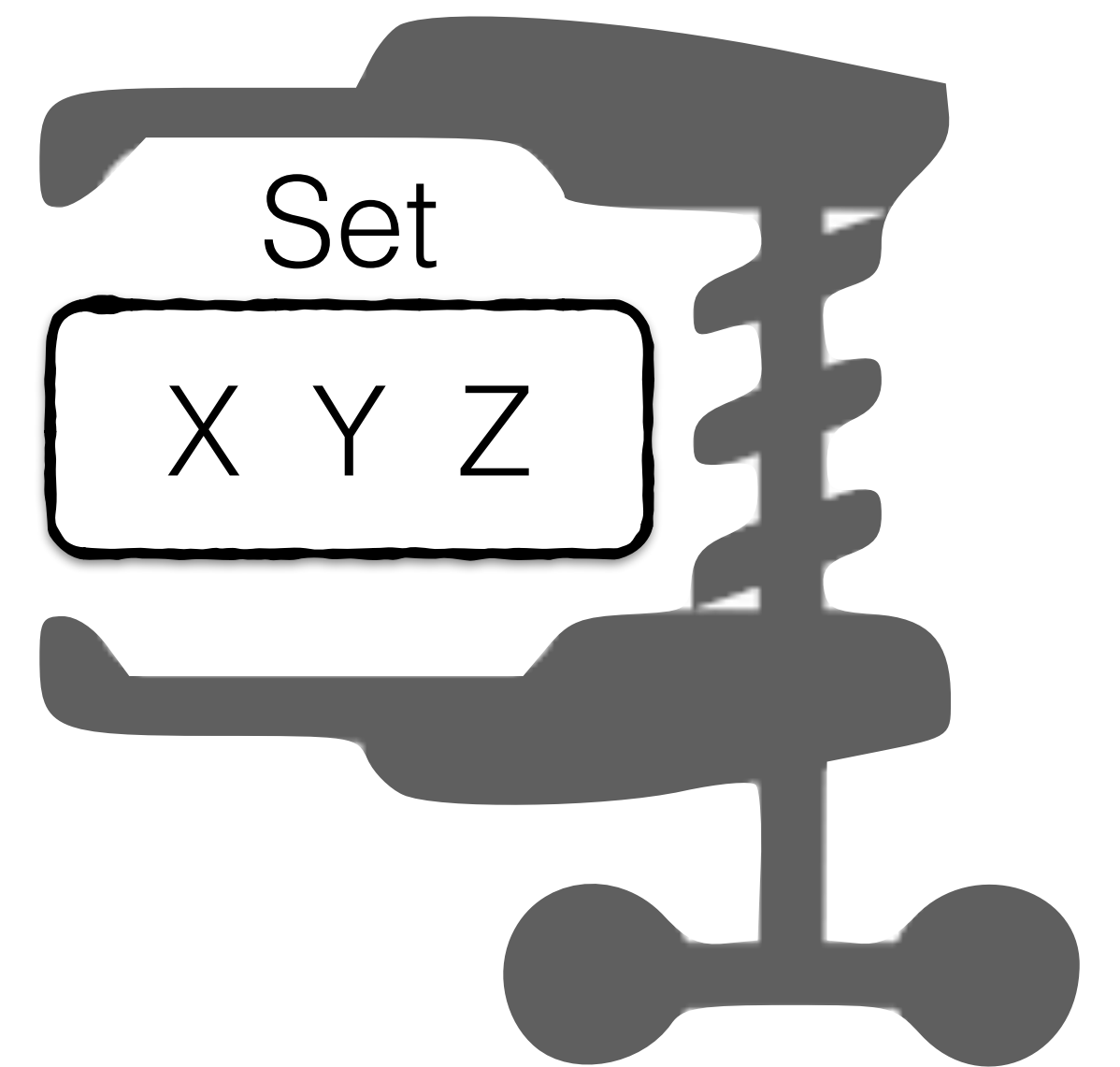


Does anything in
[A, B] exist?



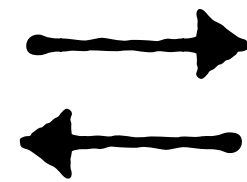
True negative
with prob $1-\epsilon$

False positive
with prob ϵ



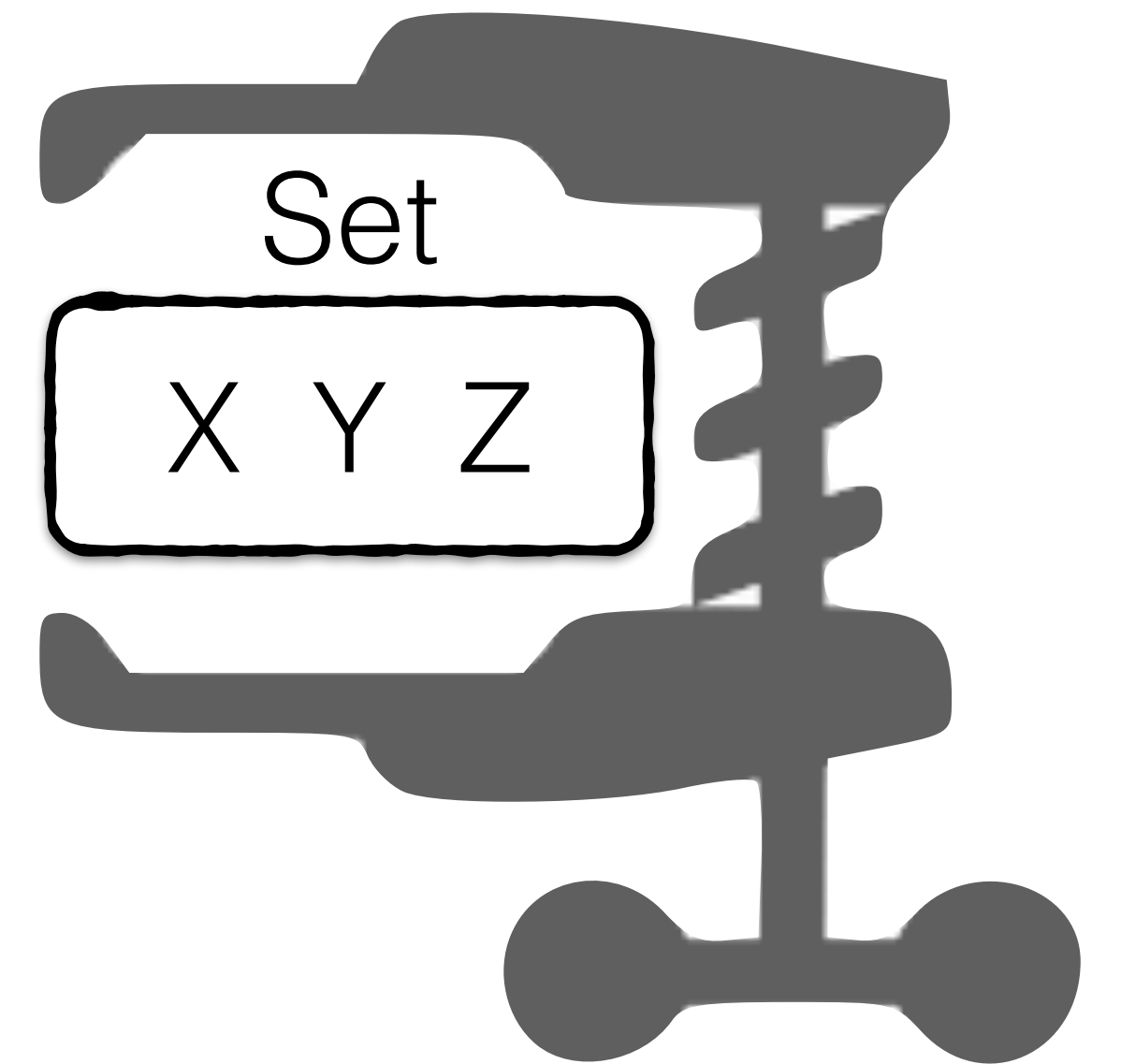
Applications?

Does anything in
[A, B] exist?



True negative
with prob $1-\epsilon$

False positive
with prob ϵ



Does anything in
[A, B] exist?

Filters



Memory

LSM-tree

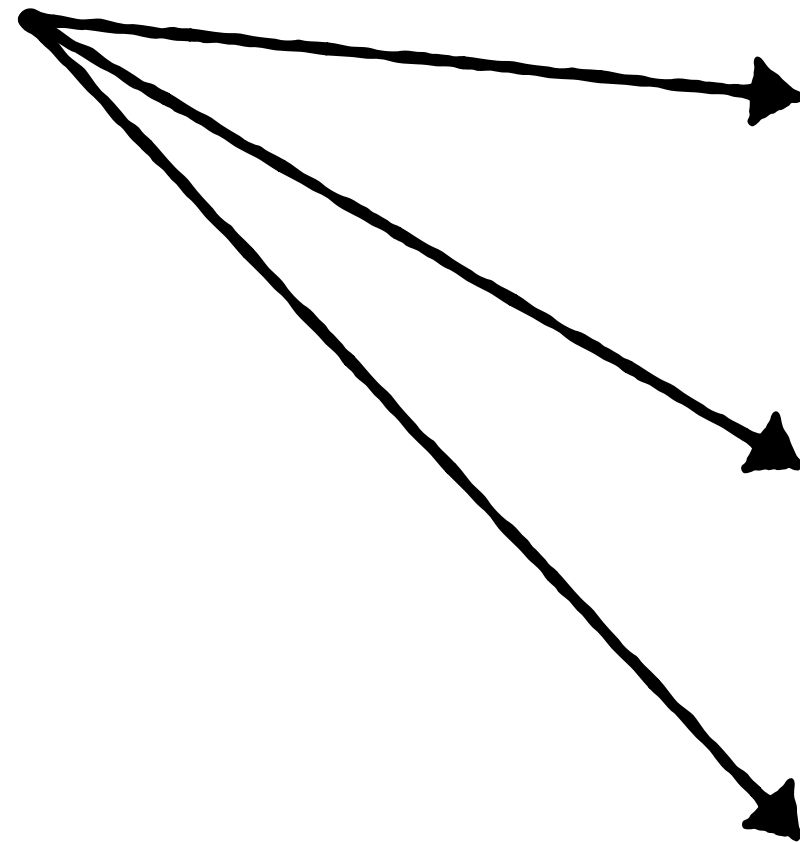
1 4 9

...

...

Storage

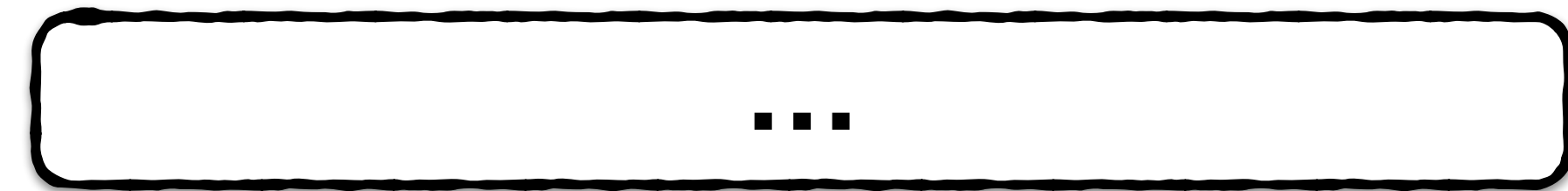
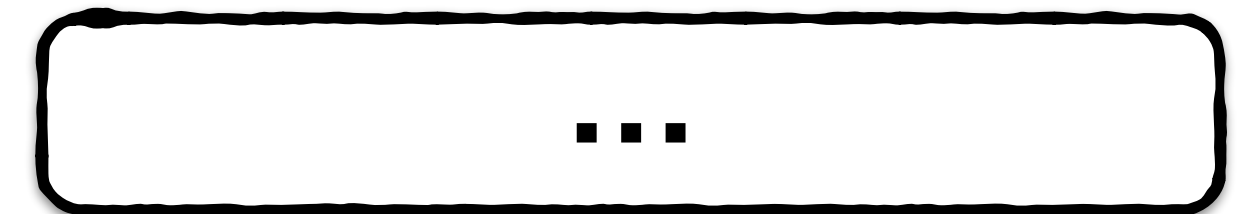
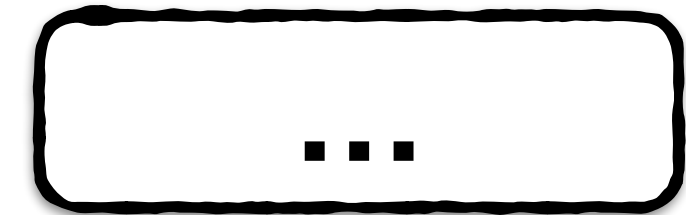
**Does anything in
[A, B] exist?**



Filters

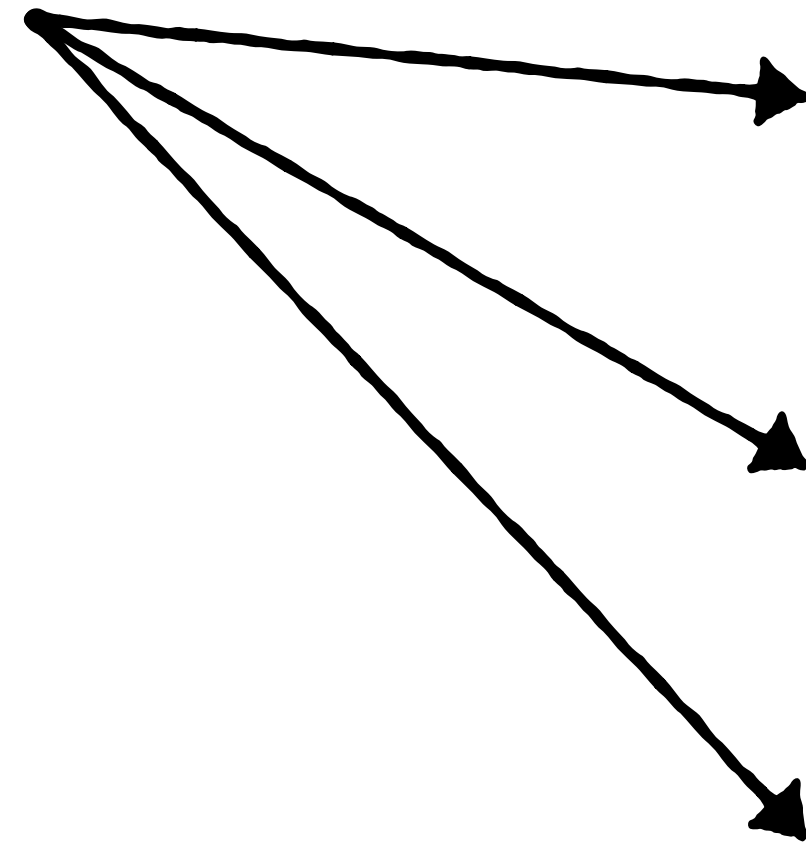


LSM-tree

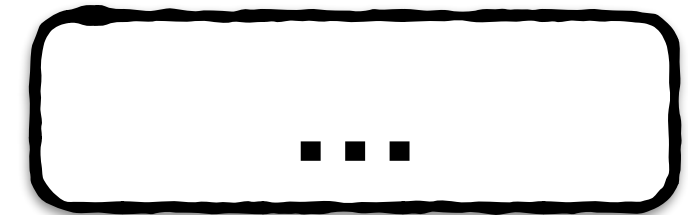


skip runs that don't contain relevant key range

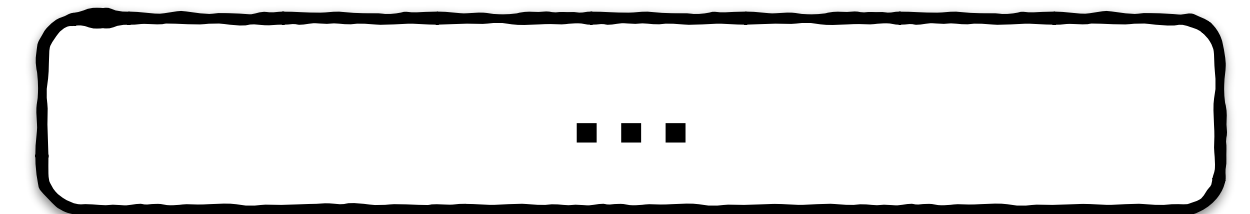
Does anything in
[A, B] exist?



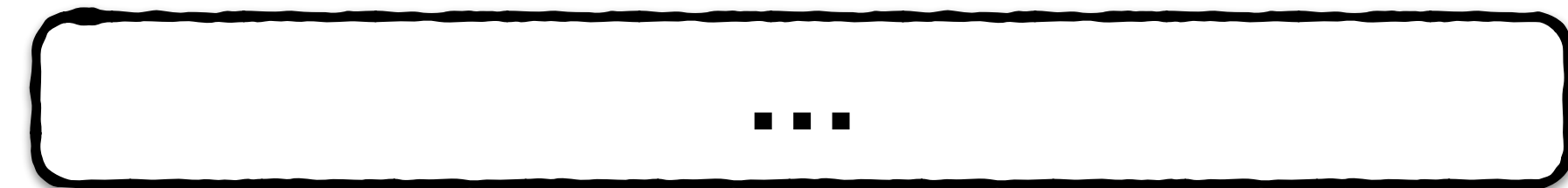
Skip



Skip

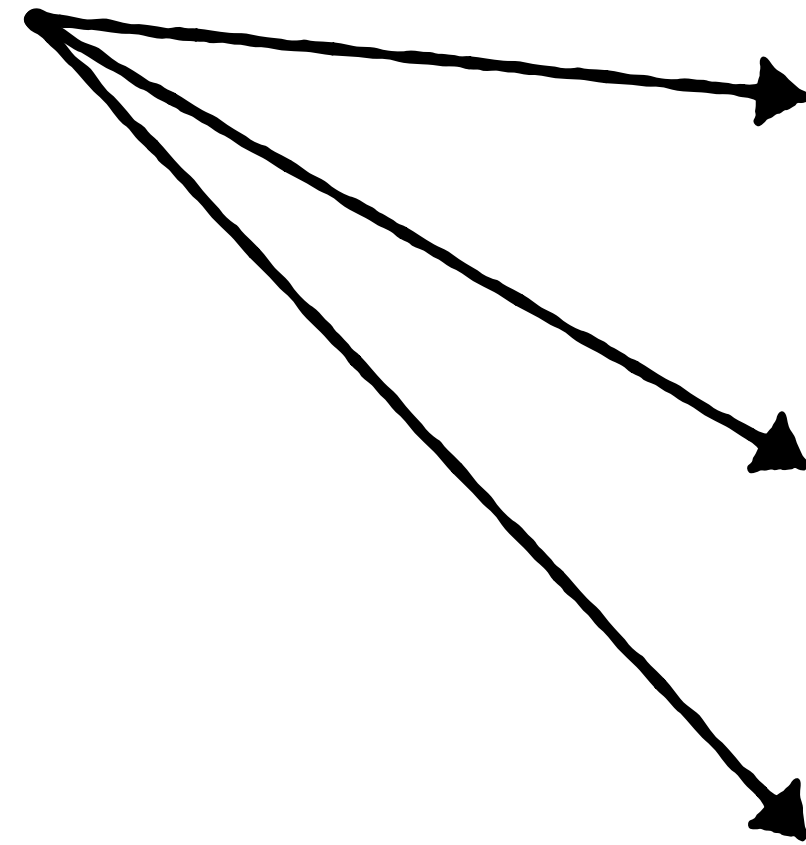


Skip



How to implement a range filter?

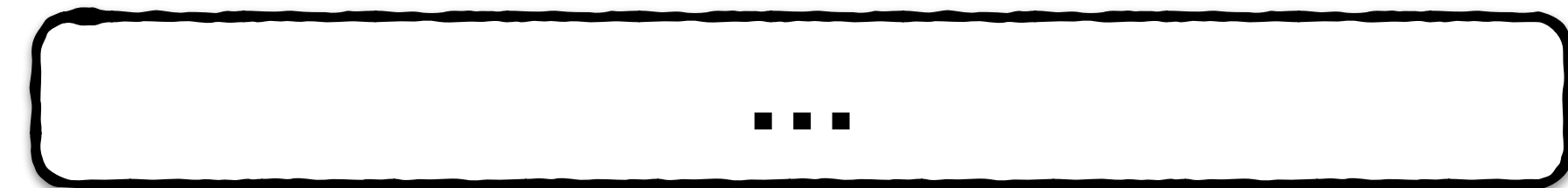
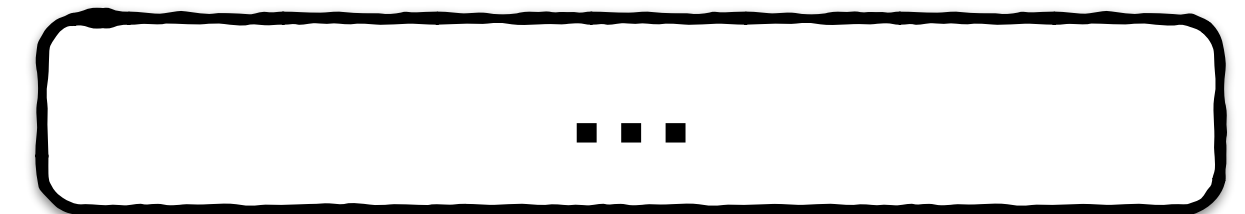
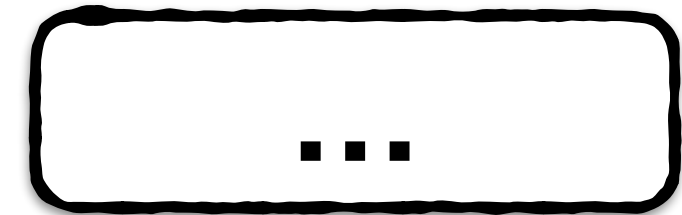
Does anything in
[A, B] exist?



Skip

Skip

Skip



Does anything in
[A, B] exist?



Bloom



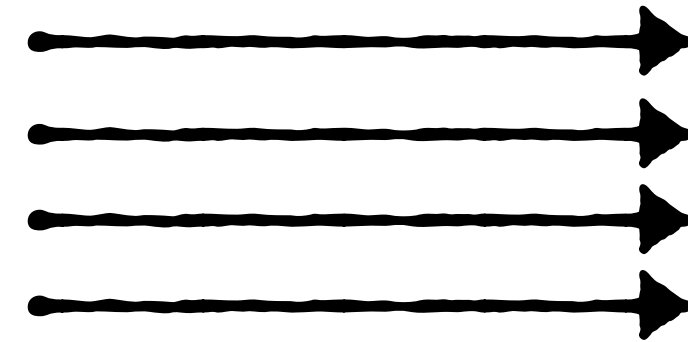
Set



Query every key in range

Does anything in
[A, B] exist?

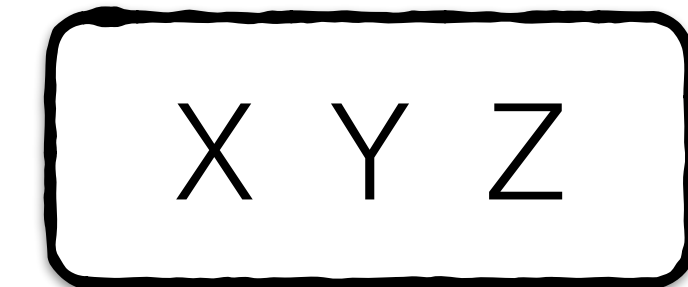
R sub-queries



$$R = B - A$$



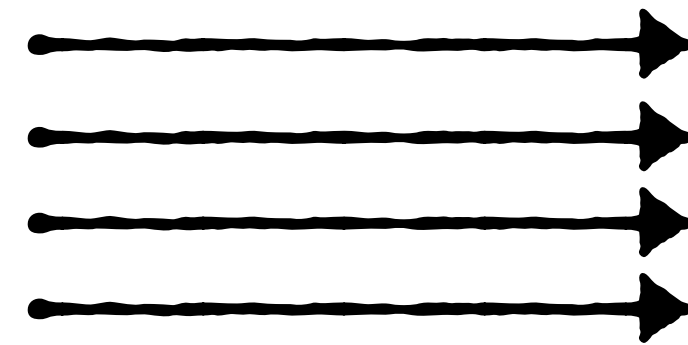
Set



Query every key in range

Does anything in
[A, B] exist?

R sub-queries



$$R = B - A$$



Set

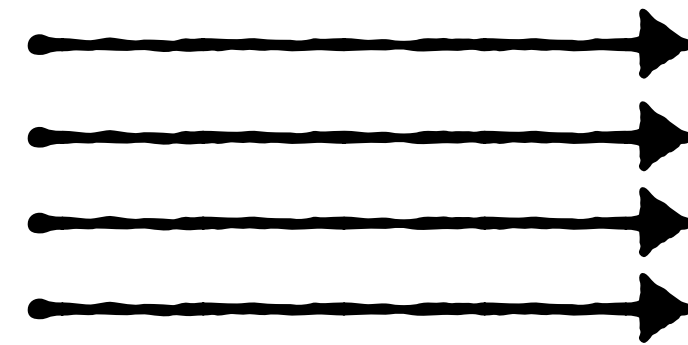


If all return negative, range is empty

Query every key in range

Does anything in
[A, B] exist?

R sub-queries



$$R = B - A$$



Set

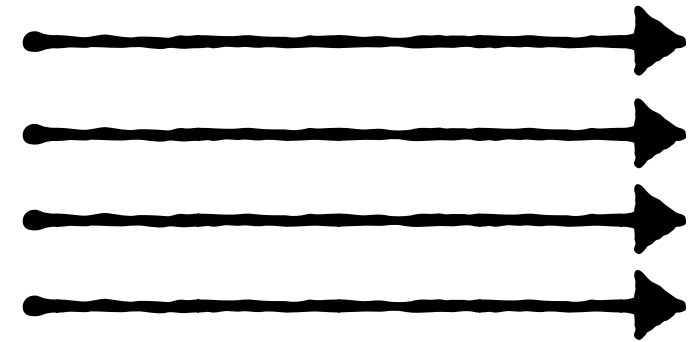


If all return negative, range is empty

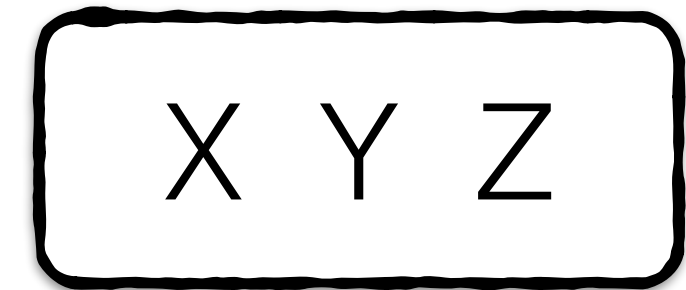
If at least one returns a positive, range may be non-empty

Query every key in range

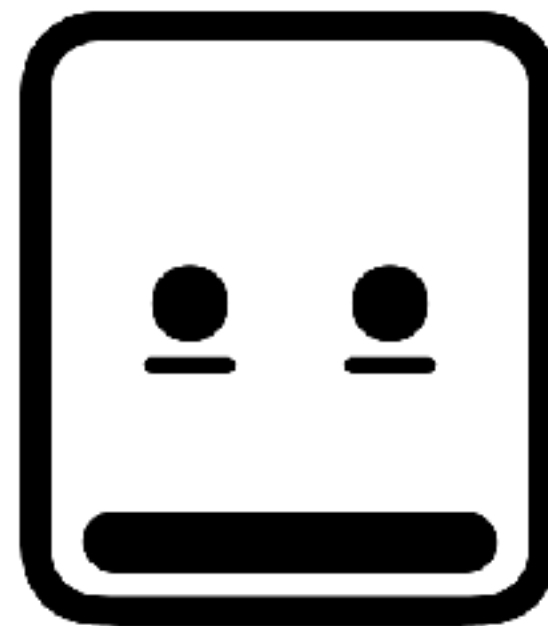
R sub-queries



Set



Does anything in
[A, B] exist?

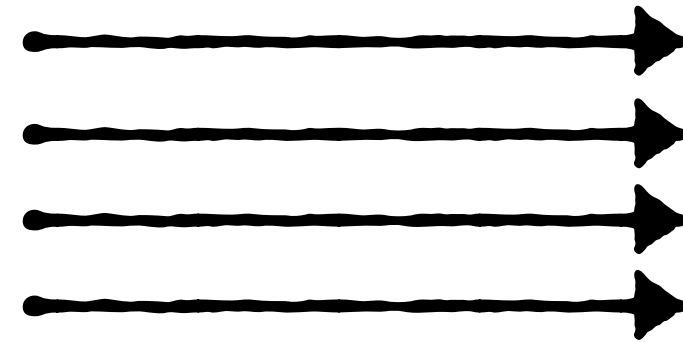


Problems...

Problem 1: Query Cost

Does anything in
[A, B] exist?

$O(R)$

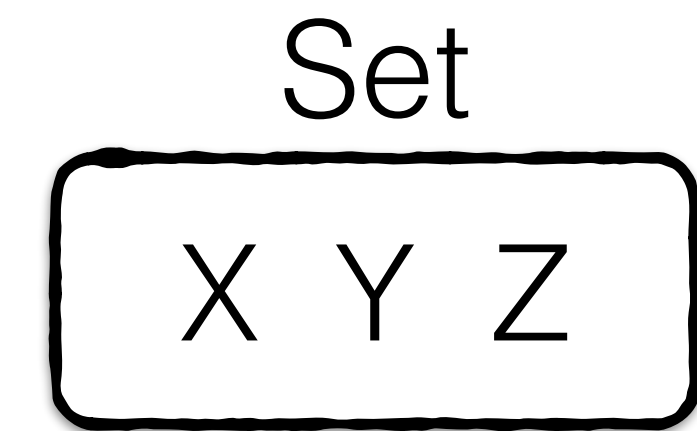
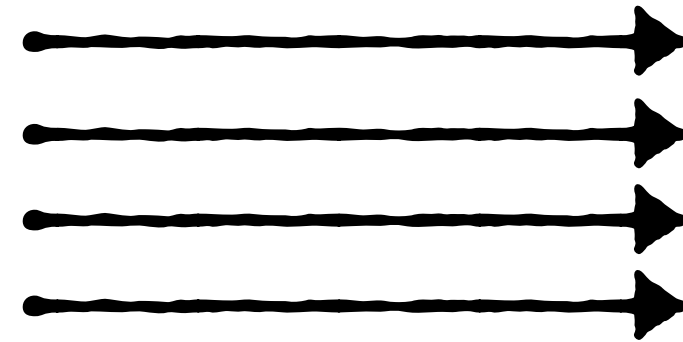


Set



Problem 2: Higher False Positive Rate

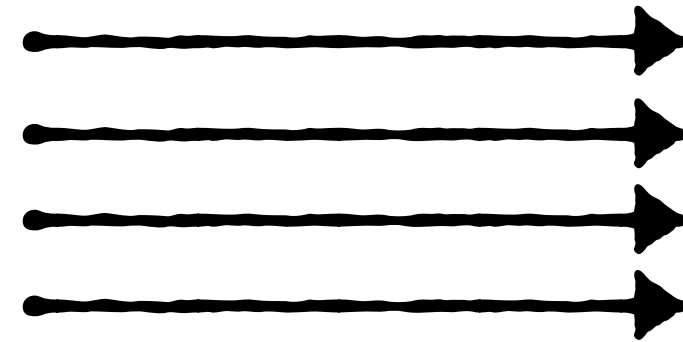
Does anything in
[A, B] exist?



Problem 2: Higher False Positive Rate

Does anything in
[A, B] exist?

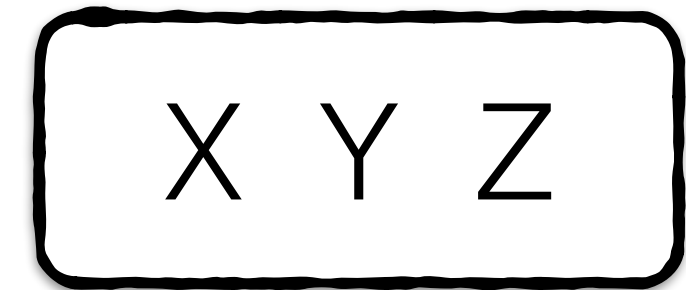
R sub-queries



FPR ϵ

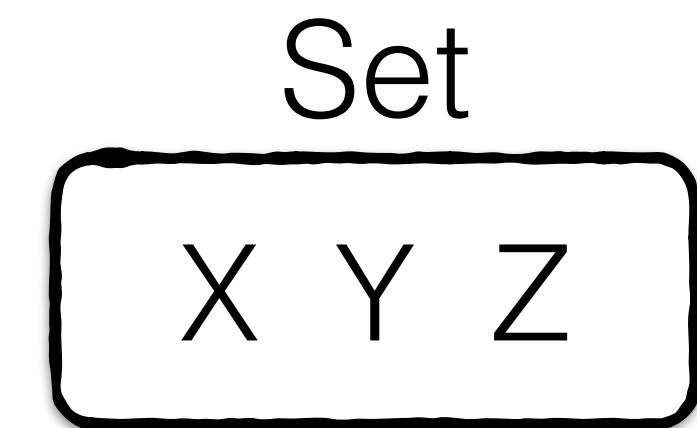
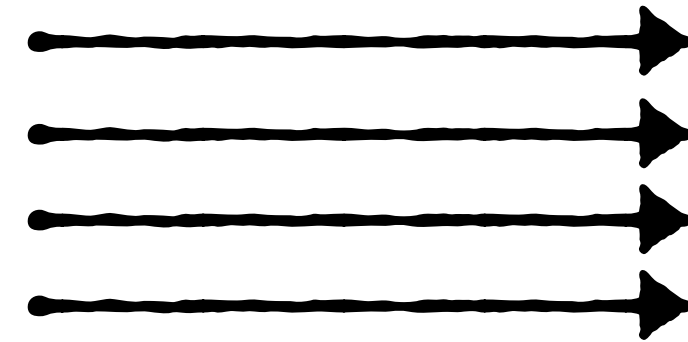


Set



Problem 2: Higher False Positive Rate

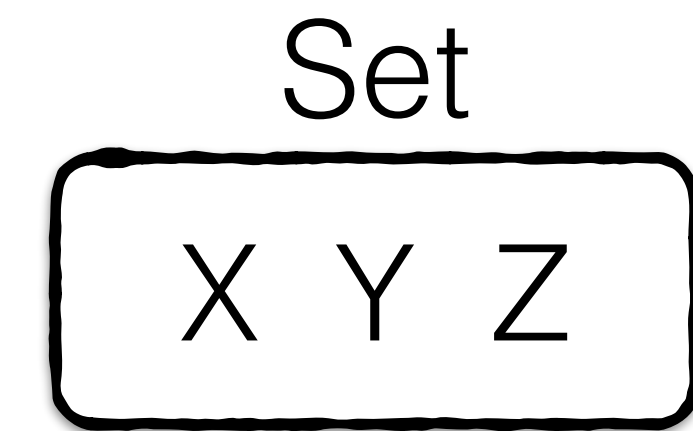
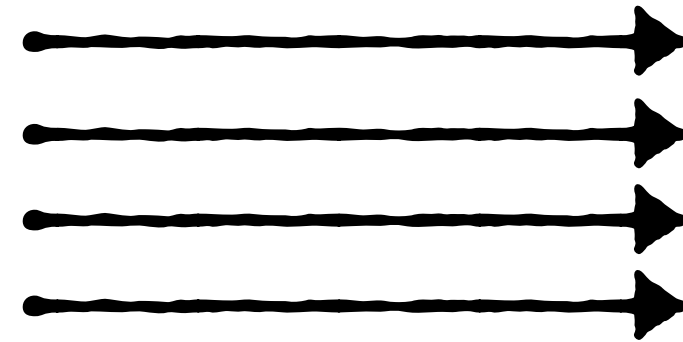
Does anything in
[A, B] exist?



$$\text{false positive rate} \approx R \cdot \epsilon$$

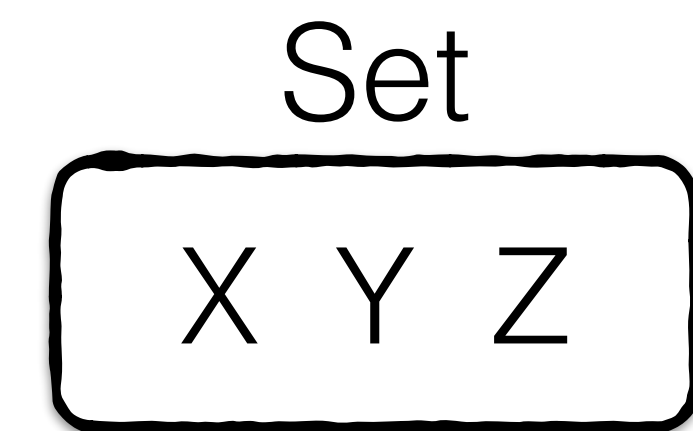
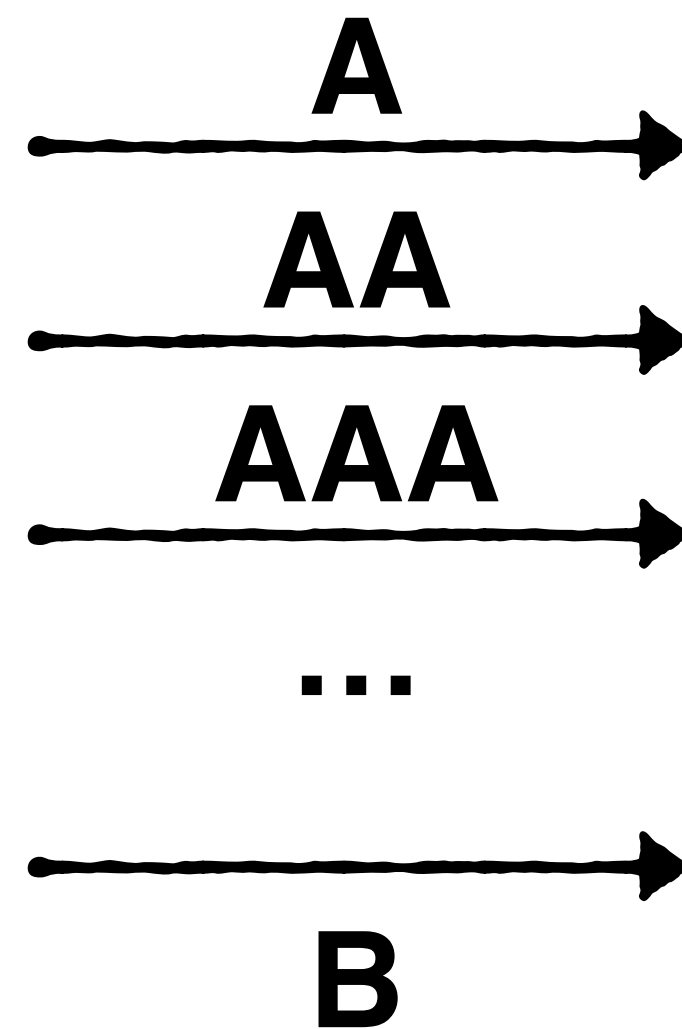
Problem 3: Does not work for infinite universe (e.g., strings)

Does anything in
[A, B] exist?



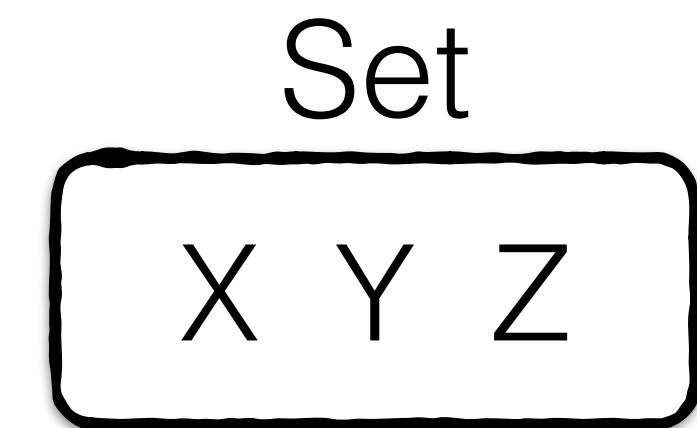
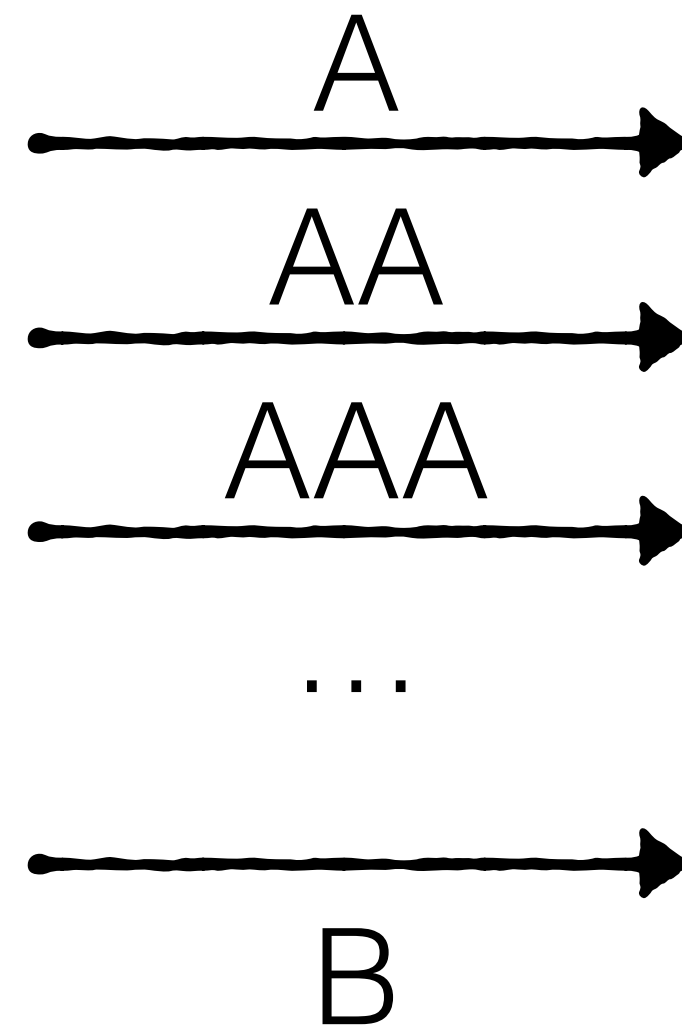
Problem 3: Does not work for infinite universe (e.g., strings)

Does anything in
[A, B] exist?



Problem 3: Does not work for infinite universe (e.g., strings)

Does anything in
[A, B] exist?



There are infinitely many possible keys in a range :)

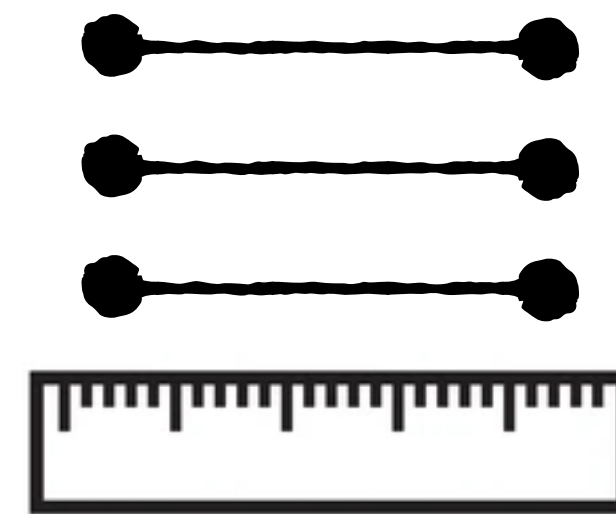
Requirements



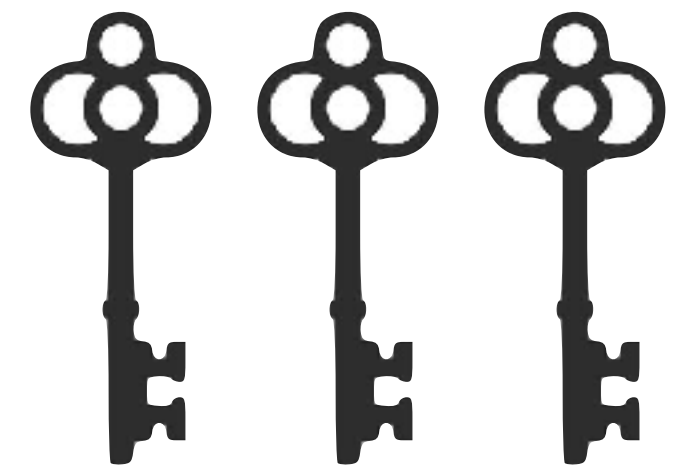
Query cost
 $O(R)$



FPR
 $O(\epsilon \cdot R)$



Bounded
queries



Fixed-length
keys



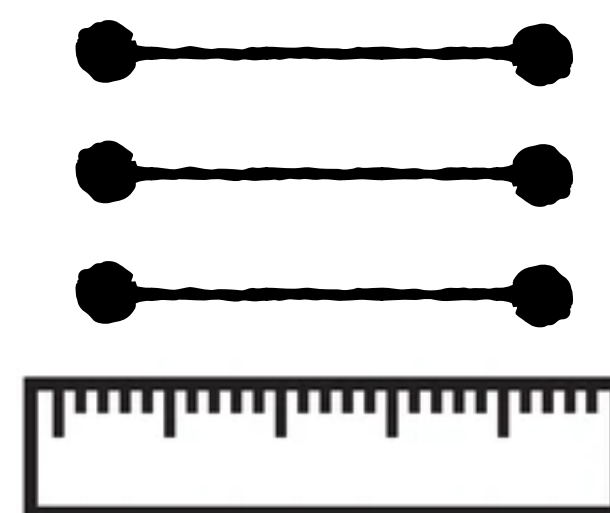
Query cost
 ~~$O(R)$~~



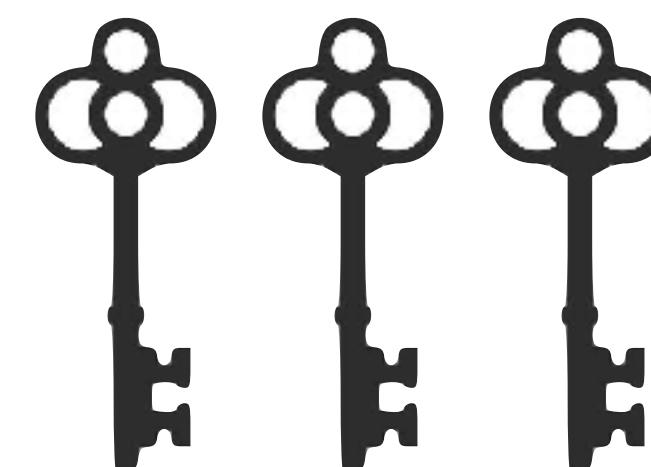
?



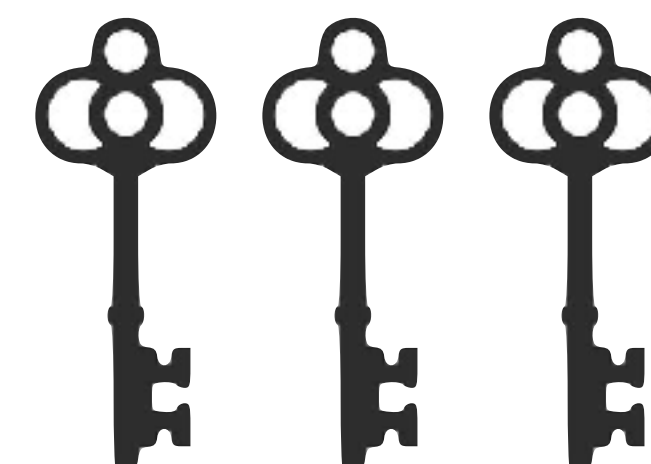
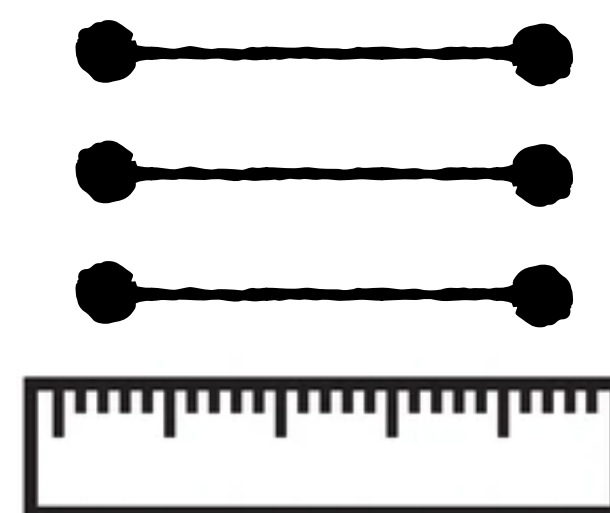
FPR
 $O(\varepsilon \cdot R)$



Bounded
queries



Fixed-length
keys



Query cost
 $O(1)$

FPR
 $O(\epsilon \cdot R)$

Bounded
queries

Fixed-length
keys



?

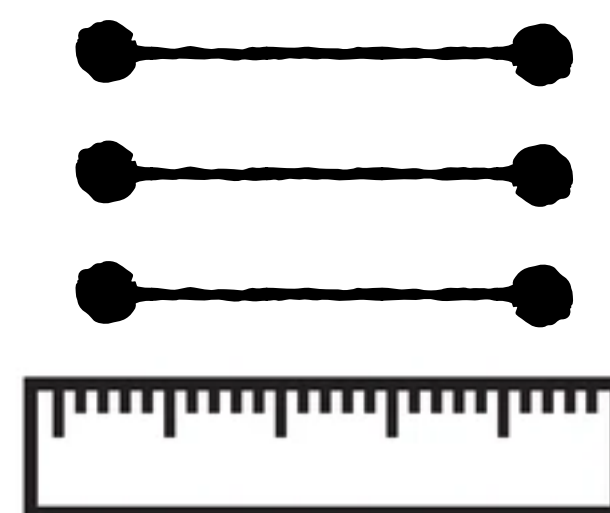


Query cost
 $O(1)$

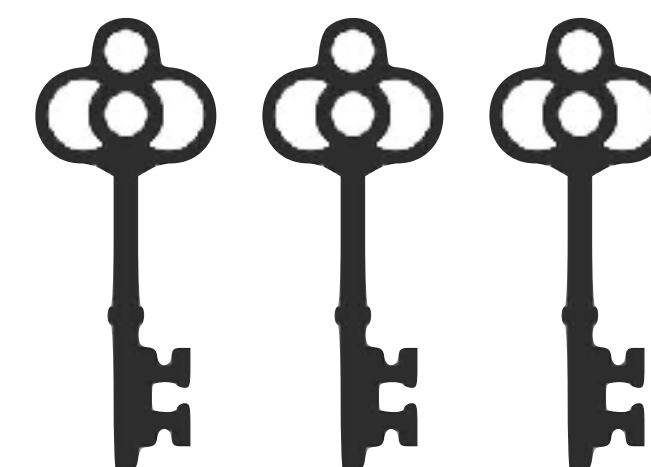


FPR
 $O(\epsilon \cdot \cancel{R})$

↑
?



Bounded
queries



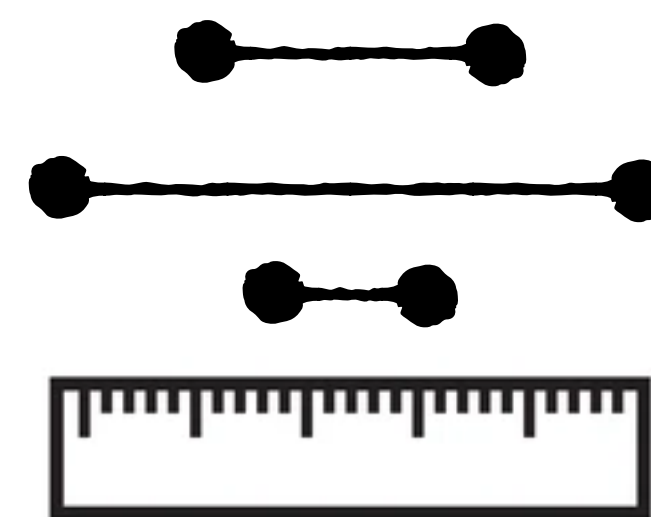
Fixed-length
keys



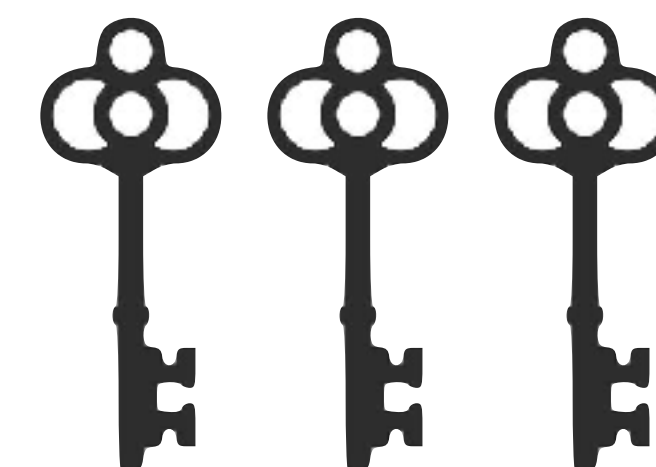
Query cost
 $O(1)$



FPR
 $O(\epsilon)$



**Var-length
queries**



Fixed-length
keys

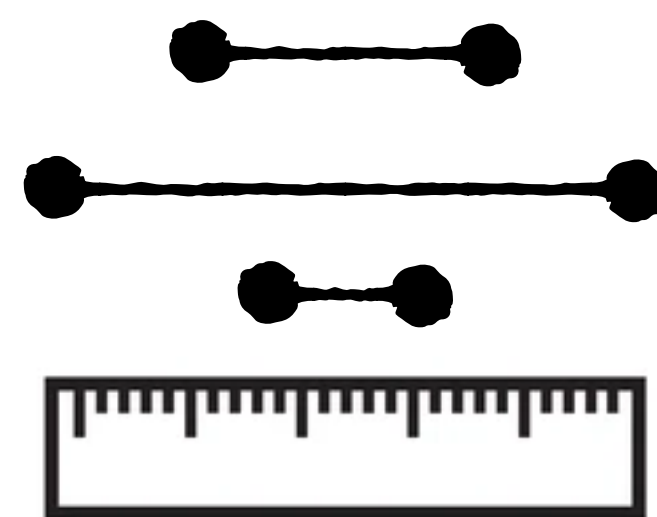




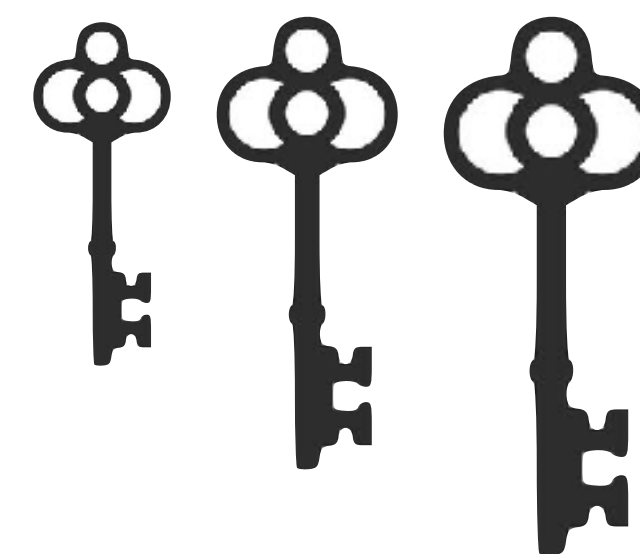
Query cost
 $O(1)$



FPR
 $O(\epsilon)$



Var-length
queries



**Var-length
keys**



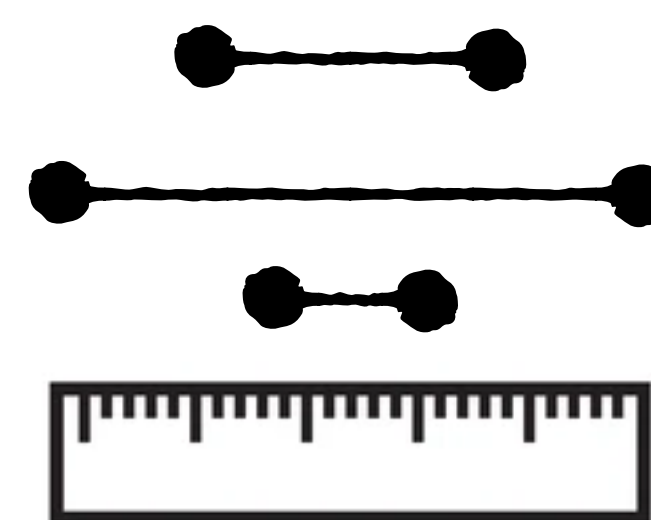
?



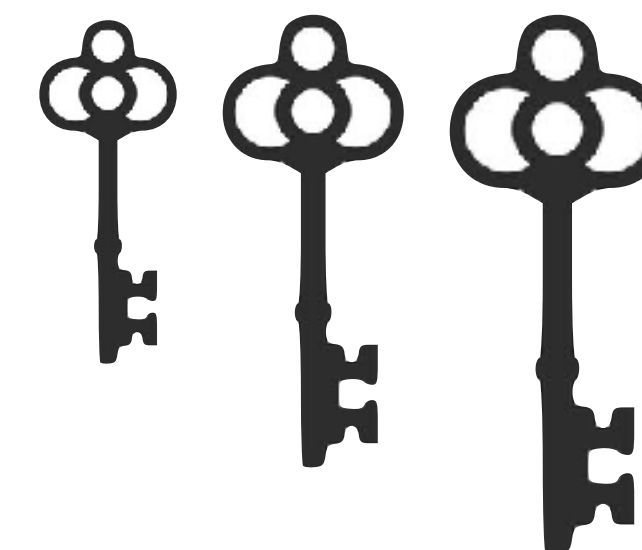
Query cost
 $O(1)$



FPR
 $O(\epsilon)$



Var-length
queries



Var-length
keys



+ Dynamic operations
(Inserts, deletes, expansions, contractions)

Hot research topic in past decade

SuRF

SIGMOD18

Rosetta

SIGMOD2020

SNARF

SIGMOD22

Proteus

SIGMOD22

REncoder

ICDE23

BloomRF

EDBT23

Grafite

SIGMOD24

Oasis

VLDB24

Memento

SIGMOD25

Hot research topic in past decade

SuRF
SIGMOD18

Rosetta
SIGMOD2020

SNARF
SIGMOD22

Proteus
SIGMOD22

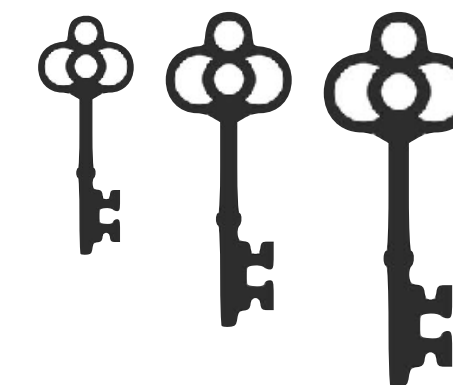
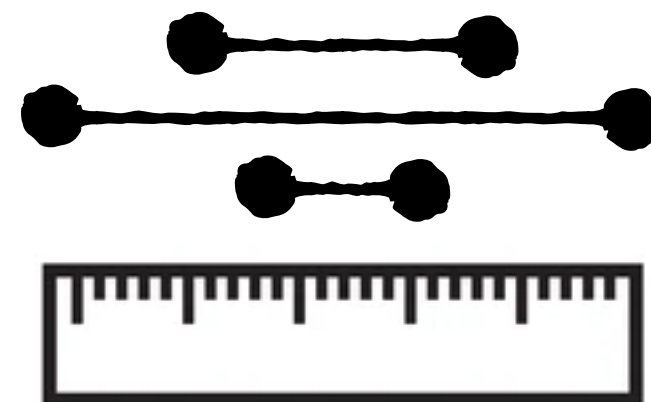
REncoder
ICDE23

BloomRF
EDBT23

Grafite
SIGMOD24

Oasis
VLDB24

Memento
SIGMOD25



SuRF
SIGMOD18

Rosetta
SIGMOD2020

SNARF
SIGMOD22

Proteus
SIGMOD22

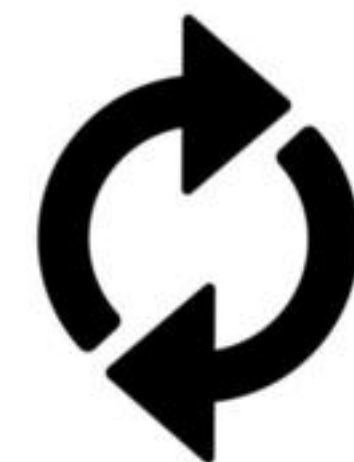
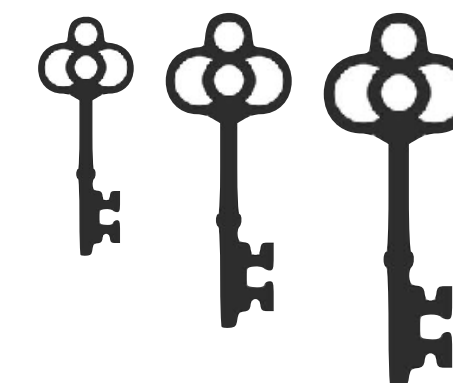
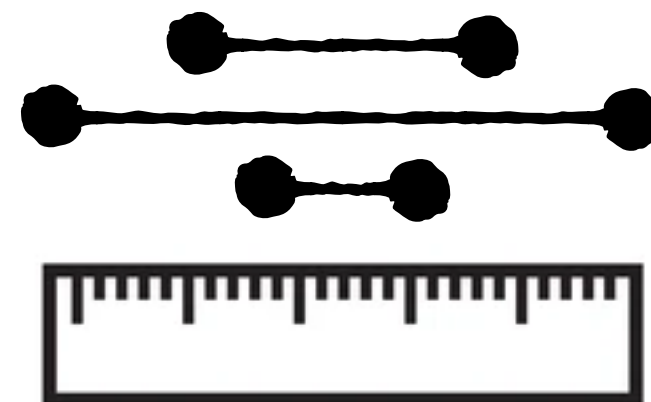
REncoder
ICDE23

BloomRF
EDBT23

Grafite
SIGMOD24

Oasis
VLDB24

Memento
SIGMOD25



SuRF
SIGMOD18

Rosetta
SIGMOD2020

SNARF
SIGMOD22

Proteus
SIGMOD22

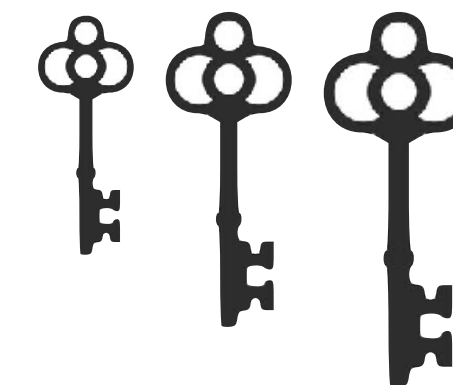
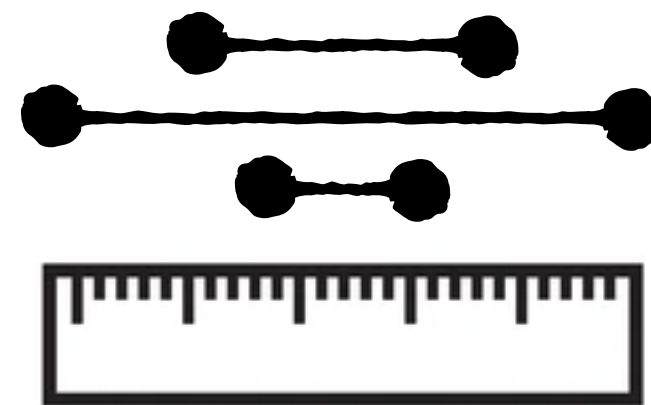
REncoder
ICDE23

BloomRF
EDBT23

Grafite
SIGMOD24

Oasis
VLDB24

Memento
SIGMOD25



SuRF
SIGMOD18

Rosetta
SIGMOD2020

SNARF
SIGMOD22

Proteus
SIGMOD22

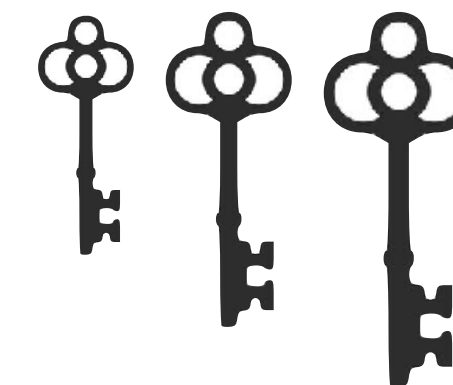
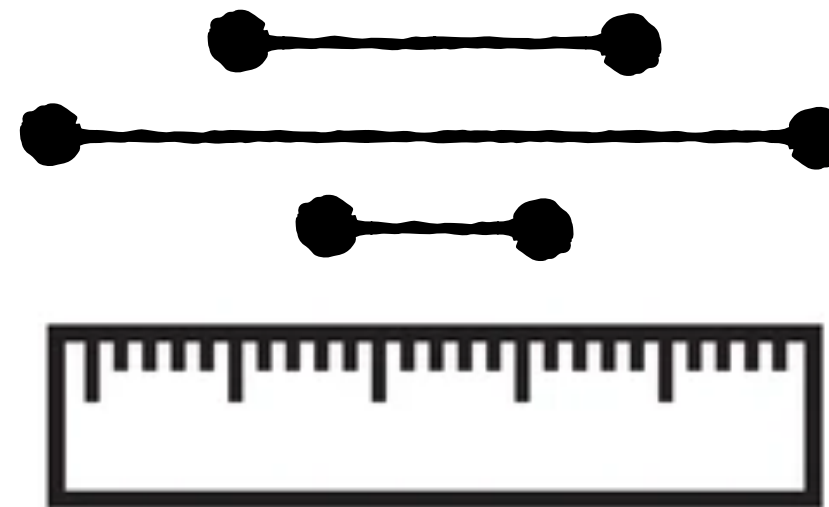
REncoder
ICDE23

BloomRF
EDBT23

Grafite
SIGMOD24

Oasis
VLDB24

Memento
SIGMOD25



SuRF
SIGMOD18

Rosetta
SIGMOD2020

SNARF
SIGMOD22

Proteus
SIGMOD22

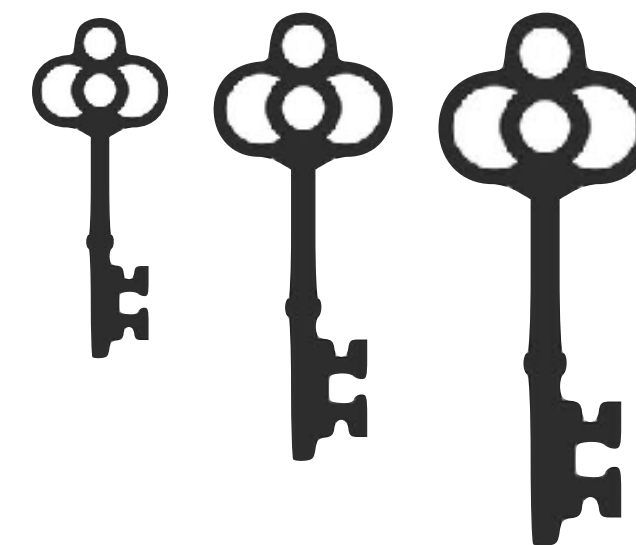
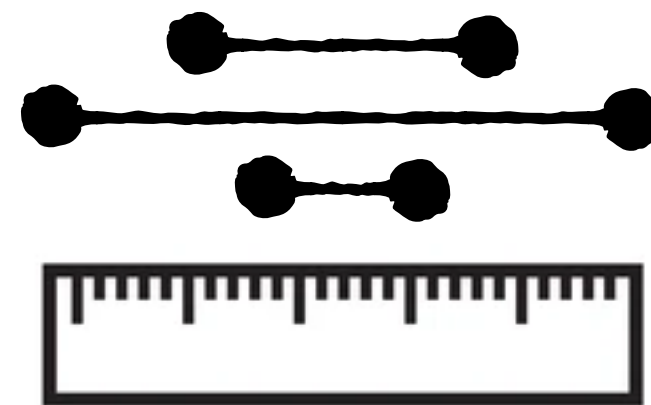
REncoder
ICDE23

BloomRF
EDBT23

Grafite
SIGMOD24

Oasis
VLDB24

Memento
SIGMOD25



SuRF
SIGMOD18

Rosetta
SIGMOD2020

SNARF
SIGMOD22

Proteus
SIGMOD22

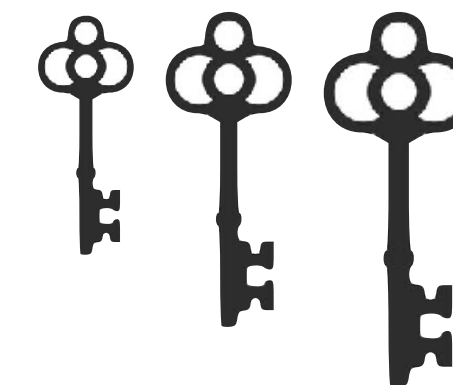
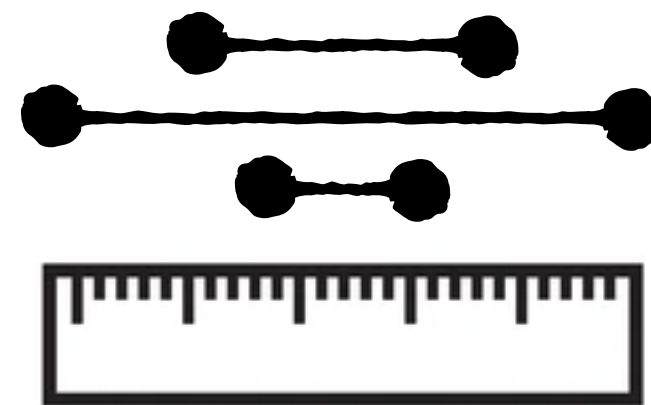
REncoder
ICDE23

BloomRF
EDBT23

Grafite
SIGMOD24

Oasis
VLDB24

Memento
SIGMOD25



SuRF	Rosetta	SNARF	Proteus
SIGMOD18	SIGMOD2020	SIGMOD22	SIGMOD22

REncoder	BloomRF	Grafite	Oasis	Memento
ICDE23	EDBT23	SIGMOD24	VLDB24	SIGMOD25

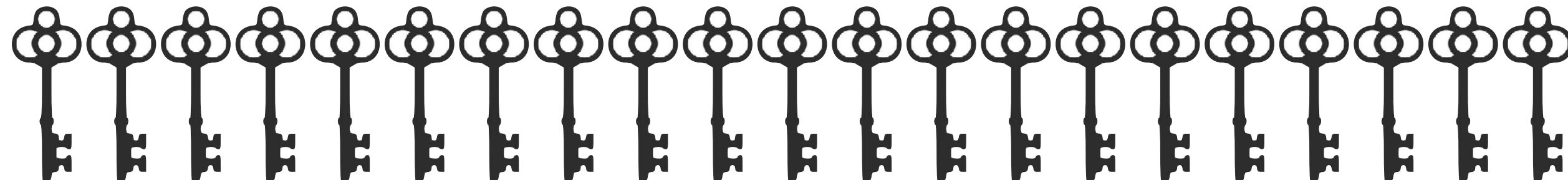
None achieve all goals simultaneously



Diva

A...

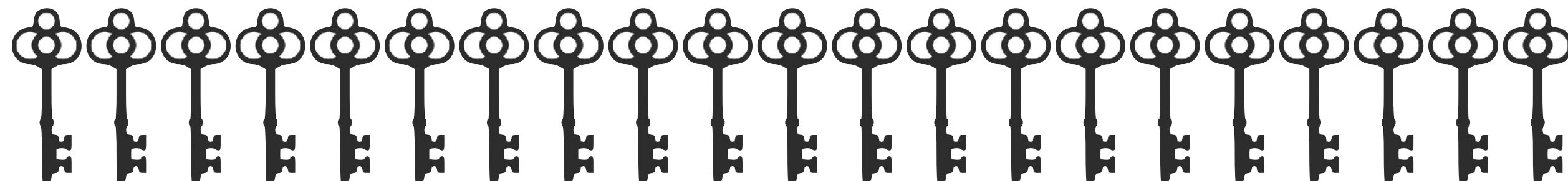
...Z



Store keys lexicographically

Query

A  B

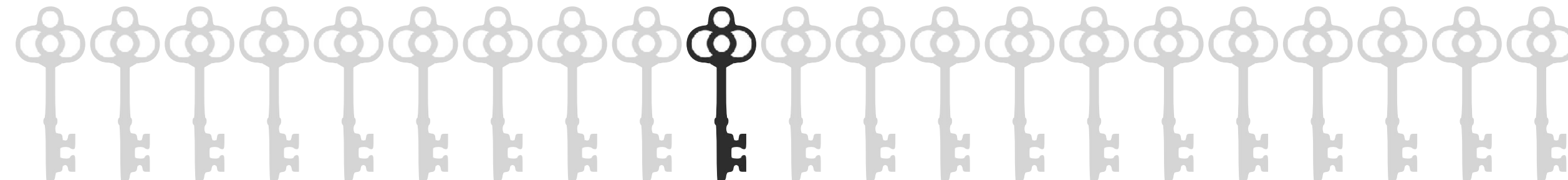


Store keys lexicographically

Query
A  **B**



Is successor(A) > B?

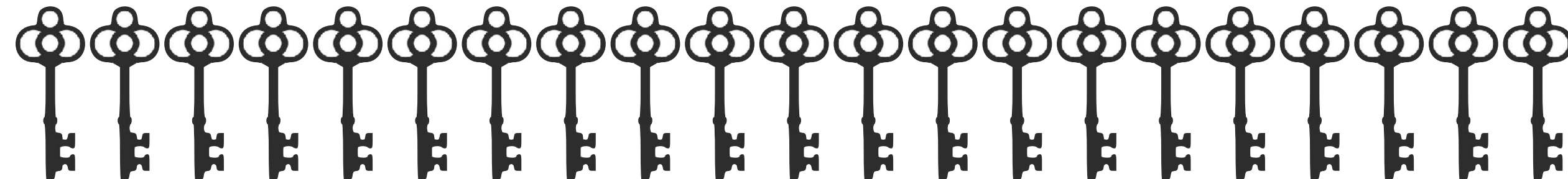


Store keys lexicographically

Query
A ————— B

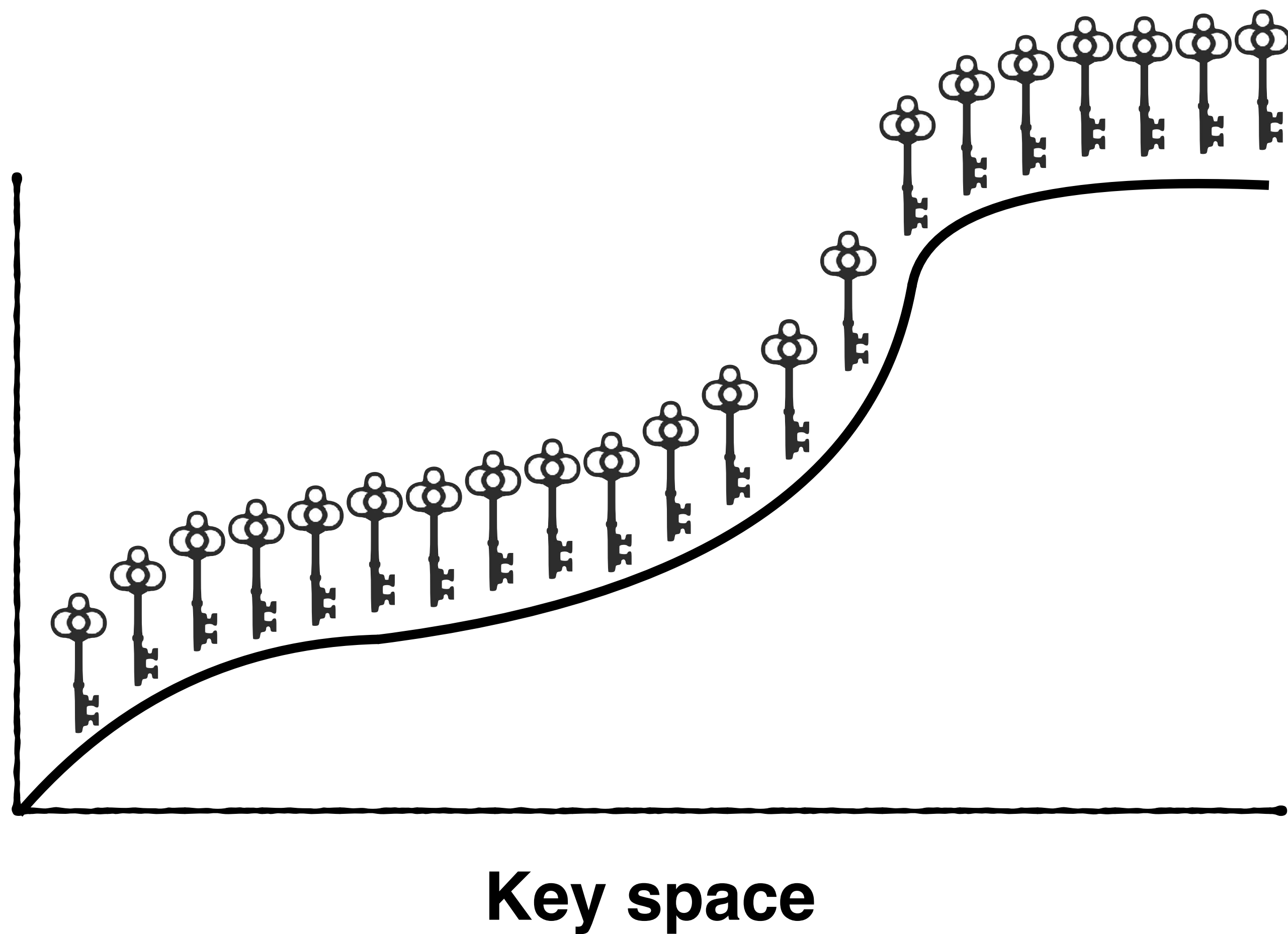


Is $\text{successor}(A) > B$?



**Requirement: support fast successor search
over compressed keys**

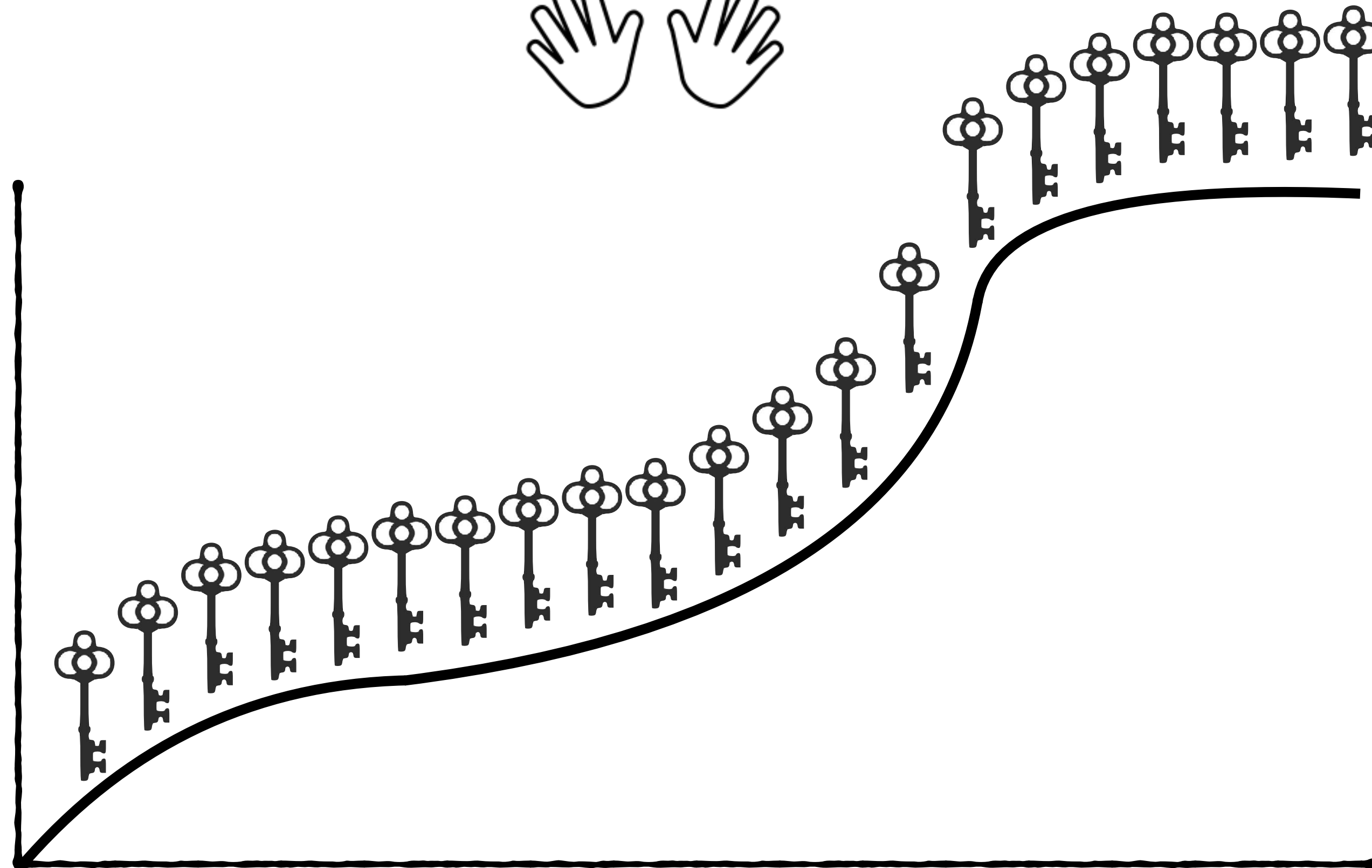
**Cumulative
Distribution Function
(CDF)**



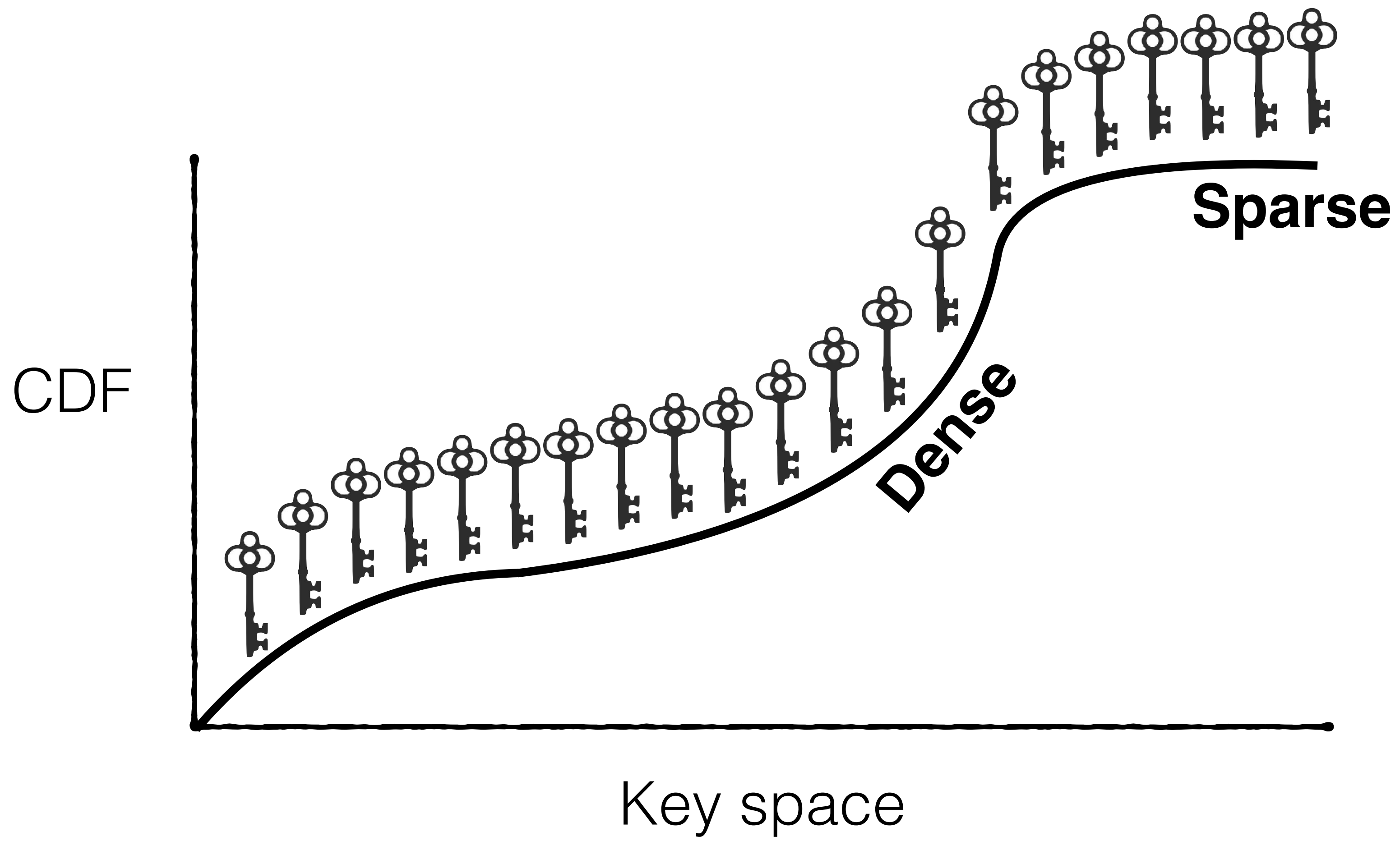
Inspiration from learned indexes

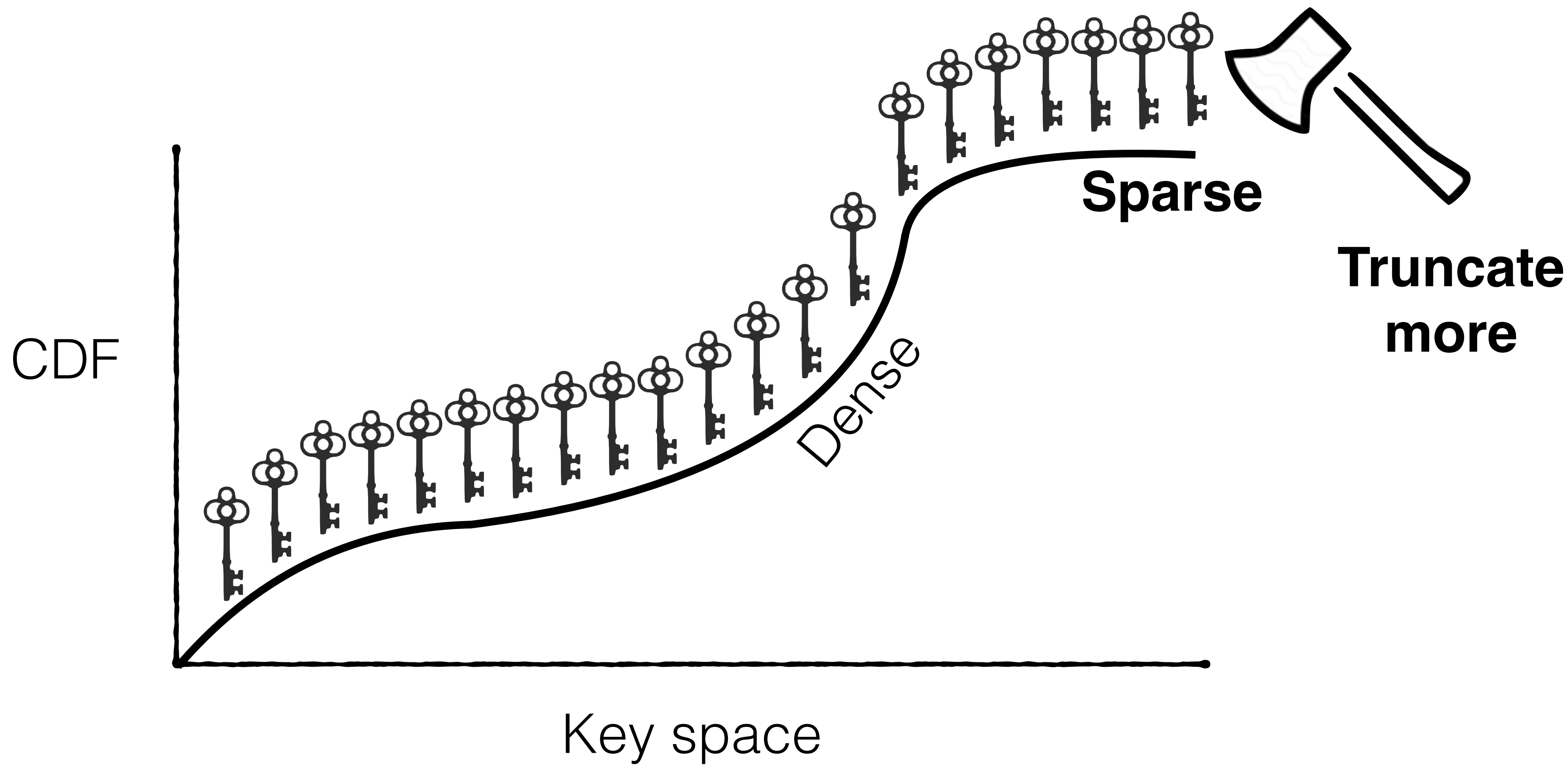


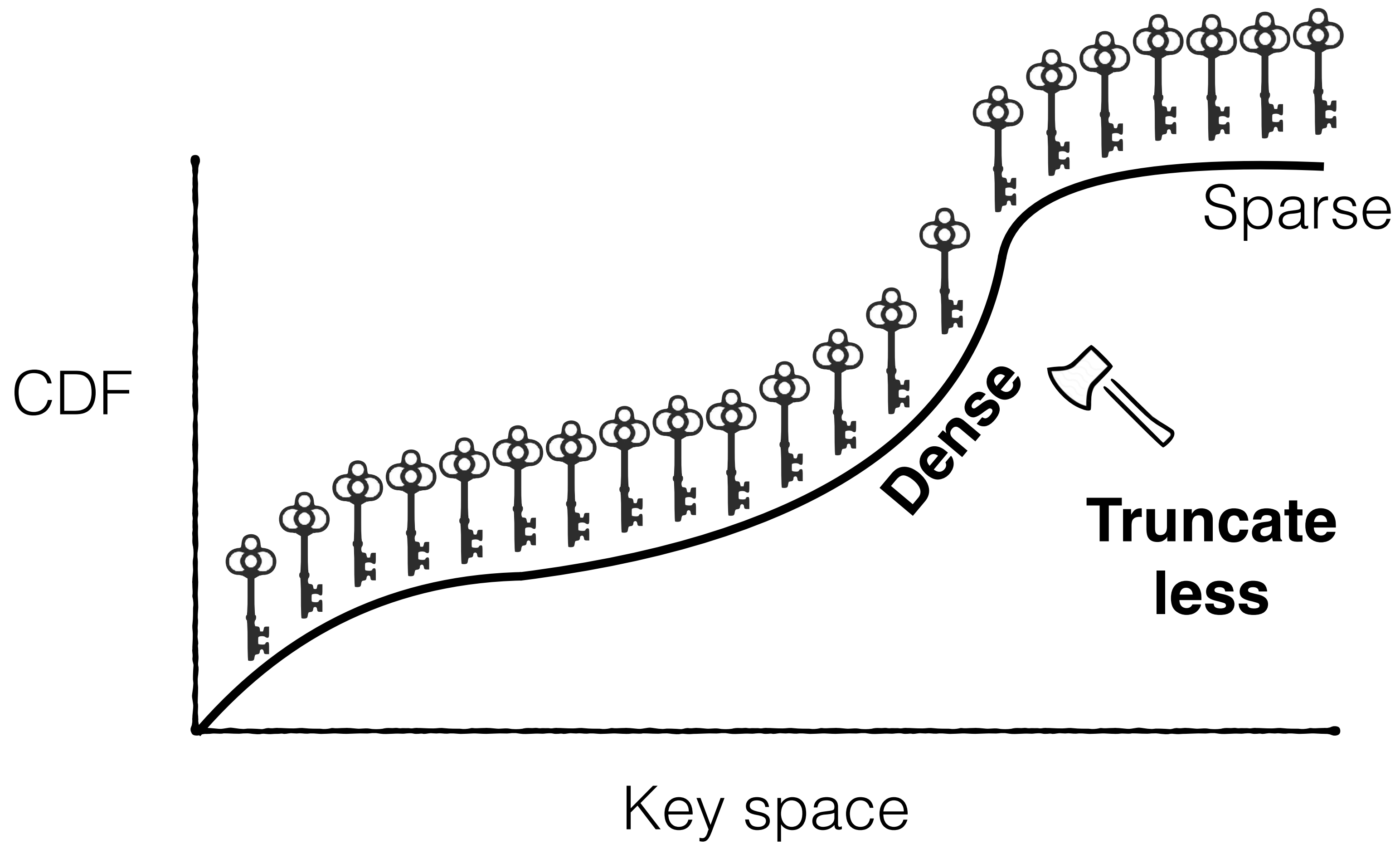
Cumulative
Distribution Function
(CDF)



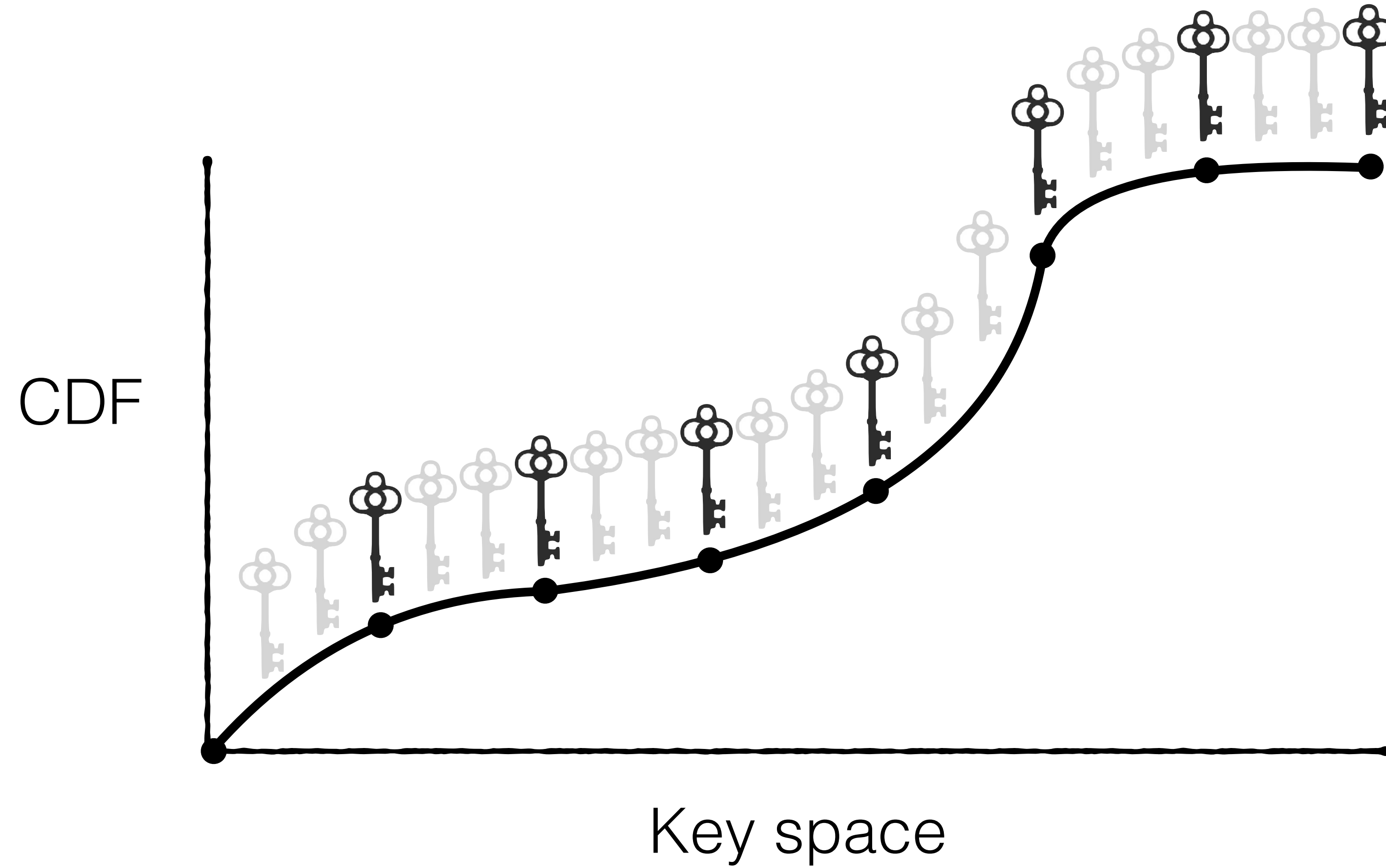
Key space





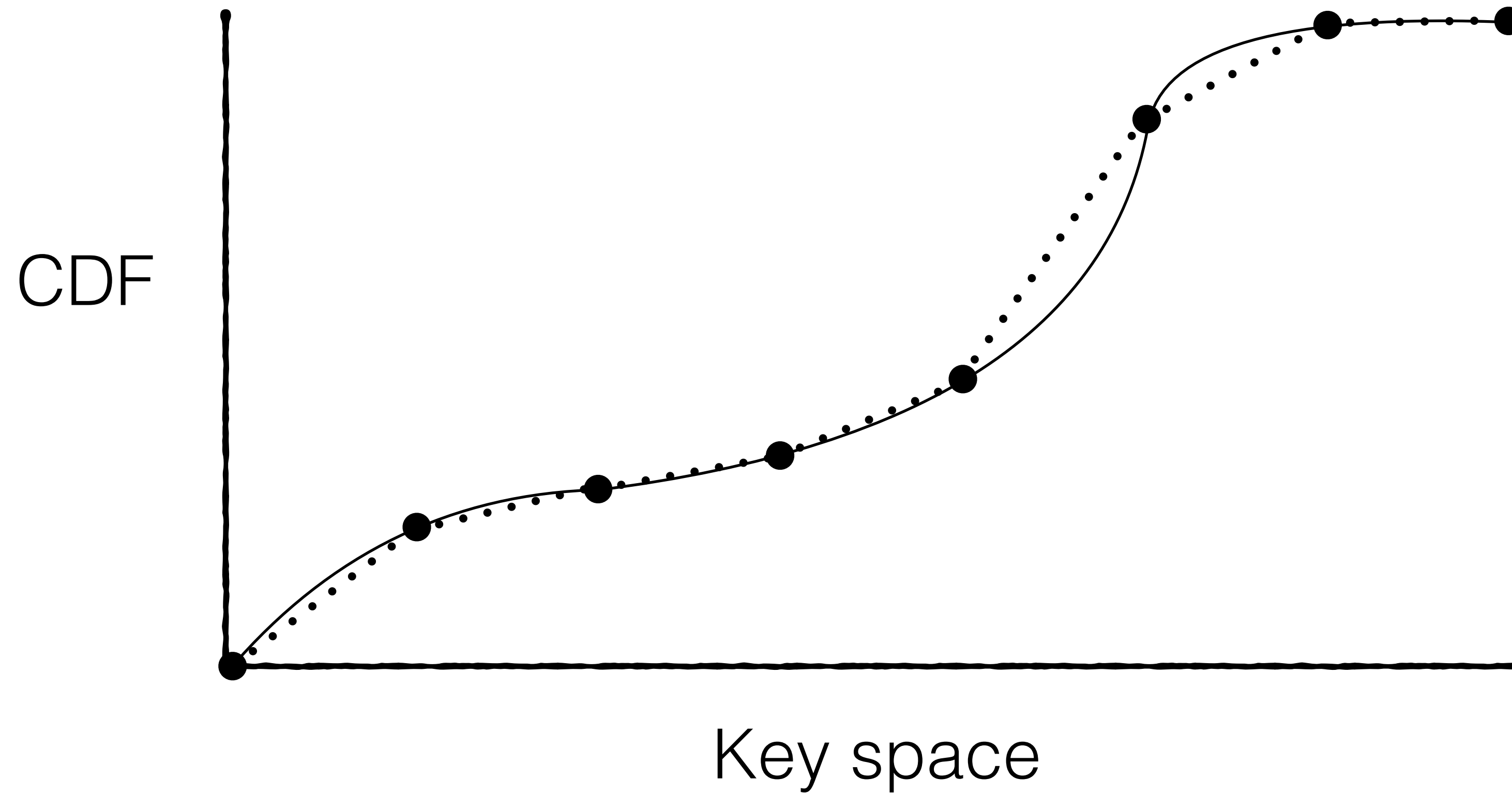


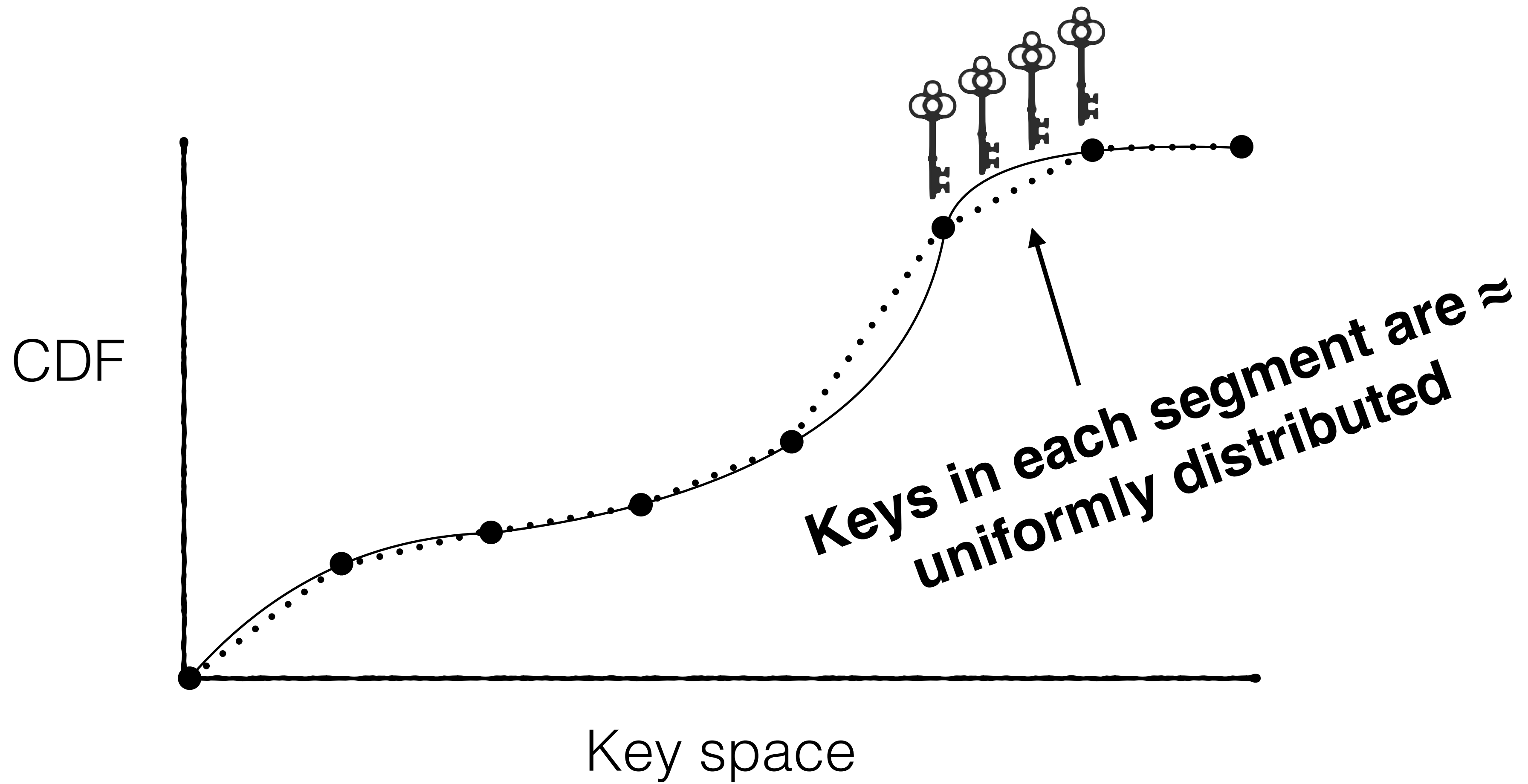
Sample every 1000th key

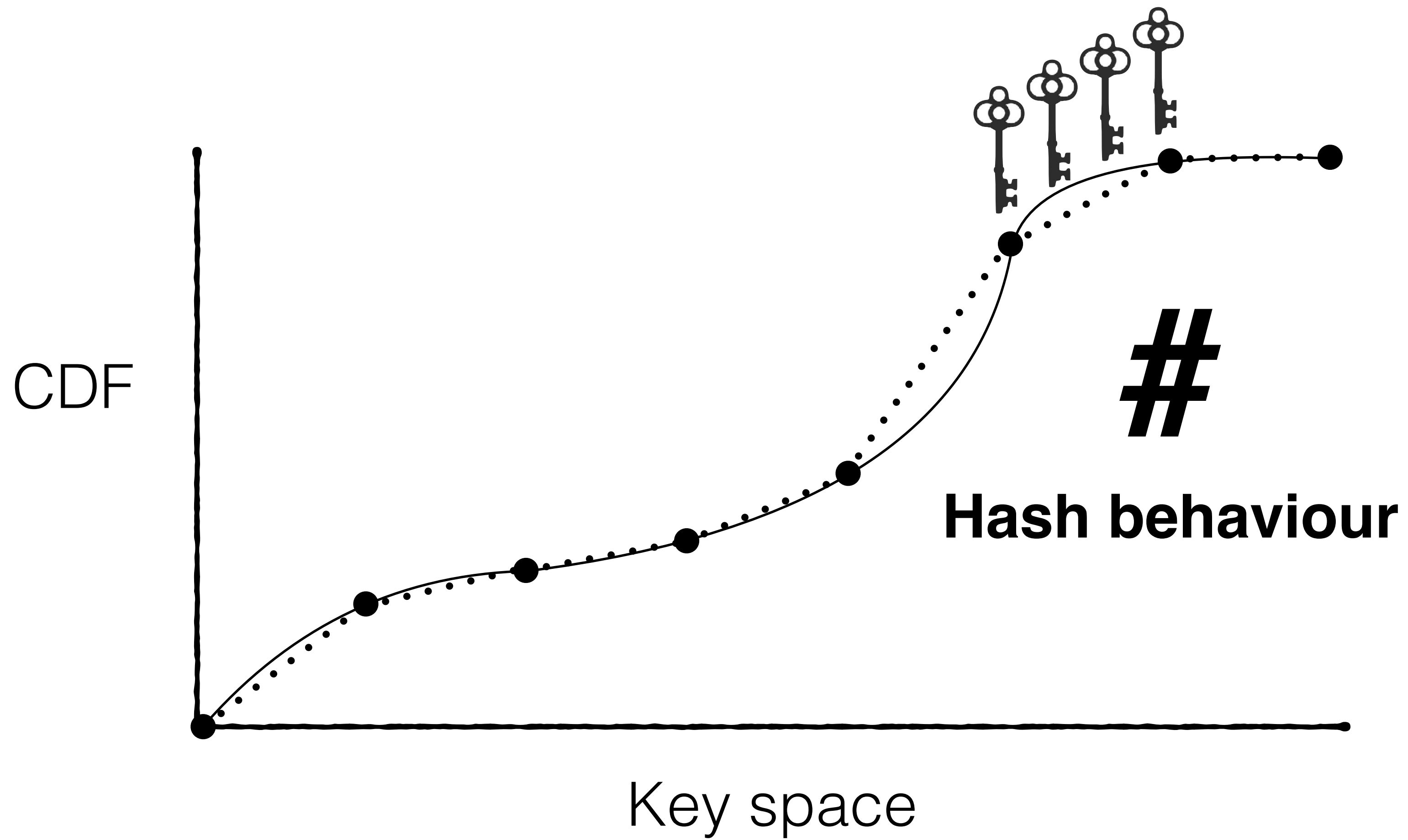


Sample every 1000th key

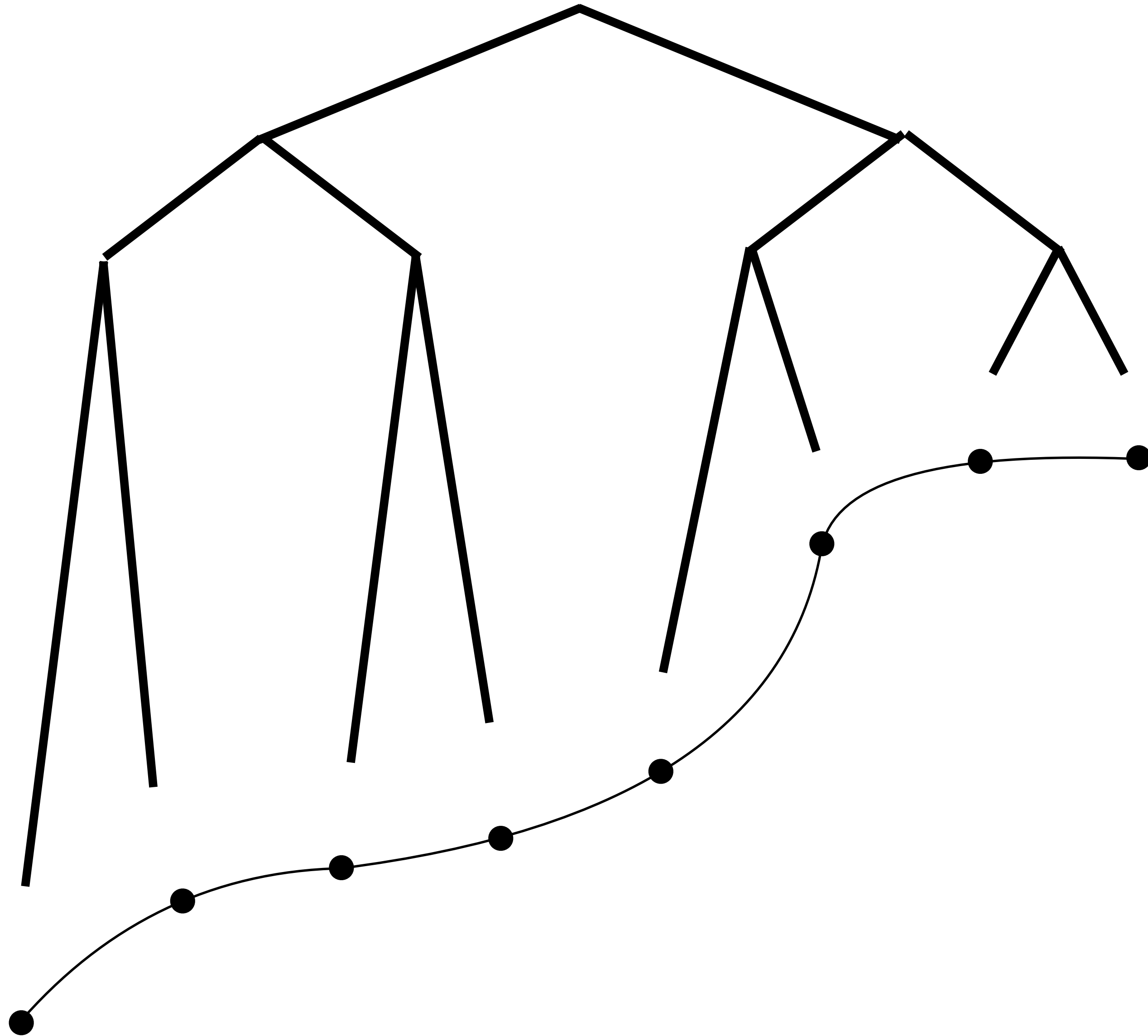
The samples approximate the distribution







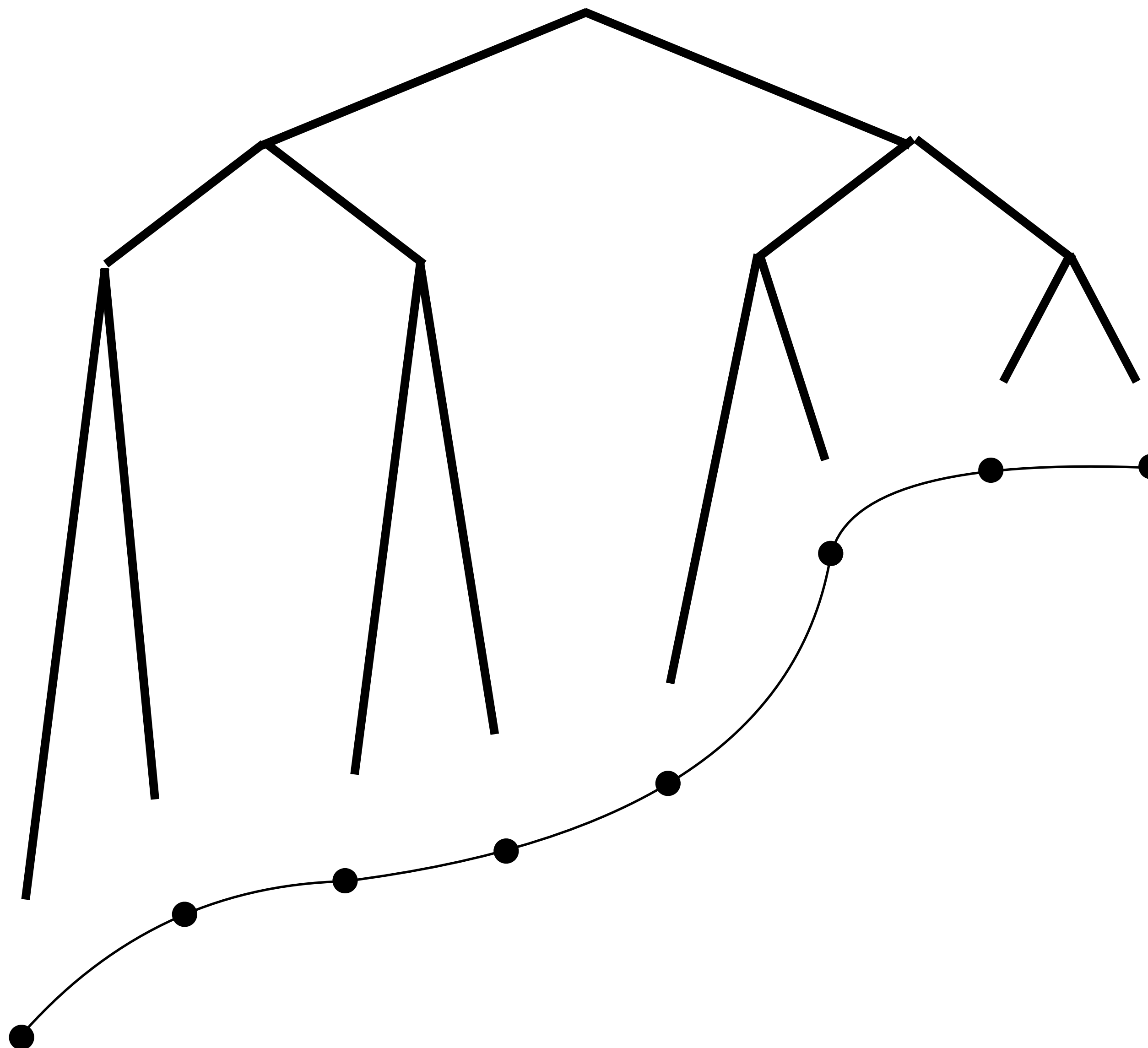
**Build tree from
samples**



Build tree from samples



0.1% of data size

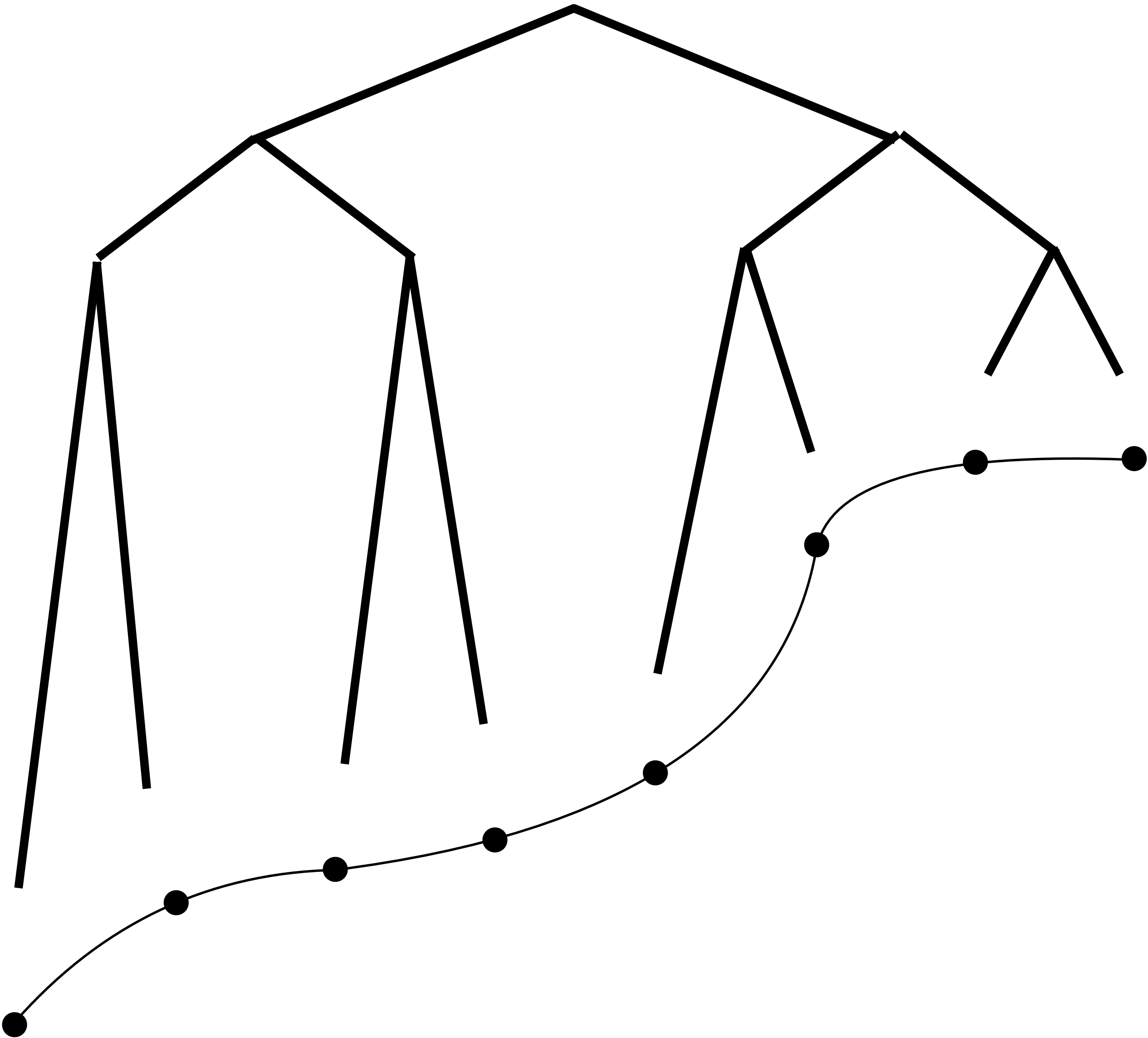


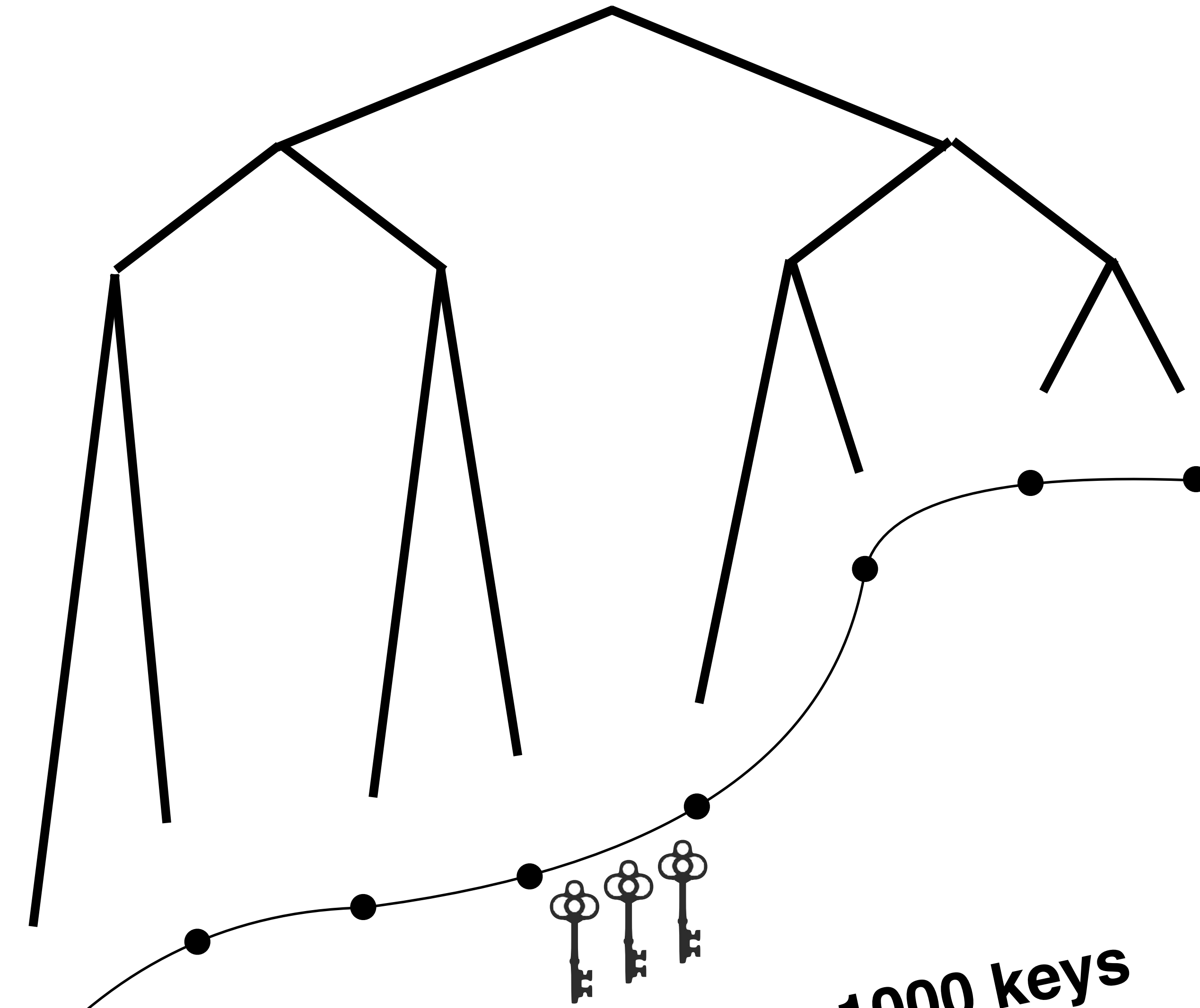
Build tree from
samples



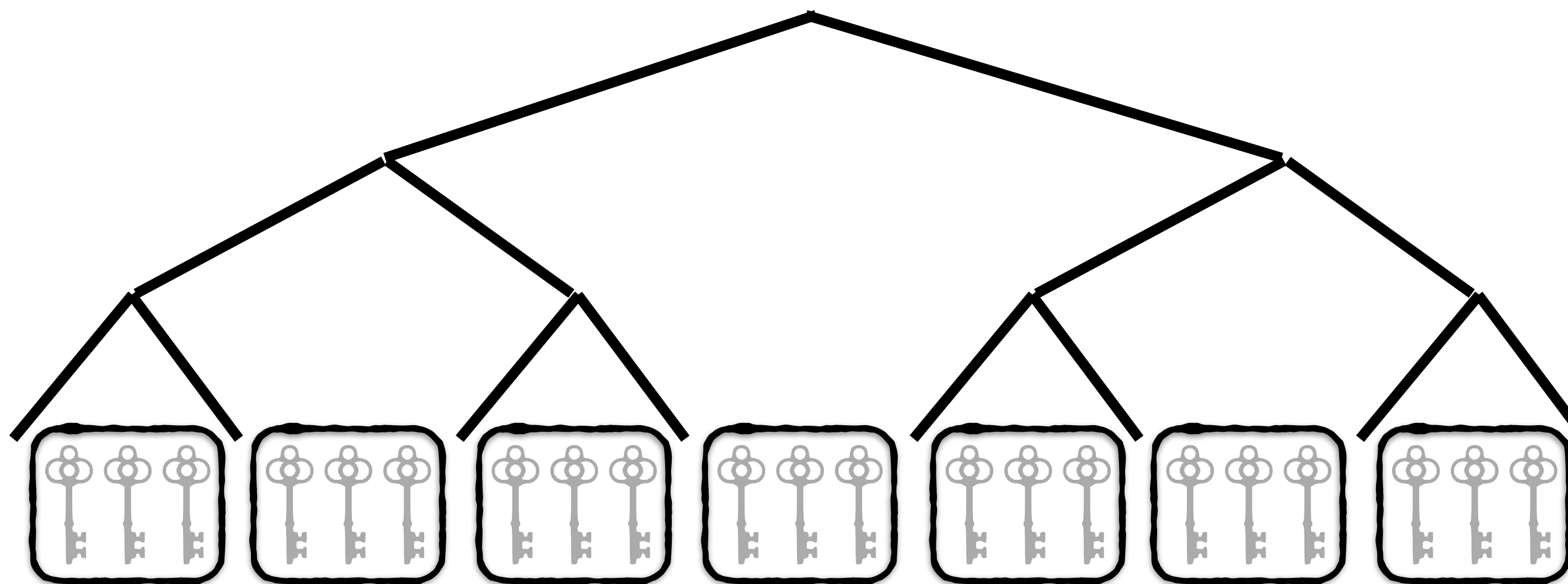
0.1% of data size

Fits in CPU caches



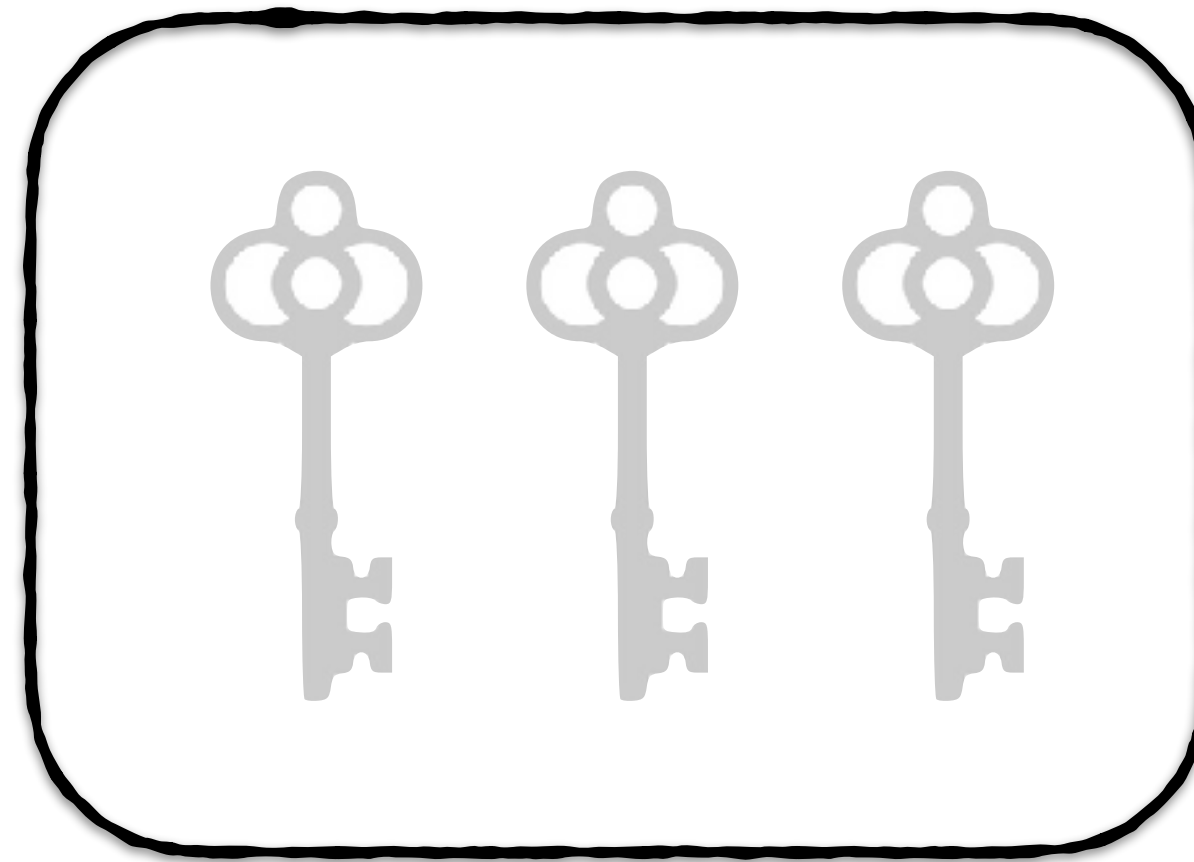


**How to organize the 1000 keys
in-between any two samples?**

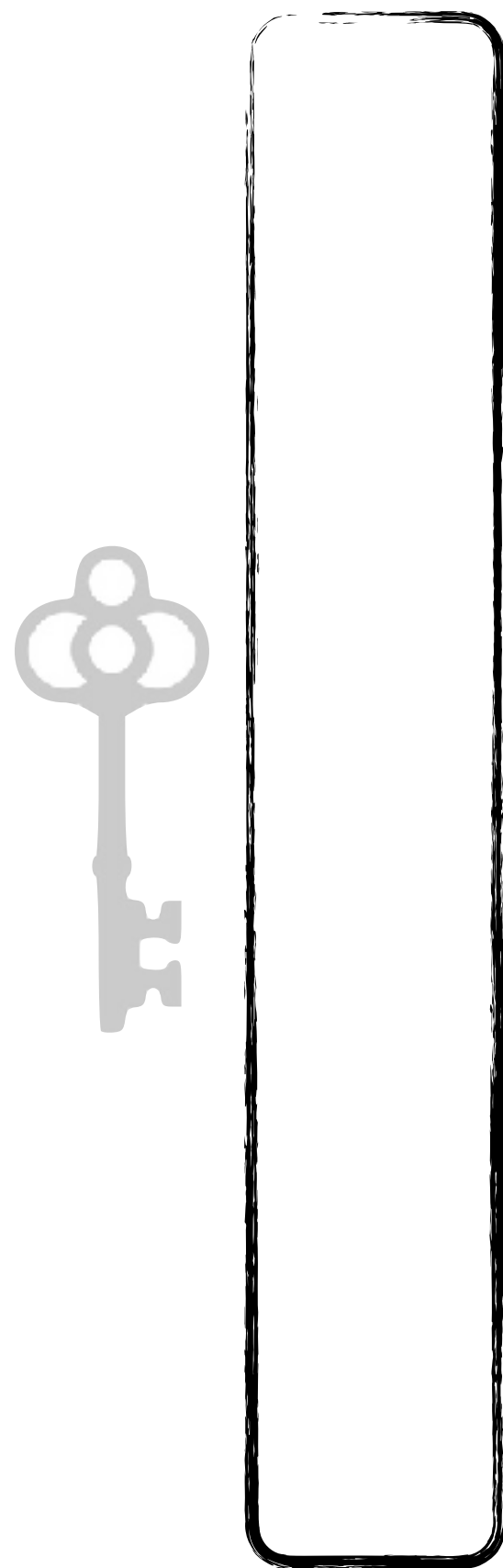


Infix Stores

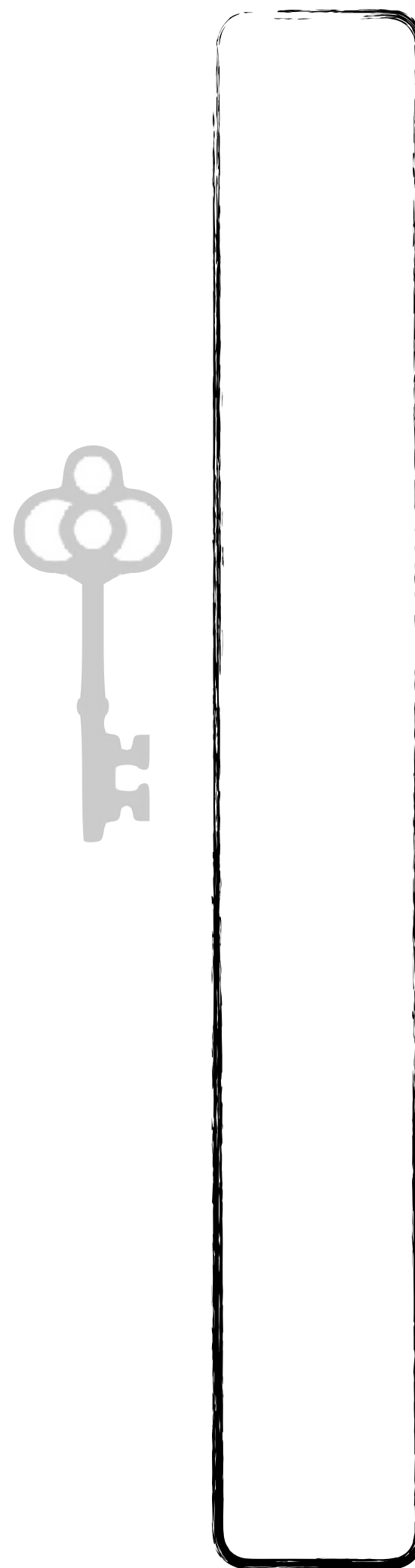
Order-preserving hash table of compressed keys



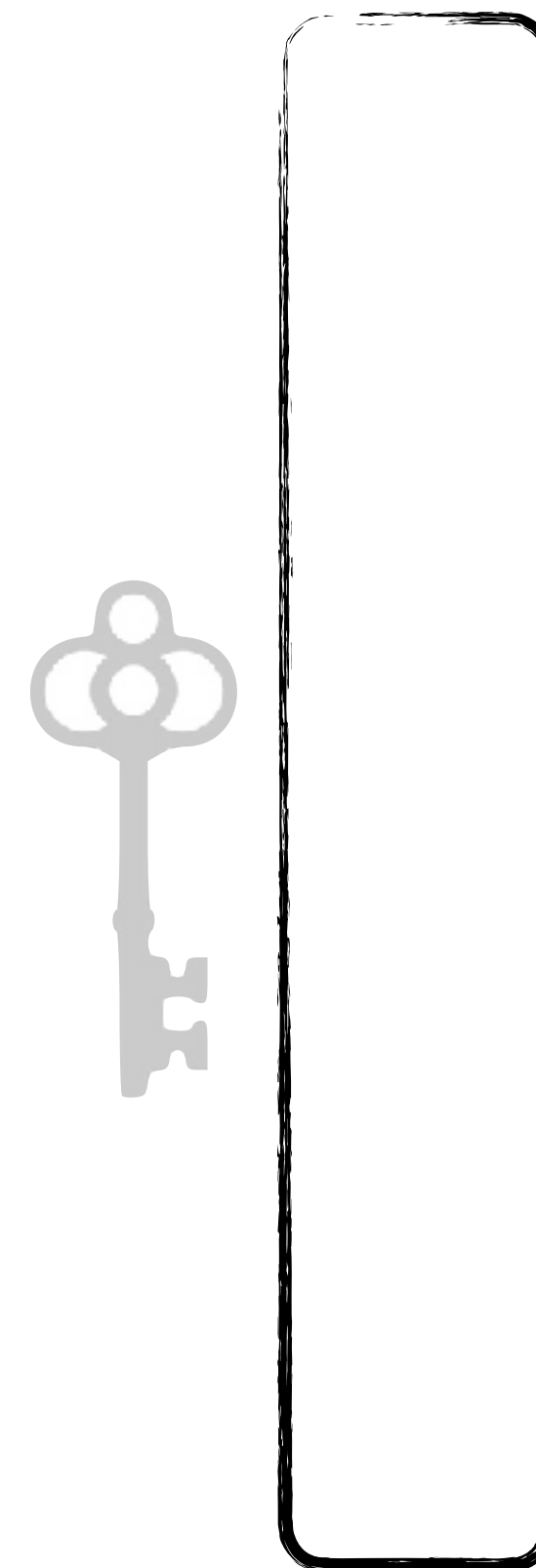
Infix Store



Key 1

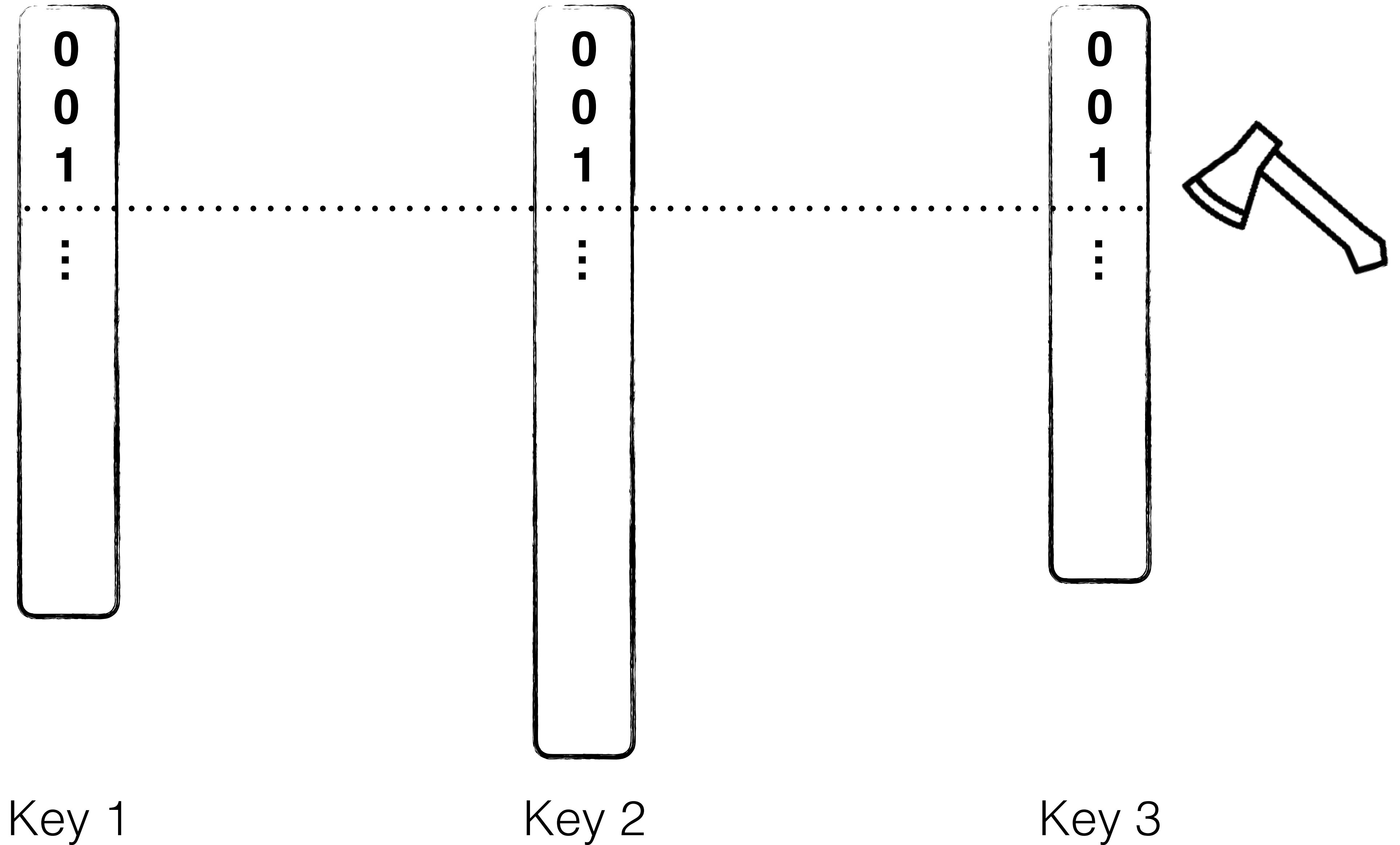


Key 2

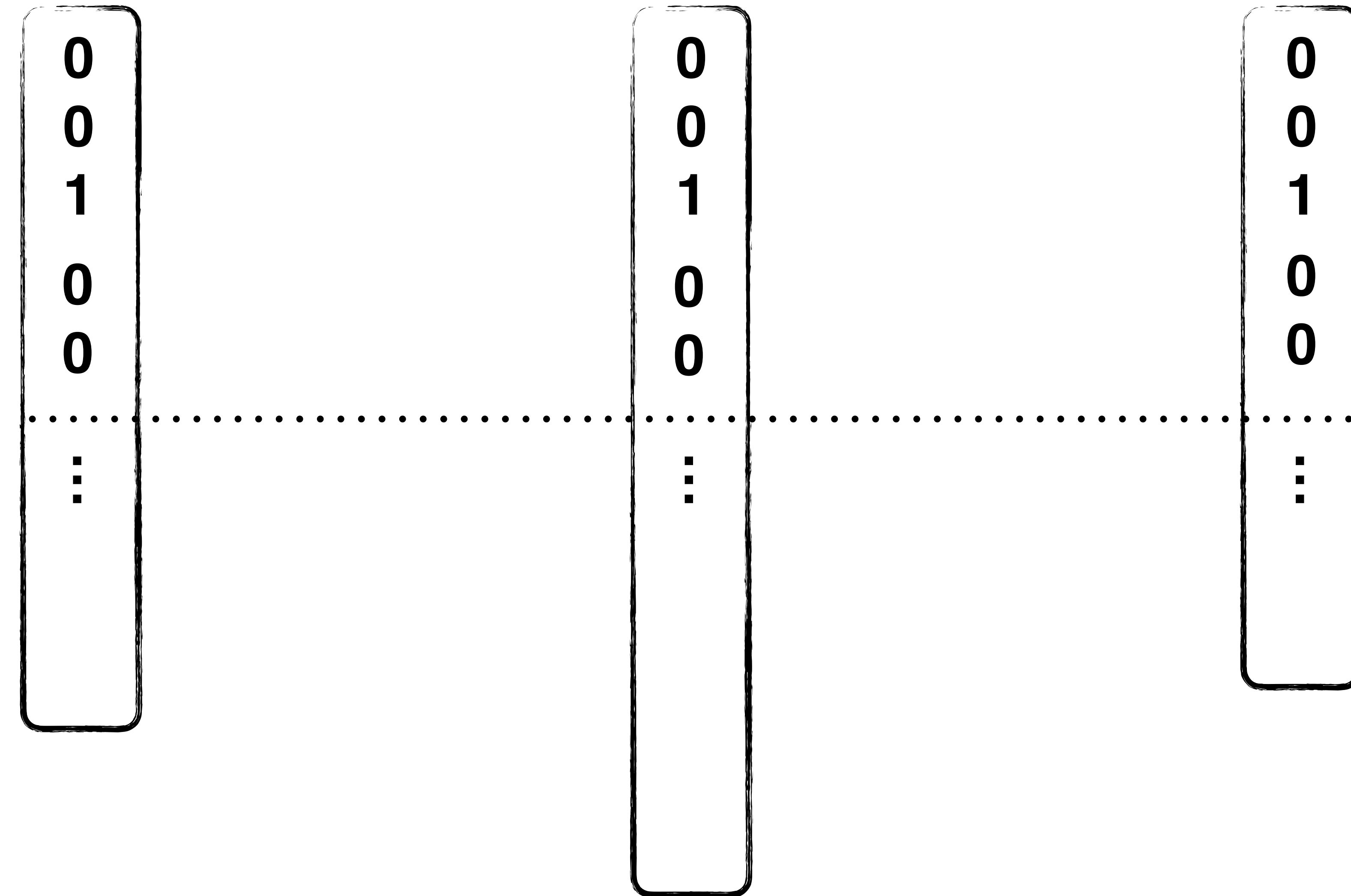


Key 3

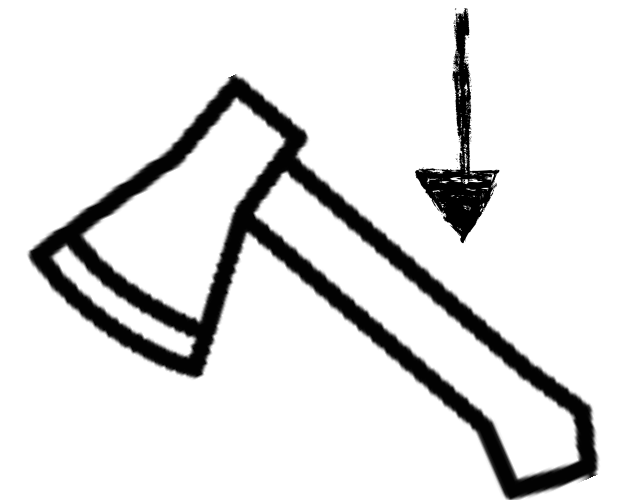
Remove greatest common prefix (GCP)



Remove greatest common prefix (GCP)



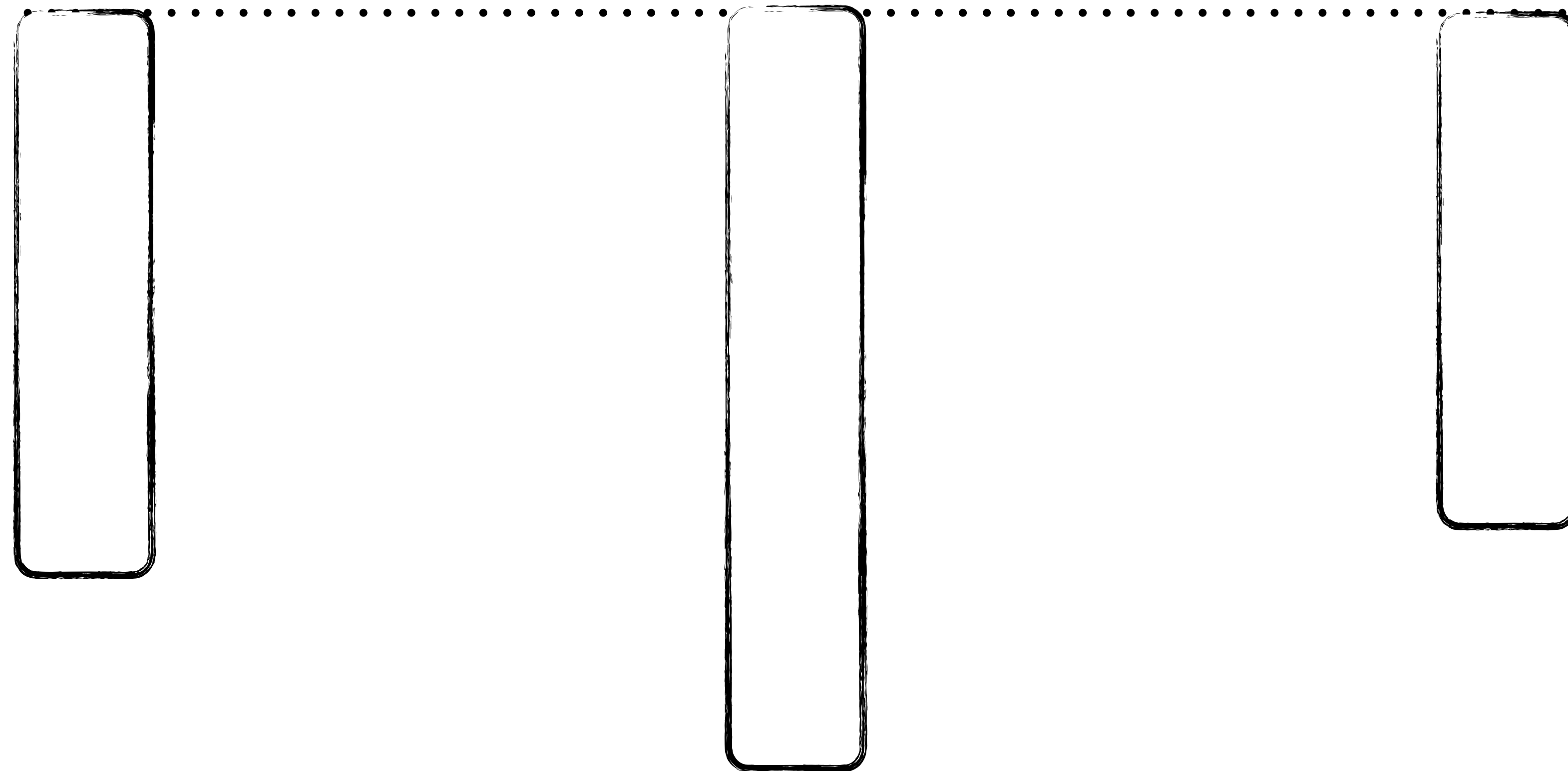
**For dense
parts of key
space**



We don't lose information

0
0
1

0
0
1

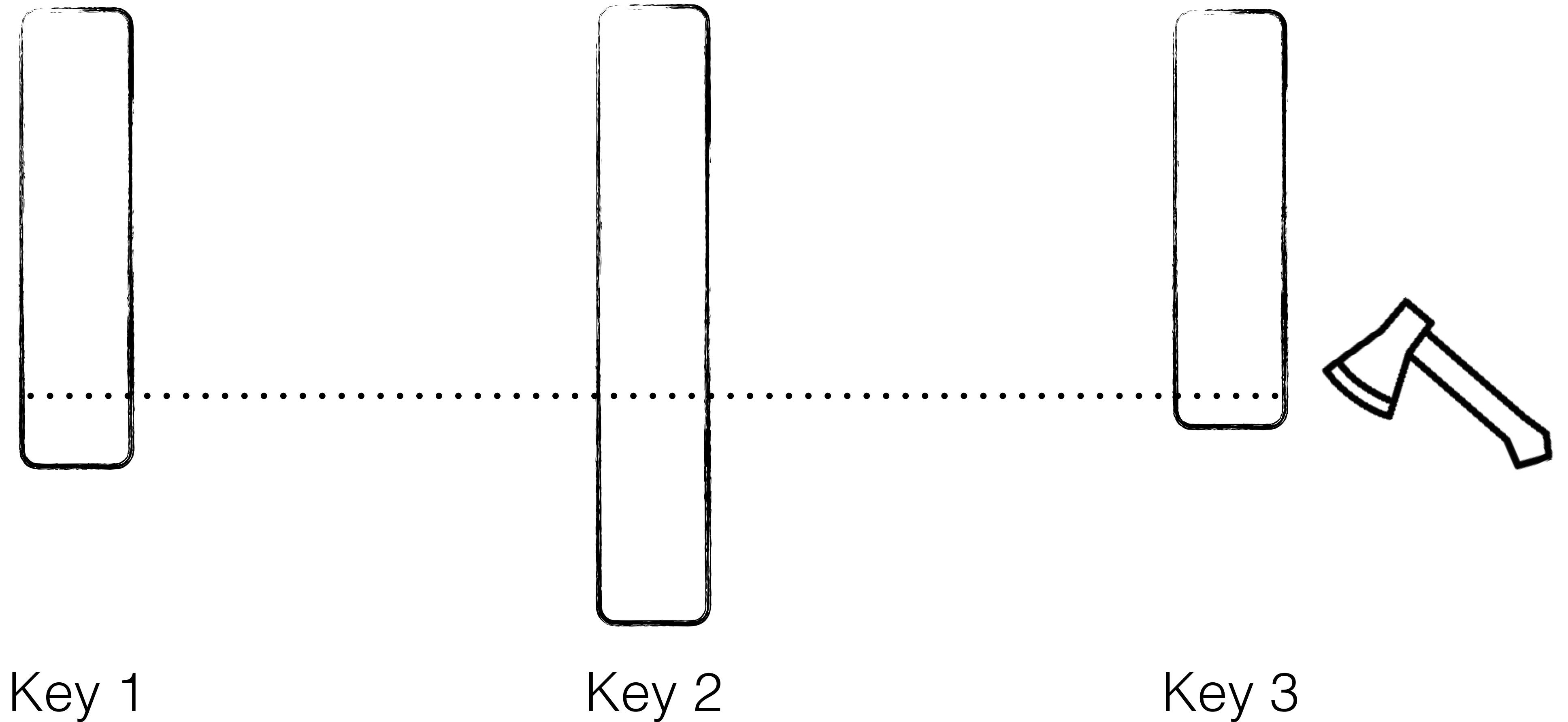


Key 1

Key 2

Key 3

Remove Suffix to homogenize keys lengths



What's left are $\approx$ uniformly distributed “infixes”

0
0
1
0
1

Key 1

0
0
0
1
1

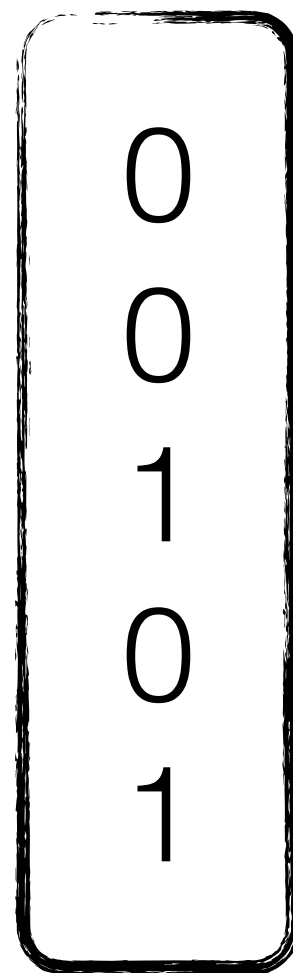
Key 2

0
1
1
1
0

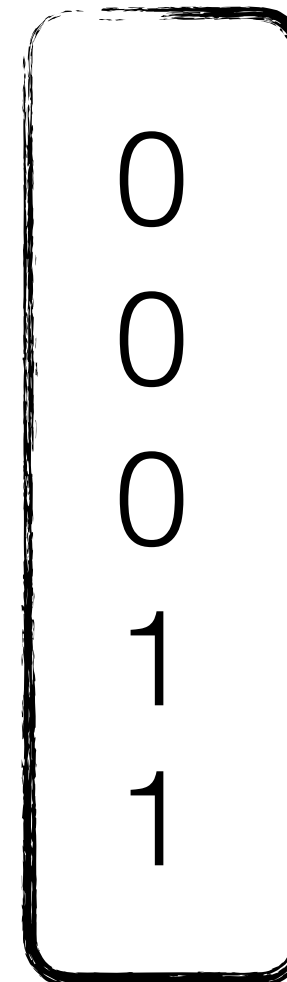
Key 3

What's left are $\approx$ uniformly distributed “infixes”

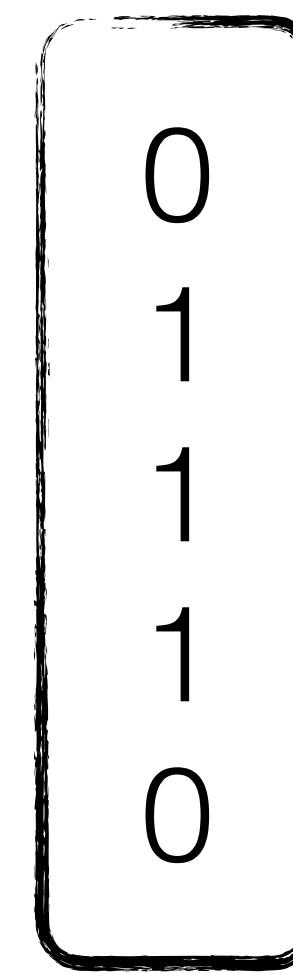
Store in hash table



Key 1



Key 2

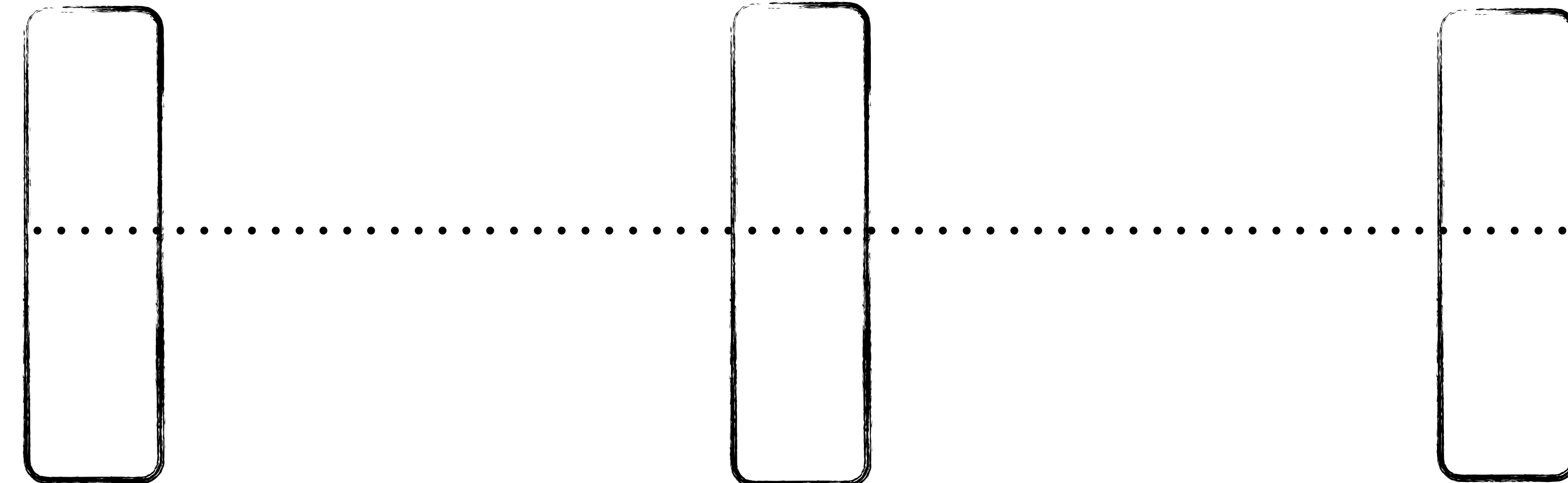


Key 3

Store in hash table

Address

Fingerprint

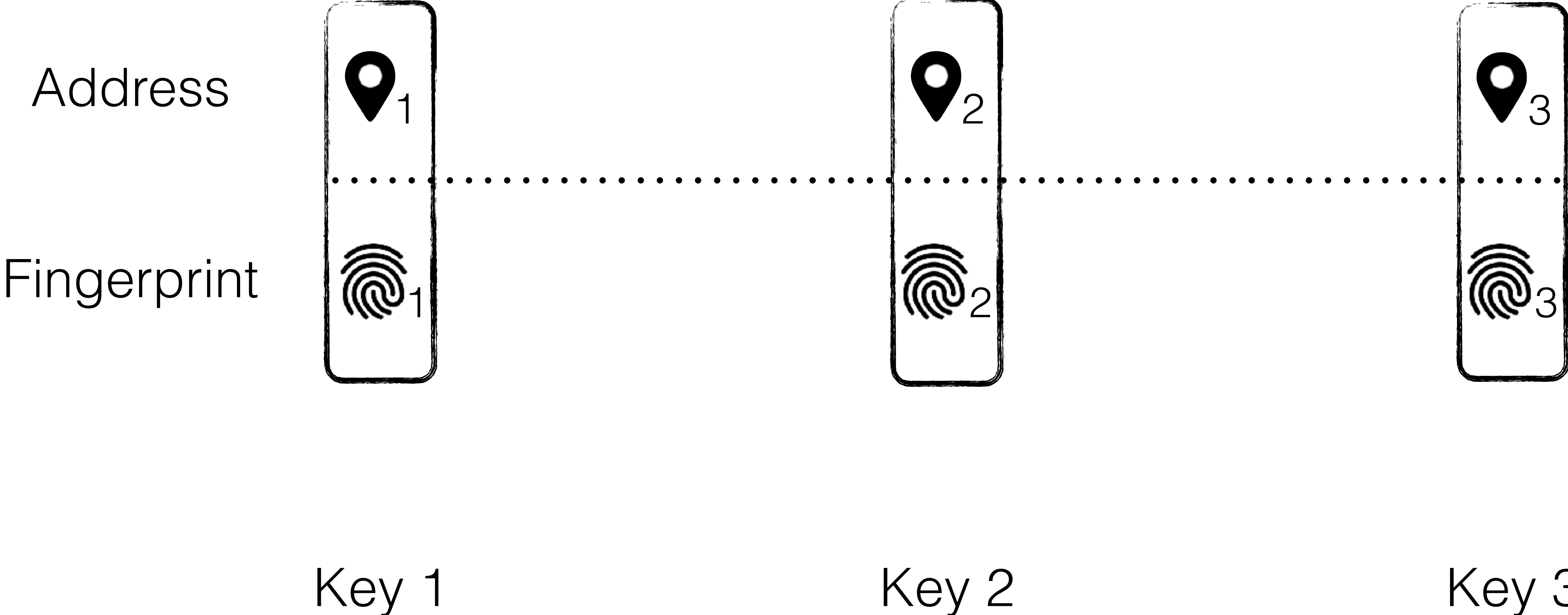


Key 1

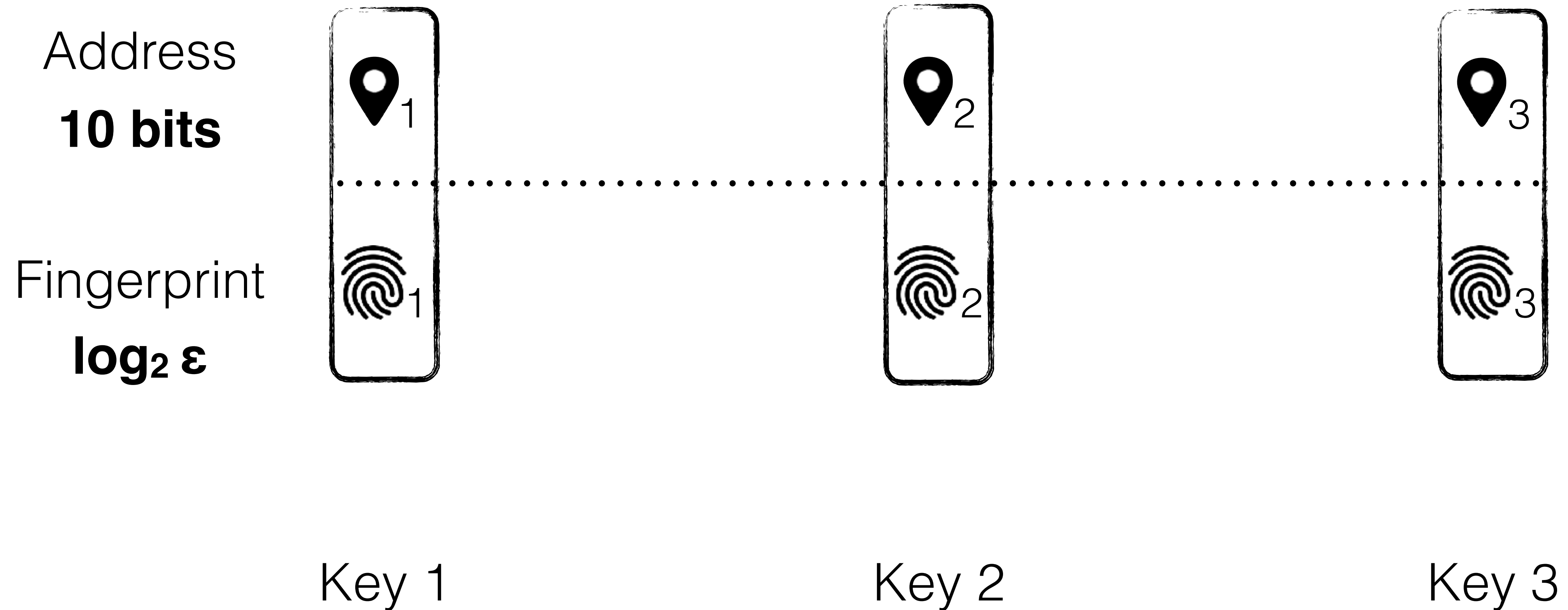
Key 2

Key 3

Divide into addresses and fingerprints



Divide into addresses and fingerprints





Compact hash table



Stored explicitly



Stored implicitly

A General-Purpose Counting Filter: Making Every Bit Count

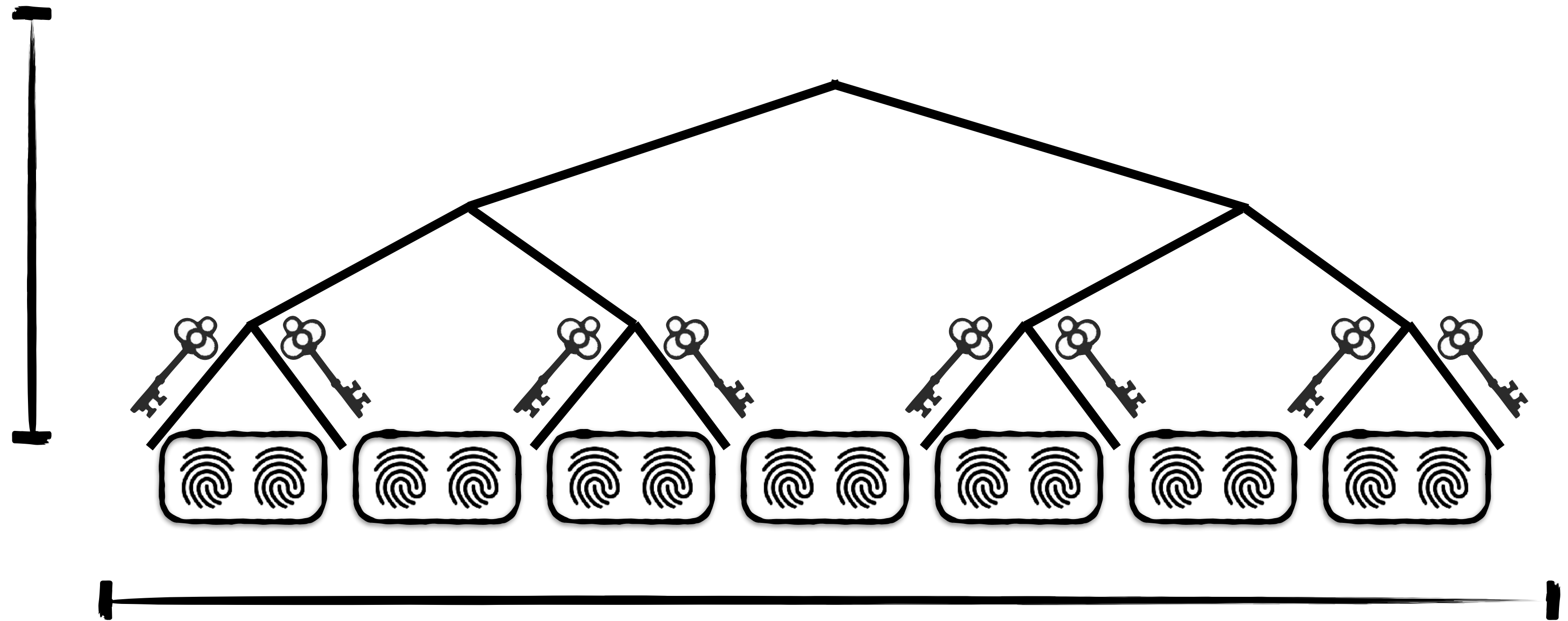
SIGMOD 2017, Prashant Pandey et al



Order-perserving hash table with $O(1)$ access time :)

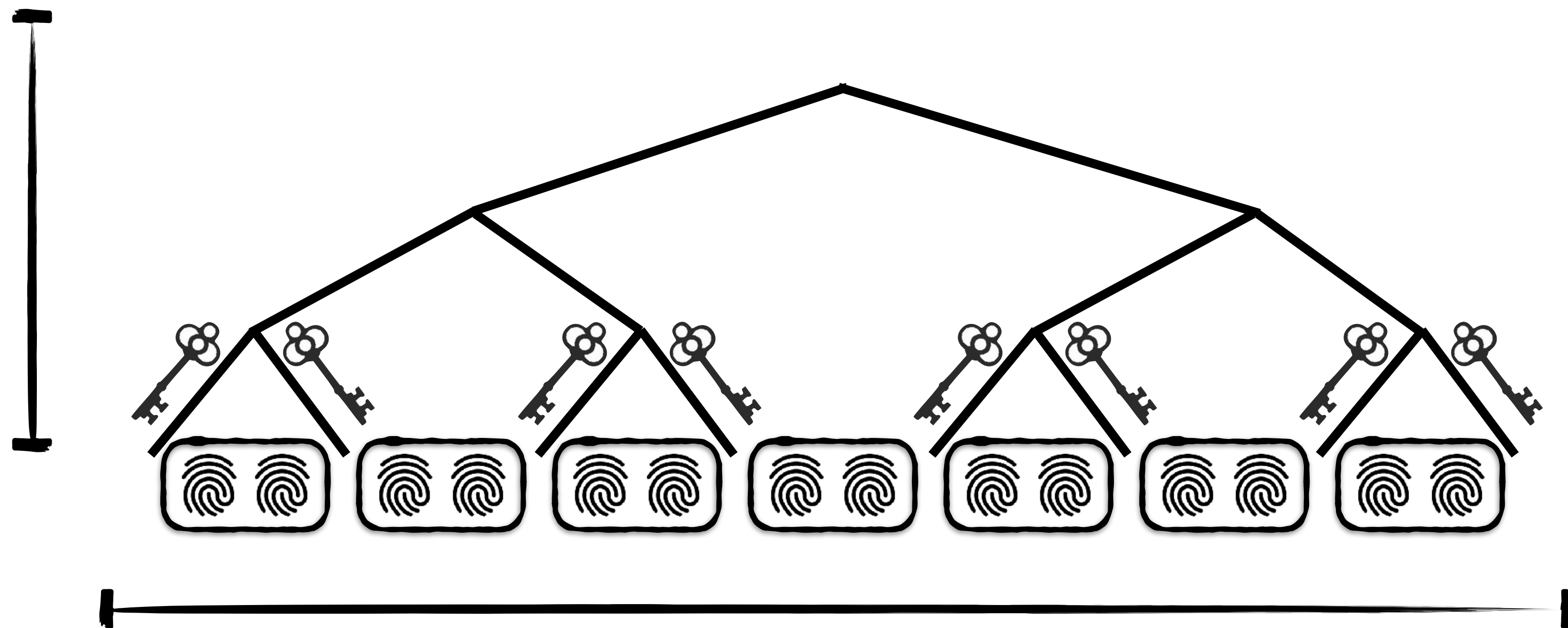


Cached Tree
Every 1000th key



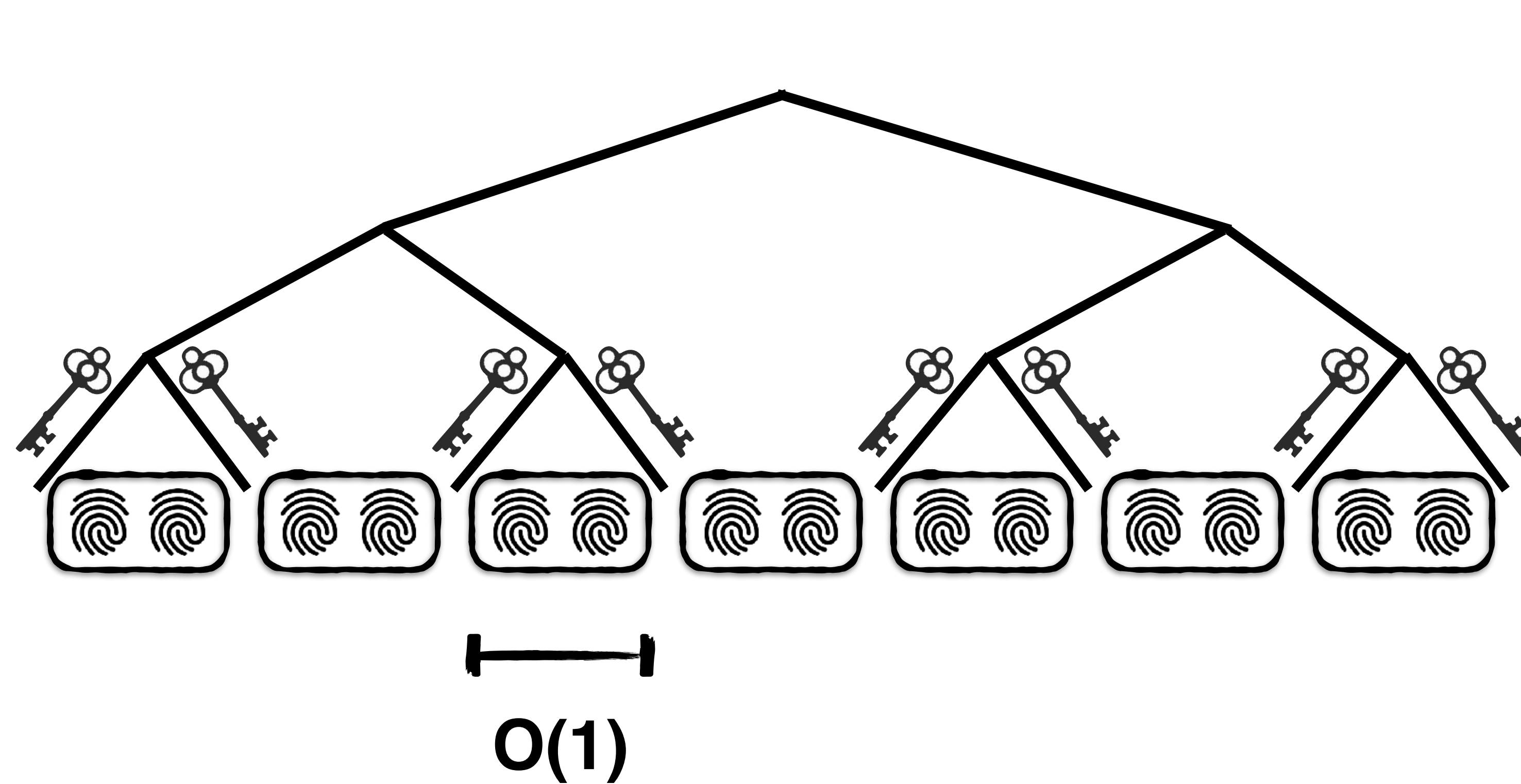
Infix stores
1000 keys each

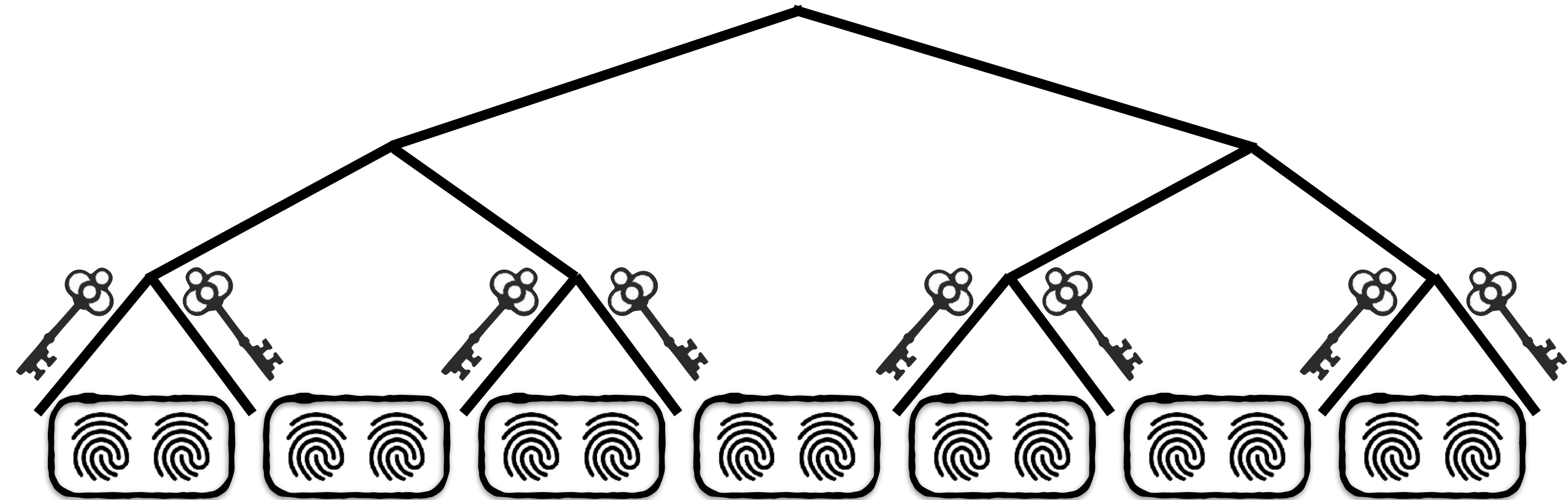
Y-fast trie
 $O(\log L)$



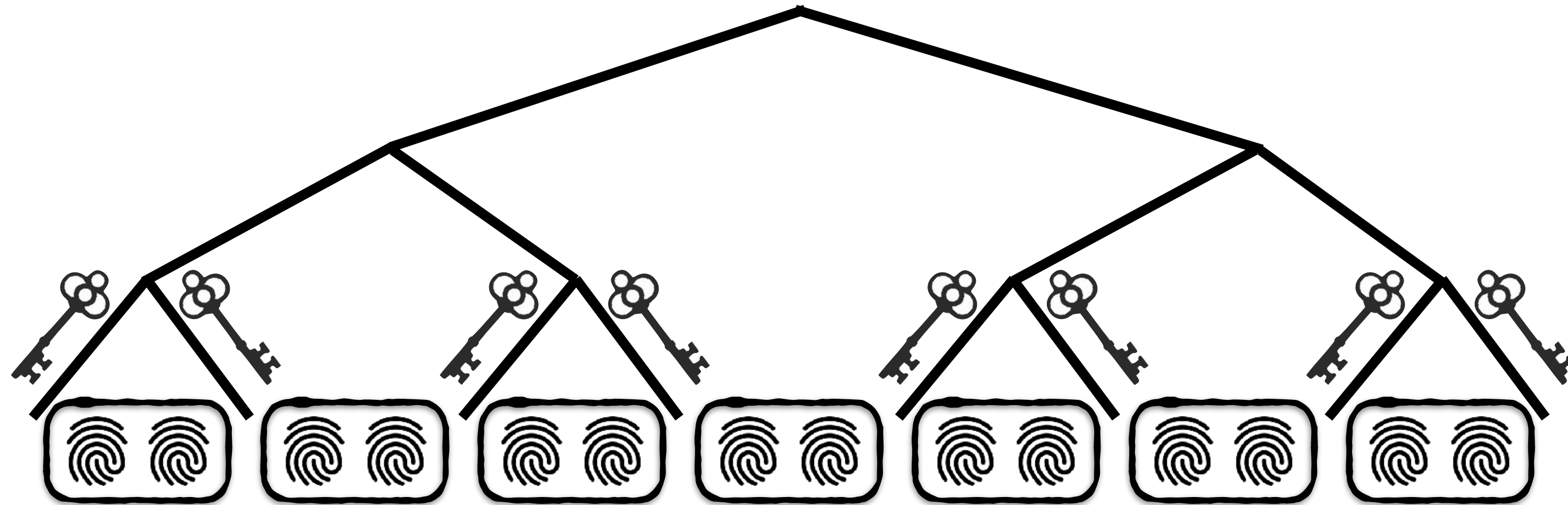
Infix stores
1000 keys each

Y-fast trie
 $O(\log L)$

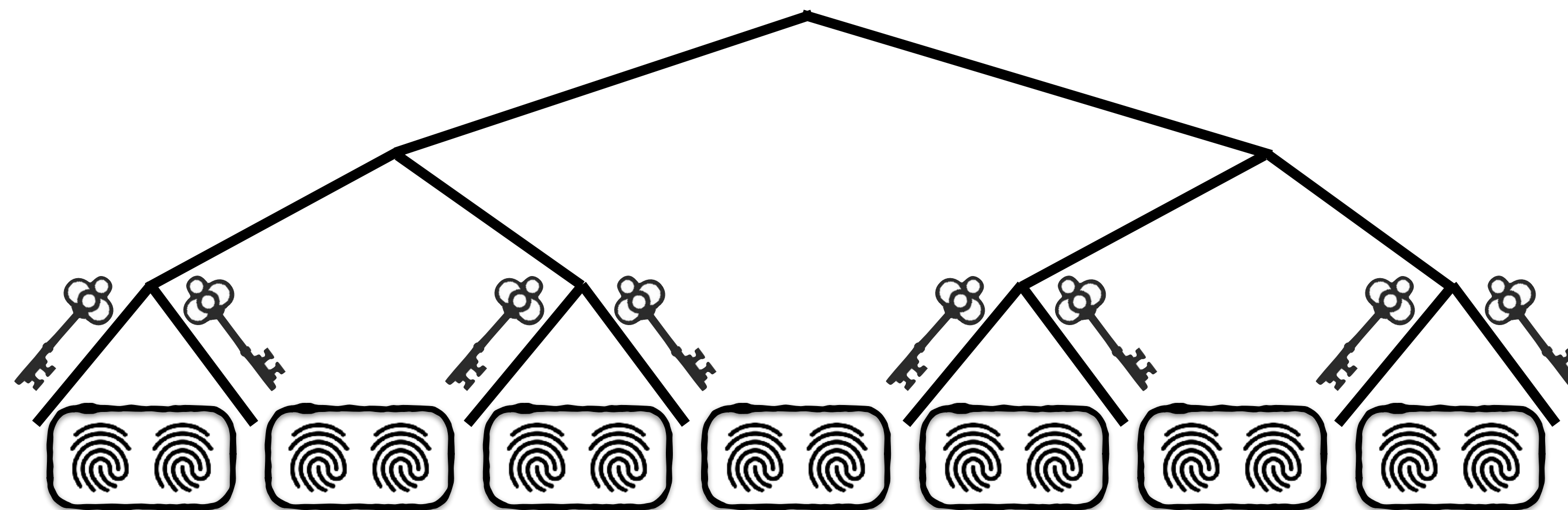




Runtime: $O(\log L) + O(1) = \mathbf{O(\log L)}$

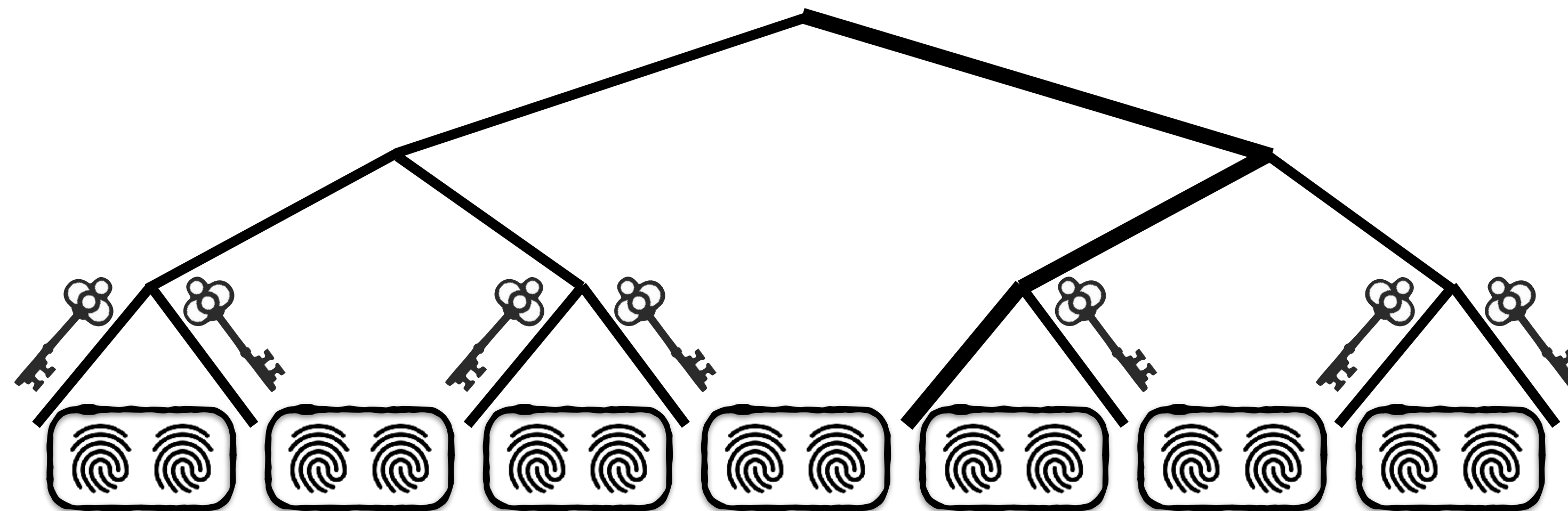


How to check if a range is empty?



Key space

Query

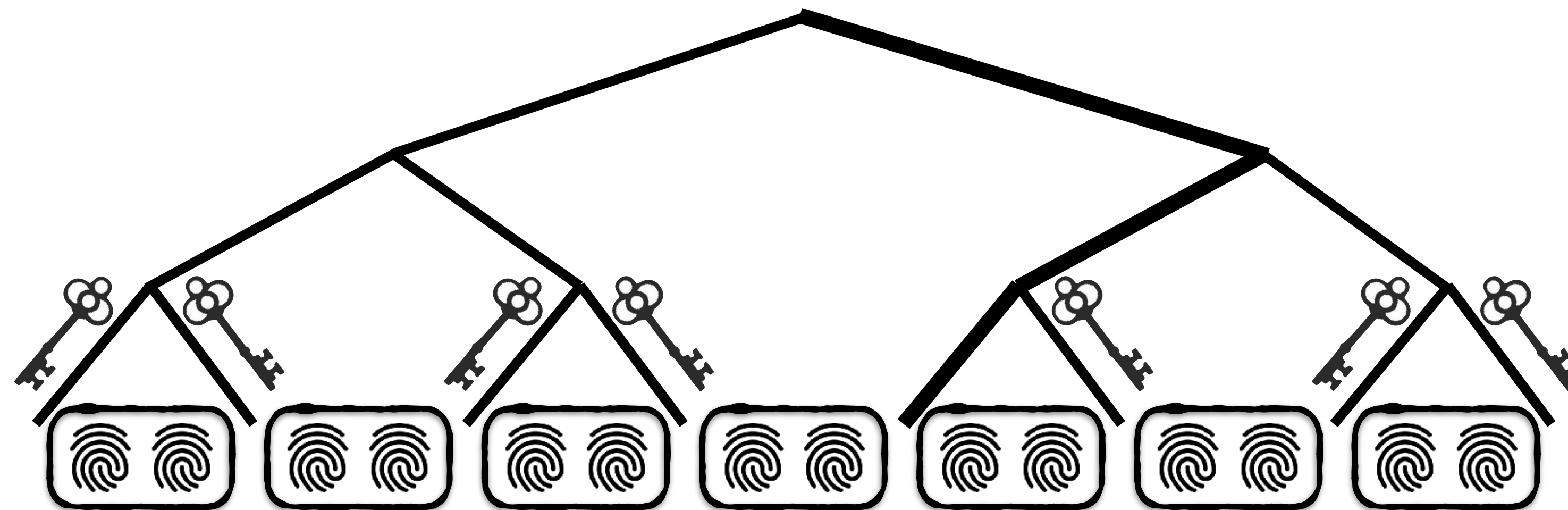


Key space



Query

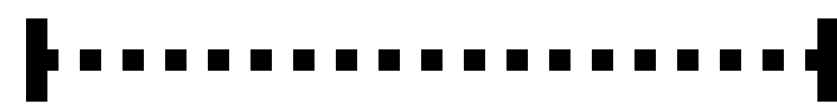
Return true positive



Key space

Query

Infix store



Query



Infix store



Query



Return true positive

Infix store



I-I

Query



Infix store



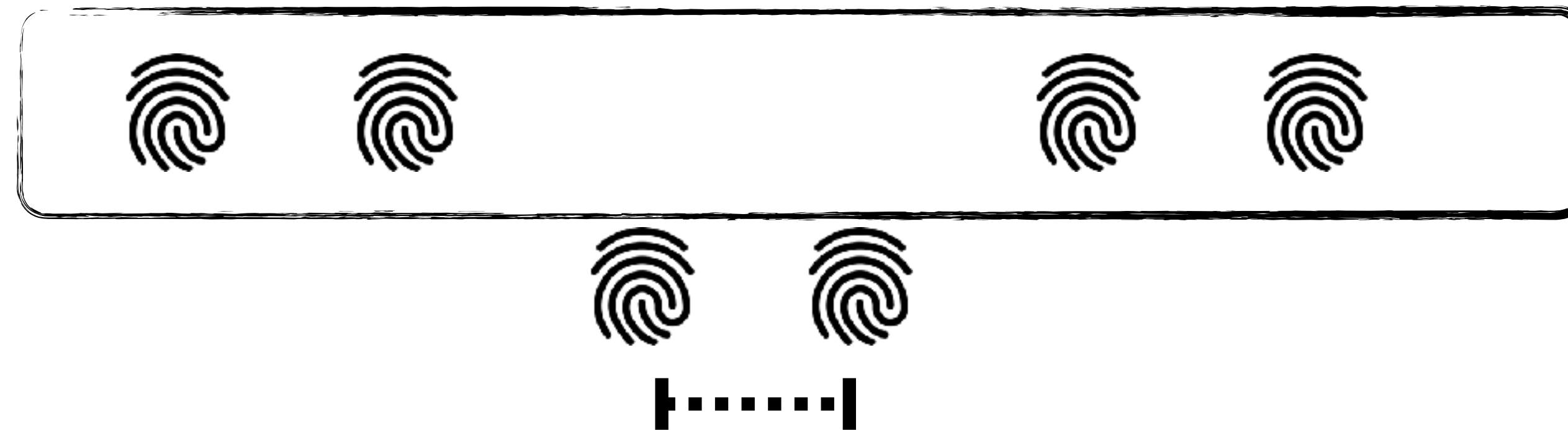
I-I

Query



Return true negative

Infix store



Query endpoints may match infixes of existing keys

FPR: $O(2^{-F})$

FPR: $O(2^{-F})$ - a semi-robust guarantee

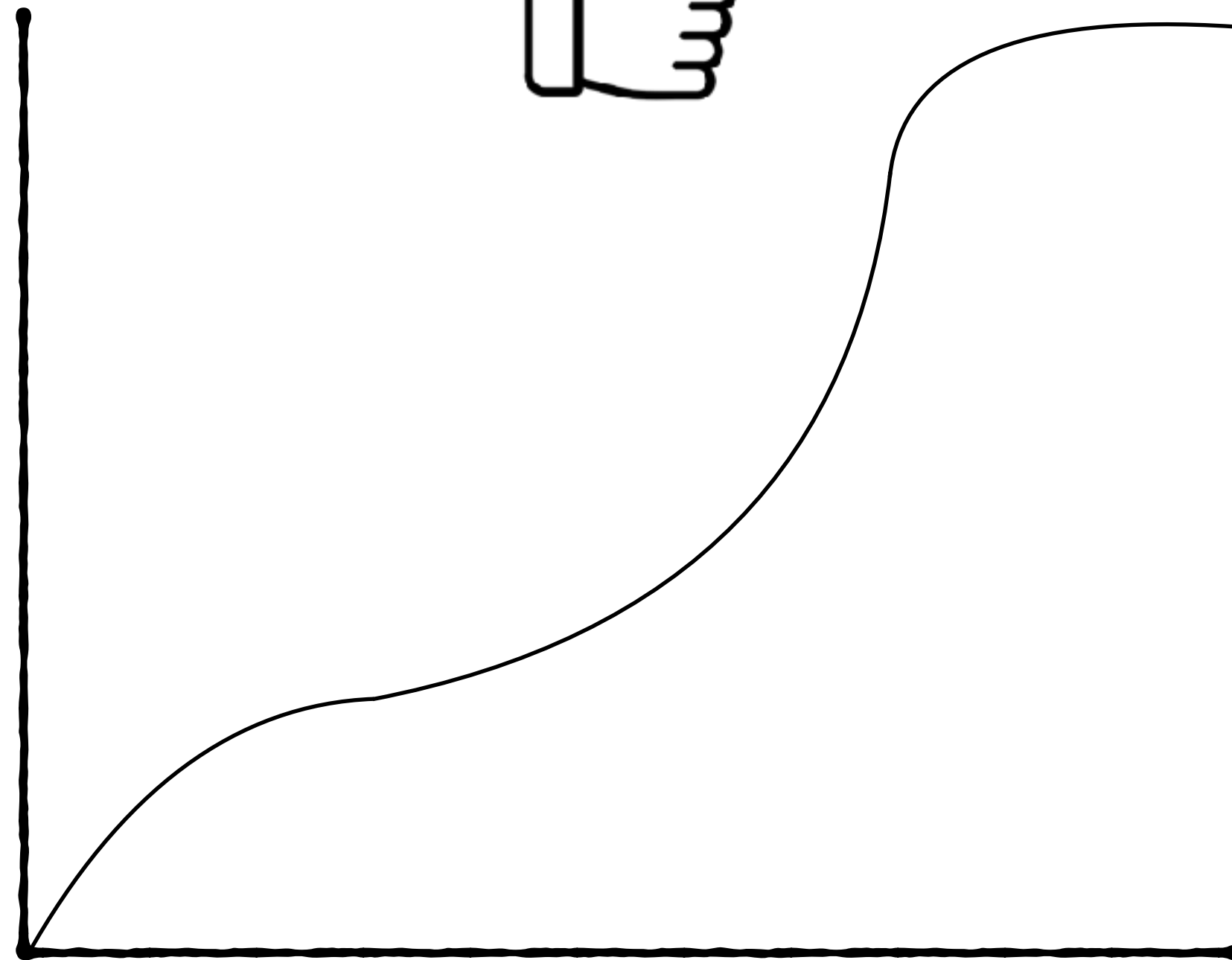


FPR: $O(2^{-F})$ - a semi-robust guarantee

Smooth distributions (Numeric data)



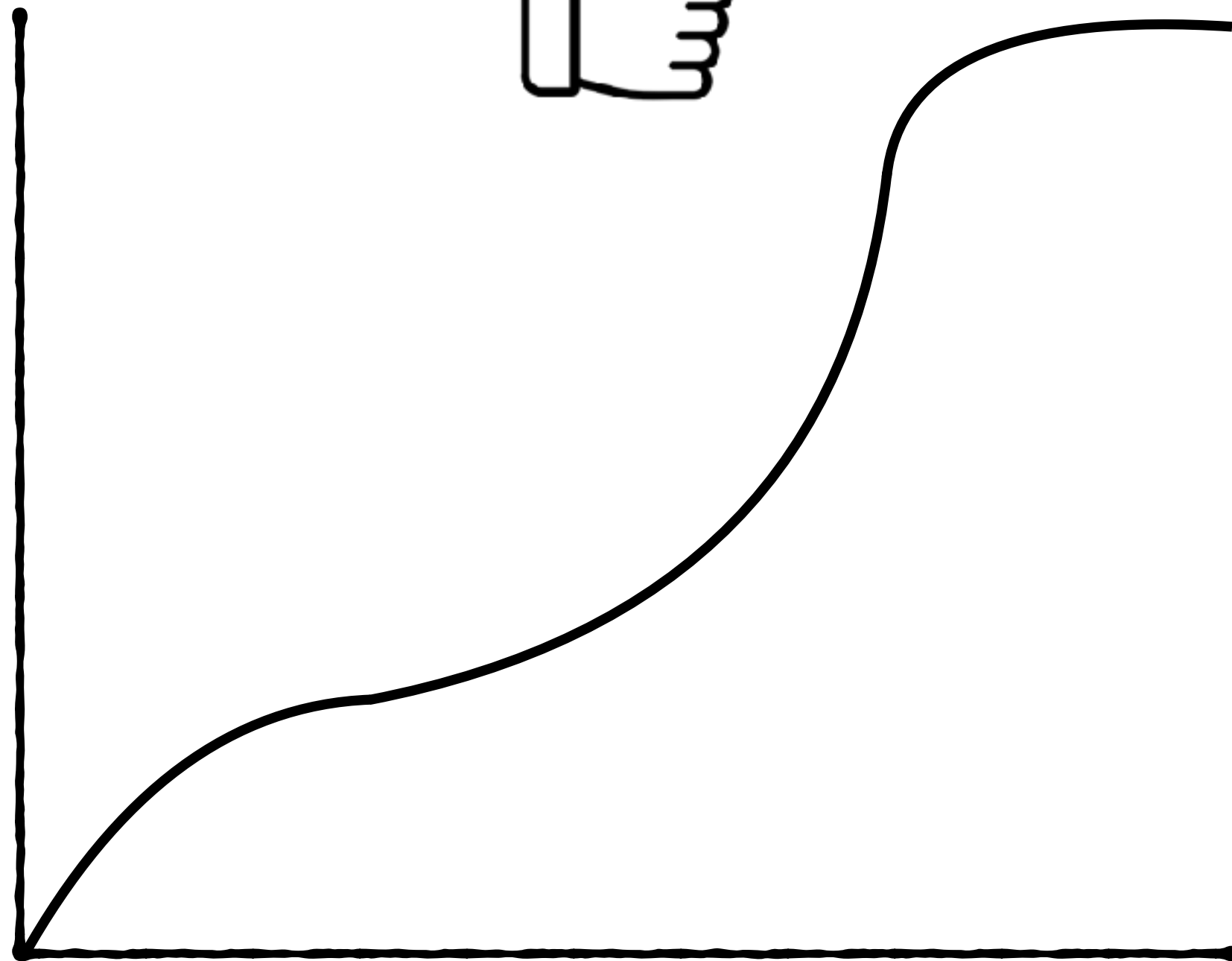
CDF



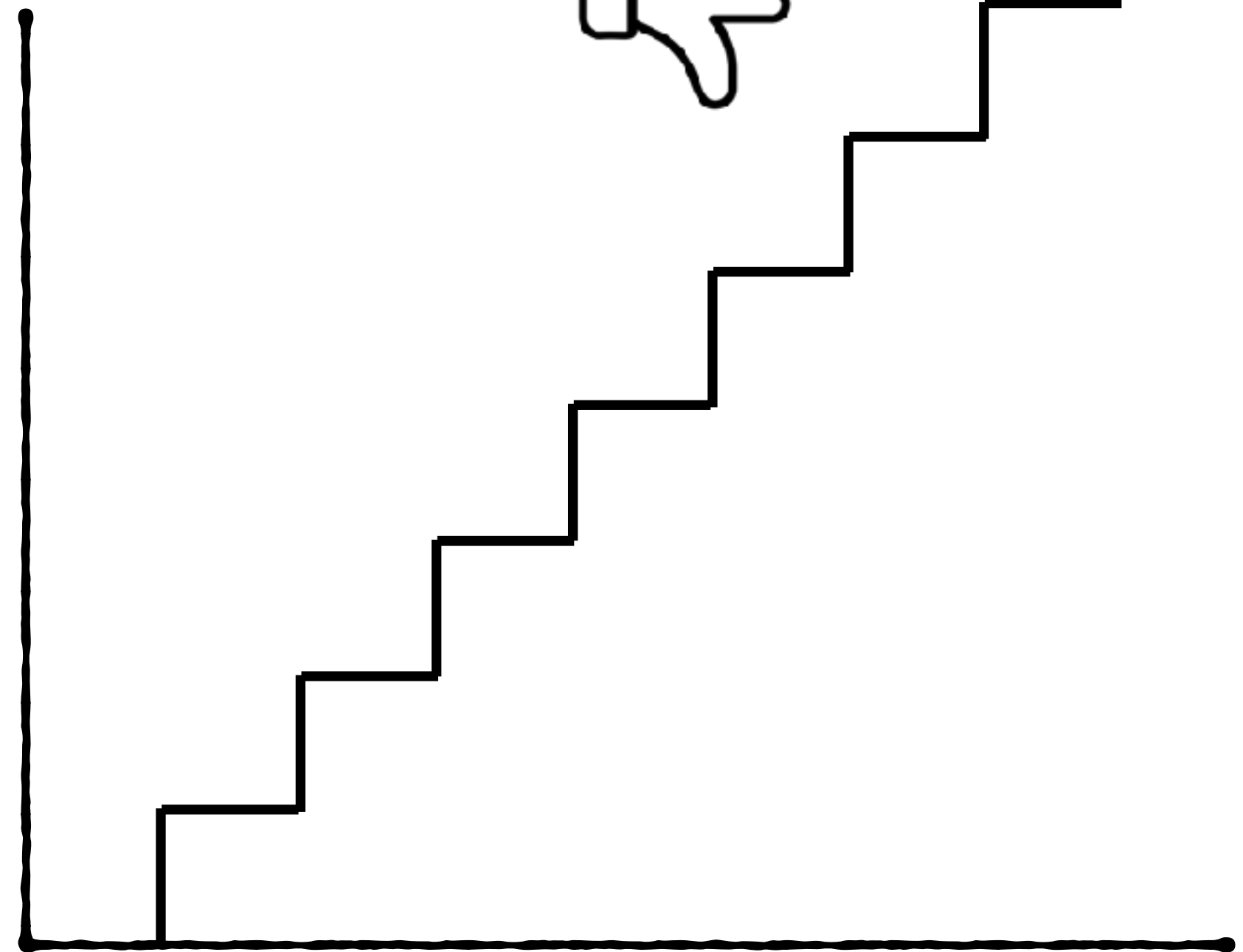
key space

FPR: $O(2^{-F})$ - a semi-robust guarantee

Smooth distributions

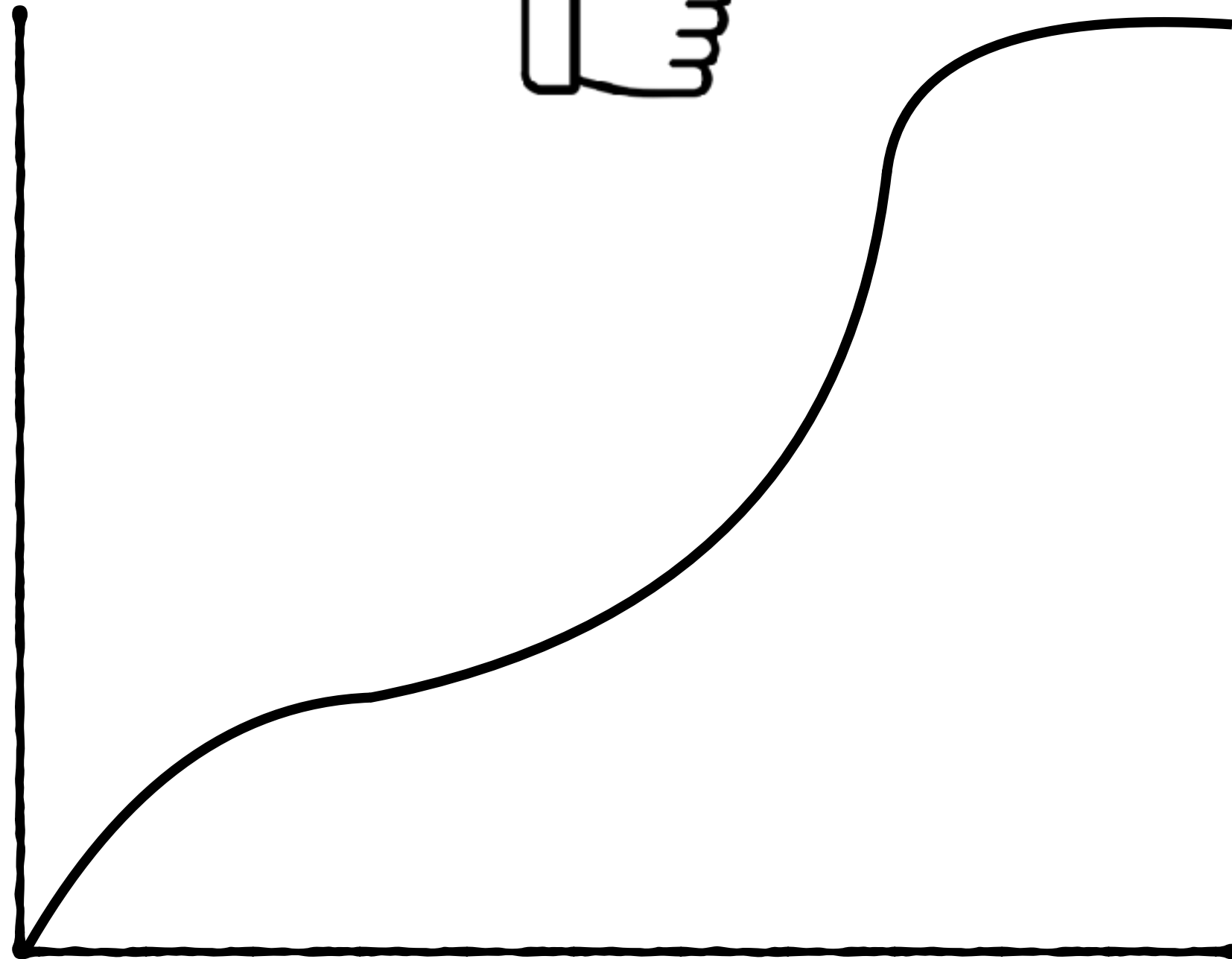


Jagged distributions

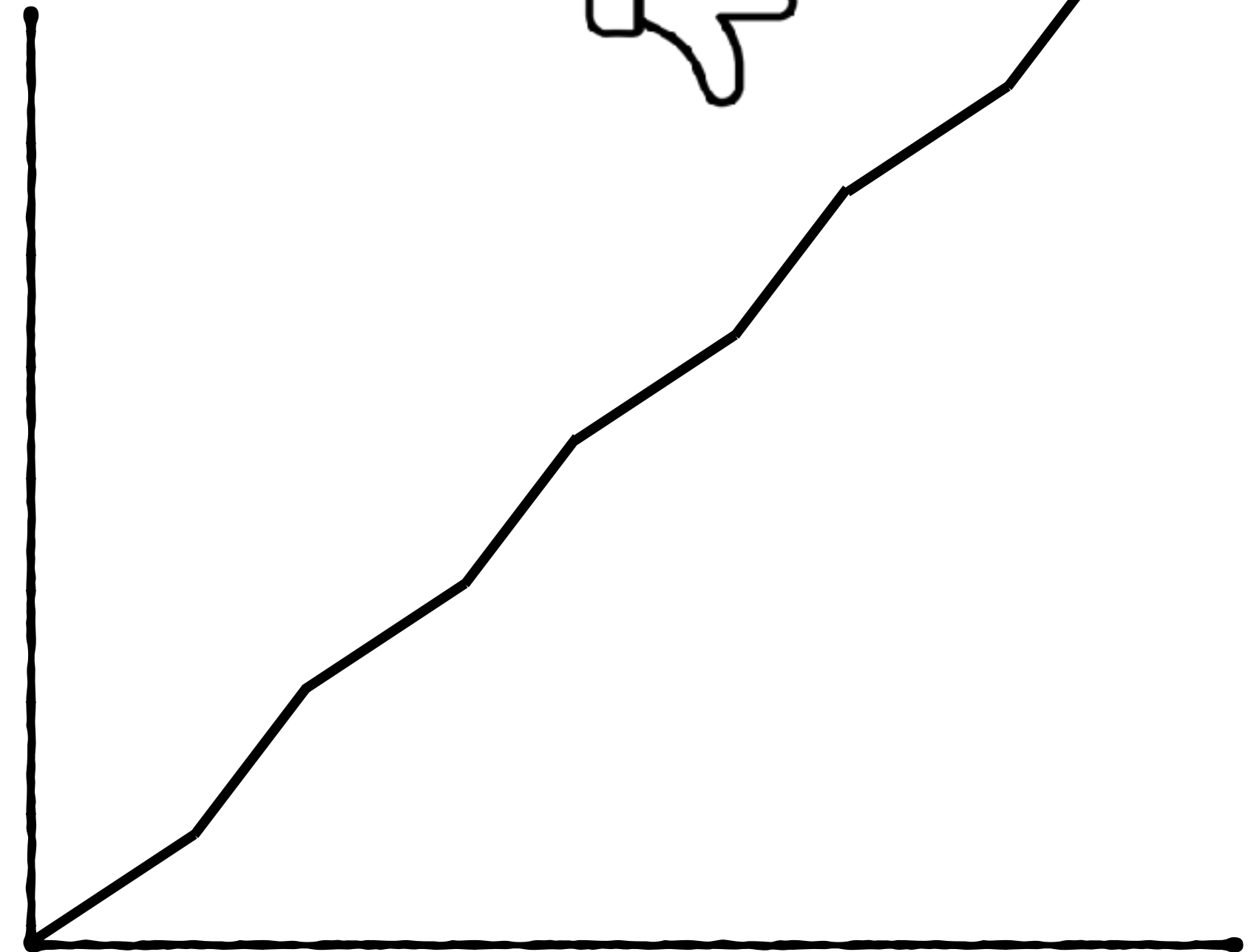


FPR: $O(2^{-F})$ - a semi-robust guarantee

Smooth distributions

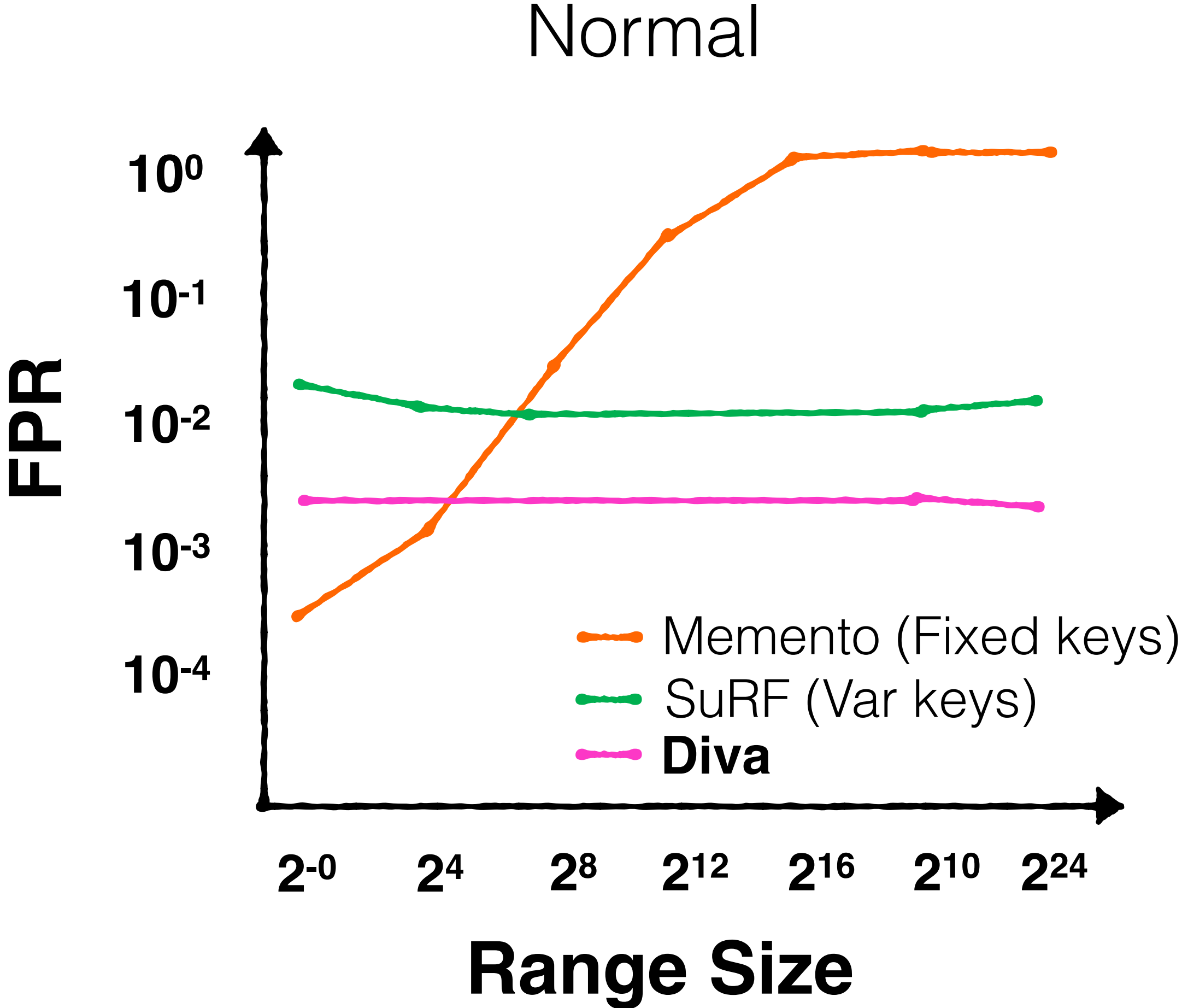


Jagged distributions

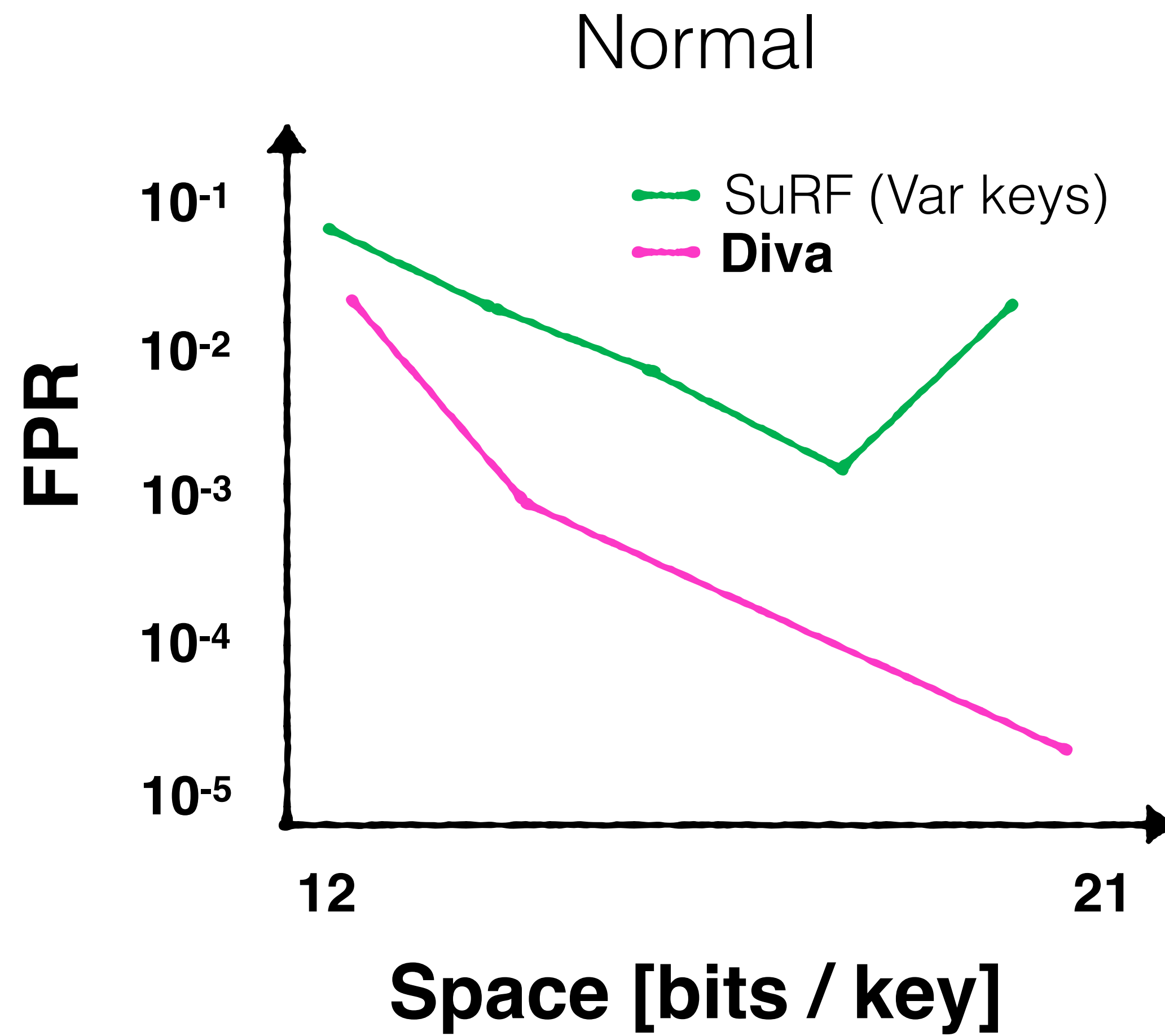


**Use entropy encoding to
smooth distribution**

Evaluation: Var-length queries



Evaluation: Var-length keys



More in the paper



Infix store details



Dynamicity



Theoretical analysis



Thank you